Архитектура компьютера

Подготовка к РК1 · темы 1–7 · подробный конспект: как решать, почему так, откуда берутся числа

Содержание

Тема	Содержание	Из твоих материалов
0	Как пользоваться; как читать формулировки заданий РК	
1	Системы счисления; целые: прямой и доп. код; разбор заданий с фото (варианты 2–4)	лаб. 1, фото
2	Вещественные числа Single/Double: откуда 127, 23, 52; перевод в обе стороны; ответы к вариантам	лаб. 2
3	Вентили, таблицы истинности, законы, упрощение, СДНФ/СКНФ, чтение схем	лаб. 3, 4
4	Карты Карно: идея, алгоритм, 5 разобранных примеров, таблицы лабораторной	лаб. 5, ДНФ/КНФ
5	Полусумматор, сумматор, умножитель 2×2 и 3×3, дешифратор, шифратор, MUX	лаб. 6, лекция 6
6	Триггеры RS, D, T, JK; временные диаграммы с таблицами событий	лаб. 7
7	Счётчики: асинхронный, синхронный, по модулю 10; ложные состояния	силабус, тема 7
8	Electronics Workbench: что и как делать	лаб. 3–7
9	Тренировка: 51 задач (Дано / Что сделать / Как записать ответ) с разбором	
	Шпаргалка на последней странице	

Как устроен этот PDF. Каждая тема написана одинаково: **1) зачем это нужно → 2) идея и почему она работает → 3) алгоритм по шагам → 4) разобранный пример, где у каждого числа указано, откуда оно взялось → 5) типичные ошибки**. В конце — тренировка: у каждой задачи написано, **что дано, что нужно сделать и как записать ответ**, а решение объясняет каждый шаг.

Как учиться за ограниченное время.

1. Прочитай «почему это работает» — после этого алгоритм не нужно зубрить, его можно вывести.
2. Закрой разобранный пример и повтори его на бумаге. Совпало — иди дальше. Не совпало — найди шаг, где разошлись.
3. Реши задачи тренировки **до** просмотра решений. Решения читай, чтобы сверить ход мысли, а не только ответ.
4. В конце пройди шпаргалку на последней странице вслух.

Как читать формулировки заданий РК

Что написано в условии	Что от тебя требуется
«Записать **прямой код** числа, интерпретируя его как 8-битное целое **без знака**»	Просто переведи число в двоичную систему и допиши нули слева, чтобы получилось ровно 8 цифр.
«Записать **дополнительный код** числа, интерпретируя его как 16-битное целое **со знаком**»	Если число ≥ 0 — двоичная запись из 16 цифр. Если отрицательное — модуль → инверсия → +1.
«Записать в десятичной системе число, если дан его **дополнительный код**»	Обратная задача: смотри на старший бит; если 1 — инвертируй, прибавь 1, переведи, поставь минус.
«Представить число в 16-й с.с. в виде **короткого** (длинного) вещественного»	Short = Single (32 бита, ответ — 8 hex-цифр). Long = Double (64 бита, ответ — 16 hex-цифр).
«Значение переменной A=430F0000$_{16}$, тип single. Найти десятичное значение»	Hex → биты → поля S, E, M → число (раздел 2.3).
«Построить таблицу истинности», «найти значения F»	Перебери все 2^n наборов входов, для каждого вычисли выход по формуле.
«Упростить выражение», «минимизировать картой Карно»	Законы алгебры логики (тема 3) или карта Карно (тема 4); в конце проверь по таблице.
«Что выводит схема», «какая схема правильная»	Прочитай схему слева направо, выпиши формулу, сравни таблицы на нескольких наборах.
«Нарисовать выходной сигнал Q(t) триггера»	Найди активные фронты C, смотри входы **до** фронта, по таблице триггера определи новое Q.
«Нарисовать схему счётчика / сумматора / дешифратора»	Таблица → формулы → вентили; все схемы этого типа разобраны в темах 5–7.

Откуда взято. Силабус (темы 1–7), твои файлы из папки «Материал АРХ» (лабораторные 2–7, лекции 6–7, ДНФ/КНФ, сумматоры и умножители) и задания с фото (прямой и дополнительный код, варианты 2–4). Для каждого типа задач дан алгоритм и несколько разобранных примеров.

Обозначения. Черта над буквой = инверсия (НЕ): $\overline{A}$. Точка или слитная запись = И (AND), «+» = ИЛИ (OR), «⊕» = исключающее ИЛИ (XOR). В файлах лабораторной знак ⊕ может отображаться как «Å» — это он и есть. Индекс внизу: 1011_2 — число в двоичной системе. Перенос и переполнение — не одно и то же, см. тему 1.

Быстрая проверка себя. Windows-калькулятор → режим «Программист»: вводишь число в DEC/HEX/BIN и сразу видишь его в других системах; переключатель BYTE / WORD задаёт 8 или 16 бит. Для схем — Logic Converter в Electronics Workbench (тема 8).

Тема 1 Системы счисления и целые числа в памяти

переводы 2/8/16/10 · беззнаковые и знаковые целые · прямой и дополнительный код · разбор заданий с фото

1.1. Что такое система счисления и зачем в компьютере двоичная

Система счисления — способ записывать числа цифрами. В **позиционной** системе значение цифры зависит от её места в числе (разряда). **Основание P** — сколько разных цифр в системе: десятичная P = 10 (цифры 0–9), двоичная P = 2 (0, 1), восьмеричная P = 8 (0–7), шестнадцатеричная P = 16 (0–9 и буквы A–F, где A = 10, B = 11, C = 12, D = 13, E = 14, F = 15).

Главная формула. значение числа = сумма «цифра × основание в степени номера разряда». Разряды нумеруются **справа налево с нуля**: $156 = 1\cdot10^2 + 5\cdot10^1 + 6\cdot10^0 = 100 + 50 + 6$. Точно так же для двоичной: $1011_2 = 1\cdot2^3 + 0\cdot2^2 + 1\cdot2^1 + 1\cdot2^0 = 8 + 0 + 2 + 1 = 11$. Меняется только основание.

Зачем двоичная? Электронный элемент (транзистор) надёжно различает только два состояния: «выключен» (0) и «включён» (1). Поэтому компьютер хранит всё только нулями и единицами. **Бит** — одна двоичная цифра; **байт** = 8 бит; **слово** часто 16 бит. Шестнадцатеричная система нужна как **короткая запись двоичной**: одна hex-цифра = ровно 4 бита (потому что $16 = 2^4$), значит байт = 2 hex-цифры. Например, $1011\ 1111_2 = BF_{16}$.

Таблица соответствия 0...15 (выучи — она нужна для быстрых переводов):

10	2	8	16	10	2	8	16
0	0000	0	0	8	1000	10	8
1	0001	1	1	9	1001	11	9
2	0010	2	2	10	1010	12	A
3	0011	3	3	11	1011	13	B
4	0100	4	4	12	1100	14	C
5	0101	5	5	13	1101	15	D
6	0110	6	6	14	1110	16	E
7	0111	7	7	15	1111	17	F

Степени двойки (выучи до 2^{16}; они нужны для переводов, диапазонов и IEEE):

65536	32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1
2^{16}	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

1.2. Перевод из десятичной в 2, 8, 16: целая часть

Дано: целое десятичное число (например, 156)
Что нужно сделать: записать его в системе с основанием P (2, 8 или 16)
Как записать ответ: число с индексом: 10011100_2

Почему это работает. любое число n можно записать как $n = P\cdot q + r$, где r — остаток от деления на P. Остаток r как раз и есть **последняя (самая младшая) цифра** числа в системе P: например, в десятичной 156 : 10 = 15 и остаток 6 — это последняя цифра. Частное q — это «число без последней цифры». Если разделить q ещё раз, получим предпоследнюю цифру, и так далее. Мы «откусываем» цифры справа налево, поэтому в записи результата их надо читать **снизу вверх**.

1. Раздели число на P нацело. Запиши **остаток** (он станет цифрой; 10...15 в hex — буквы A...F).
2. Частное раздели на P ещё раз, снова запиши остаток. Повторяй, пока частное не станет **0**.
3. Выпиши остатки **в обратном порядке** — последний остаток будет первой (старшей) цифрой.
4. Проверь: переведи ответ обратно в десятичную (раздел 1.4).

Пример: 156_{10} → 2, 16 и 8

В двоичную

156 : 2 = 78, остаток 0 → цифра **0**
78 : 2 = 39, остаток 0 → цифра **0**
39 : 2 = 19, остаток 1 → цифра **1**
19 : 2 = 9, остаток 1 → цифра **1**
9 : 2 = 4, остаток 1 → цифра **1**

В шестнадцатеричную

156 : 16 = 9, остаток 12 → цифра **C**
9 : 16 = 0, остаток 9 → цифра **9**

Читаем остатки **снизу вверх**: $156_{10} = 9C_{16}$

В восьмеричную

156 : 8 = 19, остаток 4 → цифра **4**

4 : 2 = 2, остаток 0 → цифра **0**
2 : 2 = 1, остаток 0 → цифра **0**
1 : 2 = 0, остаток 1 → цифра **1**

Читаем остатки **снизу вверх**: 156_{10} = **10011100_2**

19 : 8 = 2, остаток 3 → цифра **3**
2 : 8 = 0, остаток 2 → цифра **2**

Читаем остатки **снизу вверх**: 156_{10} = **234_8**

Итог: $156_{10} = 10011100_2 = 9C_{16} = 234_8$. Откуда число 9C: остаток 12 — это цифра C, а частное 9 — это цифра 9 (9 < 16, делить дальше нечего).

Быстрый путь: для больших чисел делить на 2 долго. Разделите на 16, получите hex (мало шагов), а потом каждую hex-цифру замените четырьмя битами: 9 = 1001, C = 1100 → **10011100**. Ответ тот же, а шагов меньше.

Запомни: в конце деления частное, которое меньше основания, **тоже записывается цифрой** (в примере 9). Частые ошибки: читать остатки сверху вниз, забыть последнюю цифру, не заменить остаток 10…15 буквой.

1.3. Перевод дробной части: умножаем на основание

Дано: правильная дробь, например 0,6875
Что нужно сделать: записать её в системе P
Как записать ответ: $0{,}1011_2$

Почему это работает. дробная часть x = $0{,}d_1d_2d_3\ldots$ в системе P. Умножим на P: цифры «сдвинутся» влево через запятую, и целая часть результата станет равна d_1 (первой цифре после запятой): в десятичной 0,375 × 10 = 3,75 — целая часть 3 это и есть первая цифра. Откидываем целую часть и умножаем остаток снова — получаем d_2, и так далее. Цифры идут **сверху вниз**.

1. Умножь дробную часть на основание P (для двоичной — на 2).
2. Целую часть результата запиши как следующую цифру (это 0 или 1 для P = 2).
3. Целую часть отбрось, а оставшуюся дробь умножай снова. Остановись, когда дробь стала **0** или когда набрано нужное число цифр.

Пример: $0{,}6875_{10}$ → двоичная

№	дробь × 2	= результат	цифра
1	0,6875 × 2	1,375	**1**
2	0,375 × 2	0,75	**0**
3	0,75 × 2	1,5	**1**
4	0,5 × 2	1	**1**

Цифры сверху вниз: **1011** → $0{,}6875_{10}$ = **$0{,}1011_2$**. Дробь стала равна 0 на четвёртом шаге, поэтому запись конечная.
Проверка: $0{,}1011_2$ = ½ + ⅛ + 1/16 = 0,5 + 0,125 + 0,0625 = 0,6875 ✓.

Почему 0,1 нельзя записать в двоичной точно

№	дробь × 2	= результат	цифра
1	0,1 × 2	0,2	**0**
2	0,2 × 2	0,4	**0**
3	0,4 × 2	0,8	**0**
4	0,8 × 2	1,6	**1**
5	0,6 × 2	1,2	**1**
6	0,2 × 2	0,4	**0**

№	дробь × 2	= результат	цифра
7	0,4 × 2	0,8	**0**
8	0,8 × 2	1,6	**1**
9	0,6 × 2	1,2	**1**
10	0,2 × 2	0,4	**0**
11	0,4 × 2	0,8	**0**
12	0,8 × 2	1,6	**1**

Цифры: **000110011001**… Остатки дроби идут по кругу: 0,2 → 0,4 → 0,8 → 0,6 → 0,2 → … (на шаге 5 дробь снова 0,2, как на шаге 1). Раз дробь повторилась, дальше всё будет повторяться — двоичная дробь **бесконечная периодическая** 0,0**0011**0011… Точно в конечное число бит она не помещается, поэтому в памяти (Single/Double) такие числа хранятся с **округлением**.

Пример с целой и дробной частью: $25{,}3125_{10}$

Шаг 1. Целую часть 25 переводим делением: 25 : 2 = 12 (ост. 1), 12 : 2 = 6 (ост. 0), 6 : 2 = 3 (ост. 0), 3 : 2 = 1 (ост. 1), 1 : 2 = 0 (ост. 1) → снизу вверх **11001**.

Шаг 2. Дробную часть 0,3125 переводим умножением:

№	дробь × 2	= результат	цифра
1	0,3125 × 2	0,625	**0**
2	0,625 × 2	1,25	**1**
3	0,25 × 2	0,5	**0**
4	0,5 × 2	1	**1**

Получили 0,0101.

Шаг 3. Склеиваем через запятую: $25{,}3125_{10}$ = **$11001{,}0101_2$**. Целая и дробная части **переводятся независимо**.

В hex и в восьмеричную (через двоичную, см. 1.5): 0001 1001 , 0101 → **$19{,}5_{16}$**; 011 001 , 010 100 → **$31{,}24_8$**.

1.4. Перевод из 2, 8, 16 в десятичную

Дано: число в системе P, например $10110{,}101_2$
Что нужно сделать: получить десятичное число
Как записать ответ: 22,625

Почему это работает. это та самая формула «цифра × основание^{разряд}». Разряды целой части считаем справа налево 0, 1, 2..., дробной — слева направо −1, −2, −3... (то есть 2^{-1} = ½, 2^{-2} = ¼, 2^{-3} = ⅛).

Пример: $10110{,}101_2$

Целая часть 10110 (вес разрядов — степени двойки справа налево):

вес	16	8	4	2	1
бит	**1**	0	**1**	**1**	0

Складываем веса там, где стоит 1: 16 + 4 + 2 = **22**.

Дробная часть 101 после запятой: $1{\cdot}2^{-1} + 0{\cdot}2^{-2} + 1{\cdot}2^{-3}$ = ½ + ⅛ = 0,5 + 0,125 = **0,625**.

Ответ: **$22{,}625_{10}$**.

Ещё два: $3F7_{16}$ и 745_8

$3F7_{16} = 3{\cdot}16^2 + F{\cdot}16 + 7 = 768 + 240 + 7$ = **1015** (буква F в числе — это 15).

$745_8 = 7{\cdot}8^2 + 4{\cdot}8 + 5 = 7{\cdot}64 + 32 + 5 = 448 + 32 + 5$ = **485**.

1.5. Быстрые переводы между 2, 8 и 16 (без десятичной)

Почему это работает. $16 = 2^4$, поэтому одна hex-цифра ровно соответствует четырём двоичным разрядам; $8 = 2^3$, поэтому одна восьмеричная цифра — трём. Значит, достаточно **разбить двоичную запись на группы** и заменить каждую группу цифрой. Никаких делений и умножений.

1. **2 → 16:** группы по **4 бита** (тетрады). **2 → 8:** группы по **3 бита** (триады).
2. Группируй **от запятой**: в целой части — справа налево, в дробной — слева направо.
3. Если в крайней группе не хватает цифр, **допиши нули**: слева у целой части, справа у дробной (на значение число это не влияет).
4. Каждую группу замени цифрой по таблице 0...15. **Обратно** (16 → 2, 8 → 2): каждую цифру замени её 4 (или 3) битами.
5. **8 ↔ 16**: через двоичную запись.

Пример: $1101011{,}0111_2$ → 16 и 8

	целая часть (справа налево)		дробная часть (слева направо)
Запись	1101011 → дописали 0 слева: 0110 1011		0111
Тетрады → hex	0110 = 6	1011 = B	0111 = 7
Триады: запись	1101011 → дописали 00 слева: 001 101 011		0111 → дописали 00 справа: 011 100
Триады → восьмеричная	1 · 5 · 3 → 153		3 · 4 → 34

Ответ: $1101011{,}0111_2$ = **6B,7**$_{16}$ = **153,34**$_8$.

Пример: 745_8 → 16 (через двоичную)

7 = 111, 4 = 100, 5 = 101 → 111100101_2. Перегруппируем по 4 справа налево: `1 1110 0101` → слева допишем 000: 0001 1110 0101 → 1 · E · 5 → **1E5**$_{16}$.

Осторожно: нули дописывают только так, чтобы число не изменилось: в целой части слева, в дробной справа. Дописать нули в другой стороне — ошибка (число станет другим).

1.6. Целые числа в памяти: беззнаковые

Ячейка памяти имеет **фиксированную длину N бит** (8, 16, 32). Из N бит можно составить ровно $\mathbf{2^N}$ разных комбинаций нулей и единиц: 8 бит — 256 комбинаций, 16 бит — 65 536. Значит, в ячейке можно хранить только 2^N различных чисел — поэтому диапазон **конечный**, и возможно **переполнение**.

Откуда формула V_max. у беззнакового числа все N бит — цифры. Наименьшее значение: все нули = 0. Наибольшее: все единицы = $2^{N-1} + \ldots + 2 + 1$. Чтобы увидеть сумму, прибавим 1: $11111111_2 + 1 = 100000000_2 = 2^8 = 256$, значит $11111111_2 = 256 - 1 =$ **255**. В общем виде $V_{max} = \mathbf{2^N - 1}$.

N бит	Комбинаций 2^N	Диапазон без знака	Тип C
8	256	0 … 255	unsigned char
16	65 536	0 … 65 535	unsigned short
32	4 294 967 296	0 … 4 294 967 295	unsigned int
N	2^N	0 … $2^N - 1$	—

Алгоритм: прямой код беззнакового целого

1. Проверь, что число вписывается в диапазон 0 … $2^N - 1$.
2. Переведи число в двоичную (для 16 бит удобно через hex, раздел 1.2).
3. Допиши нули **слева** до ровно N цифр (число не меняется, нули слева — незначащие).

Пример: 235 как 8-битное беззнаковое

Диапазон 0…255 — вписывается. Делим на 16 (быстрый путь через hex):

235 : 16 = 14, остаток 11 → цифра **B**

14 : 16 = 0, остаток 14 → цифра **E**

Читаем остатки **снизу вверх**: 235_{10} = **EB**$_{16}$

Остаток 11 — цифра B, частное 14 — цифра E, значит 235 = EB$_{16}$. E = 1110, B = 1011 → **11101011**. Всего 8 цифр, дописывать нули не нужно. Проверка: 128 + 64 + 32 + 8 + 2 + 1 = 235 ✓.

1.7. Знаковые целые: дополнительный код

Чтобы хранить и отрицательные числа, **половину комбинаций отдают под них**. Старший бит стал **знаковым**: 0 — число неотрицательное, 1 — отрицательное. Отрицательные числа записываются в **дополнительном коде**.

8-битный код	Значение (со знаком)
00000000	0
00000001	1
01111111	127 — наибольшее
10000000	−128 — наименьшее
10000001	−127
11111110	−2
11111111	−1

Запомни: диапазон со знаком: $\mathbf{-2^{N-1} \ldots 2^{N-1} - 1}$. Для 8 бит это −128 … 127, для 16 бит −32 768 … 32 767. Отрицательных чисел на одно больше, чем положительных: нуль «занял» одно место среди неотрицательных.

Откуда берётся «инверсия и +1»

Вывод. посмотрим на 8 бит. Если сложить любое число x и его **инверсию** (все биты наоборот), получится 11111111_2 = 255, потому что в каждом разряде одна цифра 0, а другая 1. Значит **инверсия x = 255 − x**. Прибавим 1: **инверсия + 1 = 256 − x**. А число 256 в 8 битах «не помещается» — старший бит (перенос) пропадает, и 256 ведёт себя как 0. Поэтому число 256 − x при сложении с x даёт 0 (как и должно быть для −x). Вывод: код 256 − x **и есть код числа −x**, и обычный сумматор складывает такие числа правильно.

Проверка на числе −5 (8 бит)

действие	результат	что это значит
модуль 5	00000101	5
инверсия	11111010	255 − 5 = 250
+1	**11111011**	251 = 256 − 5

Проверим: 5 + (−5) = 00000101 + 11111011 = 1 00000000 → перенос (9-й бит) отбрасываем → **00000000** = 0 ✓.

Алгоритм: дополнительный код целого со знаком

1. Проверь диапазон (для 8 бит −128…127, для 16 бит −32 768…32 767).
2. Число **≥ 0**: код = двоичная запись из N цифр (с нулями слева). Старший бит автоматически 0.
3. Число **< 0**: а) возьми модуль и запиши его в двоичной на N бит; б) **инвертируй** все биты (0 ↔ 1) — получится обратный код; в) **прибавь 1** — получится дополнительный код (старший бит 1).
4. Проверка: сумма код + код модуля должна дать 2^N (то есть 00…0 с переносом).

Запомни: быстрая альтернатива: код отрицательного числа = $\mathbf{2^N - |x|}$. Или идти справа налево: до первой единицы включительно цифры переписываем, остальные инвертируем.

Пример: −36 в 8 битах

Модуль 36 = 2·16 + 4 → hex 24 → 00100100. Инверсия: 11011011. Прибавляем 1: 11011011 + 1 = **11011100**.

Проверка 1: 2^8 − 36 = 220 = 11011100 ✓. Проверка 2 (правило «справа до первой единицы»): 00100100 — справа до первой 1 это «100»; переписываем «100», остальные 5 цифр 00100 → 11011 → 11011100 ✓.

Обратная задача: из дополнительного кода в десятичное

Почему это работает. если старший бит 0 — число неотрицательное, читаем как обычно. Если 1 — оно отрицательное, и код равен $2^N - |x|$. Чтобы вернуть модуль, нужно сделать то же самое ещё раз: **инвертировать и прибавить 1** (отрицание от отрицания даёт исходное число). Другой путь: значение = (код как беззнаковое) − 2^N.

1. Посмотри на **старший бит**.
2. Если 0: переведи в десятичную обычным способом (раздел 1.4). Готово.
3. Если 1: инвертируй все биты, прибавь 1, переведи полученное двоичное в десятичное — это модуль. Поставь перед ним **минус**.
4. Проверка: (беззнаковое значение кода) − 2^N должно дать то же число.

Пример: 11011100 (8 бит, со знаком)

Старший бит **1** → число отрицательное. Инверсия: 00100011. Прибавляем 1: 00100011 + 1 = 00100100. Переводим: 00100100_2 = 36. Ставим минус: **−36**.

Проверка: как беззнаковое 11011100 = 220; 220 − 256 = -36 ✓.

Вычитание через дополнительный код и переполнение

Почему это работает. вычитание заменяют сложением с отрицательным числом: A − B = A + (−B). Так в процессоре не нужна отдельная схема вычитания — достаточно сумматора и умения получать дополнительный код. Перенос из старшего разряда **отбрасывается**, потому что в N-битной ячейке он соответствует числу 2^N, а $2^N \equiv 0$ (мы работаем «по модулю 2^N»).

Пример из силабуса: 10 − 5 в 8-битном дополнительном коде

Шаг 1. 10 = A_{16} → 00001010. **Шаг 2.** Код (−5): модуль 00000101 → инверсия 11111010 → +1 → **11111011**. **Шаг 3.** Складываем столбиком:

```
  00001010   (+10)
+ 11111011   (−5)
= 1 00000101
```

Шаг 4. 9-й бит (перенос) = 1 — он не помещается в 8 бит и отбрасывается. Остаётся 00000101 = 5. ✓ Результат правильный (+5).

Значение переноса: здесь перенос 1 — нормальное явление и не означает ошибку. Он соответствует «лишнему» 256 (10 + 251 = 261 = 256 + 5).

Переполнение (overflow)

Беззнаковые, 4 бита: 1010 (10) + 0111 (7) = 10001. Пятый бит (перенос) в 4-битную ячейку не входит и отбрасывается → остаётся 0001 = **1** вместо 17. Ошибка: 17 mod 16 = 1. Для беззнаковых **перенос из старшего разряда = переполнение**.

Знаковые, 8 бит: 100 + 50 = 01100100 + 00110010 = 10010110 — старший бит стал 1, и код читается как -106, а не 150. **Правило:** переполнение знаковых — когда складываем два числа **одного знака**, а результат получился **другого знака**.

Осторожно: для знаковых чисел перенос из старшего бита и переполнение — **разные** явления (как в примере 10 − 5: перенос есть, переполнения нет). Для беззнаковых перенос = переполнение.

1.8. Разбор заданий с фото (варианты 2–4)

На фото видно задания **варианта 3 и варианта 4 целиком** и **пункт 5 варианта 2**. Все варианты устроены одинаково: 1) беззнаковое 8 бит; 2) знаковое 8 бит; 3) беззнаковое 16 бит; 4) знаковое 16 бит; 5) из дополнительного кода в десятичное. Остальные варианты решаются по тому же алгоритму. Ниже каждое число разобрано **до последней цифры**: что делим, откуда остатки, как получили код и как проверить.

Как оформить ответ на листке

Пиши коротко: **число → промежуточные шаги → ответ с группами по 4 бита**. Пример: «−36: модуль 00100100, инверсия 11011011, +1 → 1101 1100». Если просят «в 16-й системе», добавь hex-запись. Группы по 4 бита не обязательны, но с ними ты не собьёшься и преподавателю легче проверять.

Вариант 3

Задание 1. Записать прямой код числа, интерпретируя его как **восьмибитовое целое без знака** (то есть перевести в двоичную и дополнить нулями слева до 8 цифр).

191	Диапазон 0…255: 191 входит. Деление на 16: 191 : 16 = 11, остаток 15; 11 : 16 = 0, остаток 11. Читаем остатки снизу вверх: **BF_{16}**. Каждую цифру в 4 бита: B=1011 + F=1111 → **1011 1111**. Проверка: 128 + 32 + 16 + 8 + 4 + 2 + 1 = 191 ✓.	`1011 1111`
235	Диапазон 0…255: 235 входит. Деление на 16: 235 : 16 = 14, остаток 11; 14 : 16 = 0, остаток 14. Читаем остатки снизу вверх: **EB_{16}**. Каждую цифру в 4 бита: E=1110 + B=1011 → **1110 1011**. Проверка: 128 + 64 + 32 + 8 + 2 + 1 = 235 ✓.	`1110 1011`
201	Диапазон 0…255: 201 входит. Деление на 16: 201 : 16 = 12, остаток 9; 12 : 16 = 0, остаток 12. Читаем остатки снизу вверх: **$C9_{16}$**. Каждую цифру в 4 бита: C=1100 + 9=1001 → **1100 1001**. Проверка: 128 + 64 + 8 + 1 = 201 ✓.	`1100 1001`

Задание 2. Записать **дополнительный код** числа как **восьмибитового целого со знаком** (положительное — как есть; отрицательное — модуль → инверсия → +1).

61	Диапазон 0…255: 61 входит. Деление на 16: 61 : 16 = 3, остаток 13; 3 : 16 = 0, остаток 3. Читаем остатки снизу вверх: **$3D_{16}$**. Каждую цифру в 4 бита: 3=0011 + D=1101 → **0011 1101**. Проверка: 32 + 16 + 8 + 4 + 1 = 61 ✓. Число положительное, поэтому доп. код = прямой, старший бит 0.	`0011 1101`
-36	Диапазон -128…127: -36 входит. **Модуль** 36: 36 : 16 = 2, остаток 4; 2 : 16 = 0, остаток 2 → 24_{16} → 0010 0100. **Инверсия:** 1101 1011. **+1:** 1101 1011 + 1 = **1101 1100**. Проверка: 2^8 − 36 = 220 = DC_{16} = 1101 1100 ✓.	`1101 1100`

-71	Диапазон -128…127: -71 входит. **Модуль** 71: 71 : 16 = 4, остаток 7; 4 : 16 = 0, остаток 4 → 47_{16} → 0100 0111. **Инверсия:** 1011 1000. **+1:** 1011 1000 + 1 = **1011 1001**. Проверка: $2^8 - 71 = 185 = B9_{16}$ = 1011 1001 ✓.	`1011 1001`

Задание 3. То же, что в задании 1, но **шестнадцатибитовое без знака** (16 цифр).

30426	Диапазон 0…65535: 30426 входит. Деление на 16: 30426 : 16 = 1901, остаток 10; 1901 : 16 = 118, остаток 13; 118 : 16 = 7, остаток 6; 7 : 16 = 0, остаток 7. Читаем остатки снизу вверх: **$76DA_{16}$**. Каждую цифру в 4 бита: 7=0111 + 6=0110 + D=1101 + A=1010 → **0111 0110 1101 1010**.	`0111 0110 1101 1010`
25175	Диапазон 0…65535: 25175 входит. Деление на 16: 25175 : 16 = 1573, остаток 7; 1573 : 16 = 98, остаток 5; 98 : 16 = 6, остаток 2; 6 : 16 = 0, остаток 6. Читаем остатки снизу вверх: **6257_{16}**. Каждую цифру в 4 бита: 6=0110 + 2=0010 + 5=0101 + 7=0111 → **0110 0010 0101 0111**.	`0110 0010 0101 0111`

Задание 4. То же, что в задании 2, но **шестнадцатибитовое со знаком**.

31763	Диапазон 0…65535: 31763 входит. Деление на 16: 31763 : 16 = 1985, остаток 3; 1985 : 16 = 124, остаток 1; 124 : 16 = 7, остаток 12; 7 : 16 = 0, остаток 7. Читаем остатки снизу вверх: **$7C13_{16}$**. Каждую цифру в 4 бита: 7=0111 + C=1100 + 1=0001 + 3=0011 → **0111 1100 0001 0011**. Число положительное, поэтому доп. код = прямой, старший бит 0.	`0111 1100 0001 0011`
-25694	Диапазон -32768…32767: -25694 входит. **Модуль** 25694: 25694 : 16 = 1605, остаток 14; 1605 : 16 = 100, остаток 5; 100 : 16 = 6, остаток 4; 6 : 16 = 0, остаток 6 → $645E_{16}$ → 0110 0100 0101 1110. **Инверсия:** 1001 1011 1010 0001. **+1:** 1001 1011 1010 0001 + 1 = **1001 1011 1010 0010**. Проверка: $2^{16} - 25694 = 39842 = 9BA2_{16}$ = 1001 1011 1010 0010 ✓.	`1001 1011 1010 0010`

Задание 5. Записать в десятичной системе целое число, если дан его дополнительный код (обратная задача: определить знак по старшему биту и вычислить значение).

0100 1010 0011 0101	Группы по 4 бита: 0100 1010 0011 0101 → hex **4A35**. Старший бит 0 → число положительное. $4A35_{16} = 4 \cdot 16^3 + A \cdot 16^2 + 3 \cdot 16 + 5 = 16384 + 2560 + 48 + 5 =$ **18997**.	`18997`
1101 0101 0101 0110	Группы по 4 бита: 1101 0101 0101 0110 → hex D556. Старший бит **1** → число отрицательное. **Инверсия:** 0010 1010 1010 1001 (hex 2AA9). **+1:** 0010 1010 1010 1010 (hex 2AAA). $2AAA_{16} = 2 \cdot 16^3 + A \cdot 16^2 + A \cdot 16 + A = 8192 + 2560 + 160 + 10 =$ **10922**. Ставим минус: **−10922**. Проверка: 54614 − 65536 = -10922 ✓.	`-10922`

Вариант 4

Задание 1. Записать прямой код числа, интерпретируя его как **восьмибитовое целое без знака** (то есть перевести в двоичную и дополнить нулями слева до 8 цифр).

151	Диапазон 0…255: 151 входит. Деление на 16: 151 : 16 = 9, остаток 7; 9 : 16 = 0, остаток 9. Читаем остатки снизу вверх: **97_{16}**. Каждую цифру в 4 бита: 9=1001 + 7=0111 → **1001 0111**. Проверка: 128 + 16 + 4 + 2 + 1 = 151 ✓.	`1001 0111`
205	Диапазон 0…255: 205 входит. Деление на 16: 205 : 16 = 12, остаток 13; 12 : 16 = 0, остаток 12. Читаем остатки снизу вверх: **CD_{16}**. Каждую цифру в 4 бита: C=1100 + D=1101 → **1100 1101**. Проверка: 128 + 64 + 8 + 4 + 1 = 205 ✓.	`1100 1101`
163	Диапазон 0…255: 163 входит. Деление на 16: 163 : 16 = 10, остаток 3; 10 : 16 = 0, остаток 10. Читаем остатки снизу вверх: **$A3_{16}$**. Каждую цифру в 4 бита: A=1010 + 3=0011 → **1010 0011**. Проверка: 128 + 32 + 2 + 1 = 163 ✓.	`1010 0011`

Задание 2. Записать **дополнительный код** числа как **восьмибитового целого со знаком** (положительное — как есть; отрицательное — модуль → инверсия → +1).

113	Диапазон 0…255: 113 входит. Деление на 16: 113 : 16 = 7, остаток 1; 7 : 16 = 0, остаток 7. Читаем остатки снизу вверх: **71_{16}**. Каждую цифру в 4 бита: 7=0111 + 1=0001 → **0111 0001**. Проверка: 64 + 32 + 16 + 1 = 113 ✓. Число положительное, поэтому доп. код = прямой, старший бит 0.	`0111 0001`
-118	Диапазон -128…127: -118 входит. **Модуль** 118: 118 : 16 = 7, остаток 6; 7 : 16 = 0, остаток 7 → 76_{16} → 0111 0110. **Инверсия:** 1000 1001. **+1:** 1000 1001 + 1 = **1000 1010**. Проверка: $2^8 - 118 = 138 = 8A_{16}$ = 1000 1010 ✓.	`1000 1010`
-27	Диапазон -128…127: -27 входит. **Модуль** 27: 27 : 16 = 1, остаток 11; 1 : 16 = 0, остаток 1 → $1B_{16}$ → 0001 1011. **Инверсия:** 1110 0100. **+1:** 1110 0100 + 1 = **1110 0101**. Проверка: $2^8 - 27 = 229 = E5_{16}$ = 1110 0101 ✓.	`1110 0101`

Задание 3. То же, что в задании 1, но **шестнадцатибитовое без знака** (16 цифр).

23386	Диапазон 0…65535: 23386 входит. Деление на 16: 23386 : 16 = 1461, остаток 10; 1461 : 16 = 91, остаток 5; 91 : 16 = 5, остаток 11; 5 : 16 = 0, остаток 5. Читаем остатки снизу вверх: **5B5A**$_{16}$. Каждую цифру в 4 бита: 5=0101 + B=1011 + 5=0101 + A=1010 → **0101 1011 0101 1010**.	`0101 1011 0101 1010`
30977	Диапазон 0…65535: 30977 входит. Деление на 16: 30977 : 16 = 1936, остаток 1; 1936 : 16 = 121, остаток 0; 121 : 16 = 7, остаток 9; 7 : 16 = 0, остаток 7. Читаем остатки снизу вверх: **7901**$_{16}$. Каждую цифру в 4 бита: 7=0111 + 9=1001 + 0=0000 + 1=0001 → **0111 1001 0000 0001**.	`0111 1001 0000 0001`

Задание 4. То же, что в задании 2, но **шестнадцатибитовое со знаком**.

19890	Диапазон 0…65535: 19890 входит. Деление на 16: 19890 : 16 = 1243, остаток 2; 1243 : 16 = 77, остаток 11; 77 : 16 = 4, остаток 13; 4 : 16 = 0, остаток 4. Читаем остатки снизу вверх: **4DB2**$_{16}$. Каждую цифру в 4 бита: 4=0100 + D=1101 + B=1011 + 2=0010 → **0100 1101 1011 0010**. Число положительное, поэтому доп. код = прямой, старший бит 0.	`0100 1101 1011 0010`
-17862	Диапазон -32768…32767: -17862 входит. **Модуль** 17862: 17862 : 16 = 1116, остаток 6; 1116 : 16 = 69, остаток 12; 69 : 16 = 4, остаток 5; 4 : 16 = 0, остаток 4 → 45C6$_{16}$ → 0100 0101 1100 0110. **Инверсия:** 1011 1010 0011 1001. **+1:** 1011 1010 0011 1001 + 1 = **1011 1010 0011 1010**. Проверка: 2^{16} − 17862 = 47674 = BA3A$_{16}$ = 1011 1010 0011 1010 ✓.	`1011 1010 0011 1010`

Задание 5. Записать в десятичной системе целое число, если дан его дополнительный код (обратная задача: определить знак по старшему биту и вычислить значение).

0010 1100 1001 0011	Группы по 4 бита: 0010 1100 1001 0011 → hex **2C93**. Старший бит 0 → число положительное. 2C93$_{16}$ = 2·16^3 + C·16^2 + 9·16 + 3 = 8192 + 3072 + 144 + 3 = **11411**.	`11411`
1100 0011 1111 1111	Группы по 4 бита: 1100 0011 1111 1111 → hex C3FF. Старший бит **1** → число отрицательное. **Инверсия:** 0011 1100 0000 0000 (hex 3C00). **+1:** 0011 1100 0000 0001 (hex 3C01). 3C01$_{16}$ = 3·16^3 + C·16^2 + 0·16 + 1 = 12288 + 3072 + 0 + 1 = **15361**. Ставим минус: **−15361**. Проверка: 50175 − 65536 = -15361 ✓.	`-15361`

Вариант 2, задание 5 (виден на фото)

Задание 5. Записать в десятичной системе целое число, если дан его дополнительный код (обратная задача: определить знак по старшему биту и вычислить значение).

0001 1111 1011 1001	Группы по 4 бита: 0001 1111 1011 1001 → hex **1FB9**. Старший бит 0 → число положительное. 1FB9$_{16}$ = 1·16^3 + F·16^2 + B·16 + 9 = 4096 + 3840 + 176 + 9 = **8121**.	`8121`
1110 1100 0100 1111	Группы по 4 бита: 1110 1100 0100 1111 → hex EC4F. Старший бит **1** → число отрицательное. **Инверсия:** 0001 0011 1011 0000 (hex 13B0). **+1:** 0001 0011 1011 0001 (hex 13B1). 13B1$_{16}$ = 1·16^3 + 3·16^2 + B·16 + 1 = 4096 + 768 + 176 + 1 = **5041**. Ставим минус: **−5041**. Проверка: 60495 − 65536 = -5041 ✓.	`-5041`

Запомни: в 16-битных числах не считай по отдельным битам. Переведи число в hex (4 цифры), работай с hex, потом каждую hex-цифру замени 4 битами. Для проверки: перемножь и сложи по формуле hex → десятичное.

Проверяй диапазон: прочитай условие: «без знака» или «со знаком». 200 как беззнаковое 8-битное = 11001000, а как знаковое 8-битное число 200 вообще не существует (диапазон −128…127). Для знакового выясни диапазон **до** решения.

Тема 2 Вещественные числа: Single и Double (IEEE 754)

как число хранится в 32/64 битах · число → hex и hex → число · откуда 127, 23, 8…

2.1. Зачем нужен особый формат и как он устроен

Целые числа в памяти — это просто двоичная запись. Но как хранить 3,75 или 0,000125 или $6 \cdot 10^{23}$? Нужна запись, в которой **положение запятой хранится отдельно**. Это та же «научная запись», которую вы знаете из школы: 12 345 = $1{,}2345 \cdot 10^4$ (значащие цифры 1,2345 и порядок 4). В компьютере то же самое, но в двоичной системе: число = $\pm 1{,}M \cdot 2^p$.

В память записываются три поля: **S** (знак, 1 бит), **E** (порядок, «характеристика», хранится в виде *p + смещение*), **M** (мантисса — цифры после запятой). Единица перед запятой у нормализованного числа всегда есть, поэтому её **не хранят** («скрытый бит») — так экономится бит.

Формат	Всего бит	S	E	M	Смещение	hex-цифр	Как выглядит
Single (короткое, *float*)	32	1	8	23	127	8	S | EEEEEEEE | MMMMMMMMMMMMMMMMMMMMMMM
Double (длинное, *double*)	64	1	11	52	1023	16	S | 11 бит E | 52 бита M

Откуда берутся числа 127, 1023, 23, 52. Откуда 127 и 1023: порядок p может быть отрицательным (число меньше 1), а в поле E хранят только неотрицательное число. Поэтому к p прибавляют смещение, равное половине диапазона поля: для 8 бит это $2^7 - 1$ = **127**, для 11 бит $2^{10} - 1$ = **1023**. Тогда E = p + 127 всегда ≥ 0, и сравнивать вещественные числа можно как целые. **Откуда 1 + 8 + 23 = 32:** формат подогнан под 4 байта; 23 бита мантиссы + скрытая единица = 24 значащие двоичные цифры ≈ 7 десятичных знаков точности.

Диапазон порядка (Single): зарезервированы E = 0 (ноль, денормализованные) и E = 255 (∞ и NaN), поэтому для обычных чисел E = 1…254, то есть p = E − 127 = **−126 … +127**. Для Double p = −1022 … +1023.

Что хранится	S	E (Single)	M
+0 и −0	0 / 1	00000000	все нули
±∞ (бесконечность)	0 / 1	11111111	все нули
NaN (не число)	любой	11111111	не все нули

Запомни: в ответе Single записывают **8 hex-цифр**, Double — **16 hex-цифр**. Если получилось меньше — не забыли про ведущие нули; если больше — ошиблись в числе бит.

2.2. Алгоритм: десятичное число → hex (по шагам, с пояснением откуда что берётся)

Дано: десятичное число (например, 21,625), указан формат Single или Double
Что нужно сделать: записать его машинный код в шестнадцатеричной системе
Как записать ответ: 8 hex-цифр для Single (например, 41AD0000) или 16 для Double

1. **Знак.** «+» → S = 0, «−» → S = 1. Дальше работаем с модулем числа.
2. **В двоичную.** Целую часть — делением на 2, дробную — умножением на 2 (тема 1). Склей через запятую.
3. **Нормализуй.** Сдвинь запятую так, чтобы слева осталась одна «1». Сколько разрядов сдвинули — это порядок p (влево → плюс, вправо → минус).
4. **Порядок.** E = p + 127 (Single) или p + 1023 (Double); запиши в двоичной на 8 или 11 бит.
5. **Мантисса.** Всё, что после запятой (без скрытой единицы), добавь нулями справа до 23 или 52 бит. Если цифр больше — округли.
6. **Собери и переведи в hex.** S | E | M → группы по 4 бита → hex-цифры.

Пример 1 (из силабуса). 21,625 → Single

Шаг 1. Знак

Число положительное, поэтому бит знака S = **0** (0 = плюс, 1 = минус). Дальше работаем с модулем **21,625**.

Шаг 2. Переводим модуль в двоичную систему

Целая часть 21 (делением на 2 или 16):

21 : 2 = 10, остаток 1 → цифра **1**

10 : 2 = 5, остаток 0 → цифра **0**

5 : 2 = 2, остаток 1 → цифра **1**

2 : 2 = 1, остаток 0 → цифра **0**

1 : 2 = 0, остаток 1 → цифра **1**

Читаем остатки **снизу вверх**: 21_{10} = **10101_2**

Дробная часть 0,625 умножаем на 2 и записываем целую часть результата (так мы выделяем двоичные цифры по одной):

№	дробь × 2	= результат	цифра
1	0,625 × 2	1,25	**1**
2	0,25 × 2	0,5	**0**
3	0,5 × 2	1	**1**

Дробь стала 0 — запись закончилась: 0,**101**.

Склеиваем: $21{,}625_{10}$ = **$10101{,}101_2$**.

Шаг 3. Нормализация (получаем вид 1,M × 2^p)

Здесь запятая стоит после 5-й цифры; чтобы слева осталась одна «1», сдвигаем её влево на 4 разряда → порядок p = +4. Получили **$1{,}0101101 \times 2^4$**.

Почему так: сдвиг запятой на 1 разряд влево — это деление на 2, поэтому, чтобы число не изменилось, к порядку прибавляем 1. Сдвиг вправо — наоборот, порядок уменьшается.

Шаг 4. Порядок со смещением

E = p + 127 = 4 + 127 = **131**. Откуда 127: смещение = $2^{8-1} - 1 = 127$. Двоичная запись E на 8 бит: 131 = 128 + 2 + 1 → `10000011`.

Шаг 5. Мантисса M

Берём всё, что стоит **после запятой** в нормализованной записи (единицу перед запятой **не пишем** — это скрытый бит): `0101101`. До 23 бит дописываем нули справа: `01011010000000000000000` (нули справа в дробной части не меняют значения).

Шаг 6. Собираем и переводим в hex

Склеиваем поля S | E | M: `0 | 10000011 | 01011010000000000000000`. Длина кода 32 бита.

группа из 4 бит	`0100`	`0001`	`1010`	`1101`	`0000`	`0000`	`0000`	`0000`
hex-цифра	**4**	**1**	**A**	**D**	**0**	**0**	**0**	**0**

Ответ: $41AD0000_{16}$ (для проверки подставьте результат в алгоритм из раздела 2.4 — получится исходное число).

Пример 2. −0,15625 → Single (число меньше 1: порядок отрицательный)

Шаг 1. Знак

Число отрицательное, поэтому бит знака S = **1** (0 = плюс, 1 = минус). Дальше работаем с модулем **0,15625**.

Шаг 2. Переводим модуль в двоичную систему

Целая часть 0 → в двоичной 0.

Дробная часть 0,15625 умножаем на 2 и записываем целую часть результата (так мы выделяем двоичные цифры по одной):

№	дробь × 2	= результат	цифра
1	0,15625 × 2	0,3125	**0**
2	0,3125 × 2	0,625	**0**
3	0,625 × 2	1,25	**1**
4	0,25 × 2	0,5	**0**
5	0,5 × 2	1	**1**

Дробь стала 0 — запись закончилась: 0,**00101**.

Склеиваем: $0{,}15625_{10}$ = **$0{,}00101_2$**.

Шаг 3. Нормализация (получаем вид 1,M × 2^p)

Здесь после запятой 2 нуля и потом первая «1»; сдвигаем запятую вправо на 3 → порядок p = −3. Получили **1,01 × 2^{-3}**.

Почему так: сдвиг запятой на 1 разряд влево — это деление на 2, поэтому, чтобы число не изменилось, к порядку прибавляем 1. Сдвиг вправо — наоборот, порядок уменьшается.

Шаг 4. Порядок со смещением

E = p + 127 = −3 + 127 = **124**. Откуда 127: смещение = $2^{8-1} - 1 = 127$. Двоичная запись E на 8 бит: 124 = 64 + 32 + 16 + 8 + 4 → `01111100`.

Шаг 5. Мантисса M

Берём всё, что стоит **после запятой** в нормализованной записи (единицу перед запятой **не пишем** — это скрытый бит): `01`. До 23 бит дописываем нули справа: `01000000000000000000000` (нули справа в дробной части не меняют значения).

Шаг 6. Собираем и переводим в hex

Склеиваем поля S | E | M: `1 | 01111100 | 01000000000000000000000`. Длина кода 32 бита.

группа из 4 бит	`1011`	`1110`	`0010`	`0000`	`0000`	`0000`	`0000`	`0000`
hex-цифра	**B**	**E**	**2**	**0**	**0**	**0**	**0**	**0**

Ответ: $BE200000_{16}$ (для проверки подставьте результат в алгоритм из раздела 2.4 — получится исходное число).

Что нового: дробь начинается с нулей (0,00101), поэтому запятую пришлось сдвигать **вправо**, и порядок вышел отрицательным. E = −3 + 127 = 124 — число положительное, как и положено для поля E.

Пример 3. −312,3125 → Double (пример из лабораторной)

Шаг 1. Знак

Число отрицательное, поэтому бит знака S = **1** (0 = плюс, 1 = минус). Дальше работаем с модулем **312,3125**.

Шаг 2. Переводим модуль в двоичную систему

Целая часть 312 (делением на 2 или 16):

312 : 16 = 19, остаток 8 → цифра **8**

19 : 16 = 1, остаток 3 → цифра **3**

1 : 16 = 0, остаток 1 → цифра **1**

Читаем остатки **снизу вверх**: 312_{10} = **138_{16}**

Это hex **138**; каждая цифра → 4 бита: 1=0001 3=0011 8=1000 → **100111000** (ведущие нули не пишем).

Дробная часть 0,3125 умножаем на 2 и записываем целую часть результата (так мы выделяем двоичные цифры по одной):

№	дробь × 2	= результат	цифра
1	0,3125 × 2	0,625	**0**
2	0,625 × 2	1,25	**1**
3	0,25 × 2	0,5	**0**
4	0,5 × 2	1	**1**

Дробь стала 0 — запись закончилась: 0,**0101**.

Склеиваем: $312{,}3125_{10}$ = **$100111000{,}0101_2$**.

Шаг 3. Нормализация (получаем вид 1,M × 2^p)

Здесь запятая стоит после 9-й цифры; чтобы слева осталась одна «1», сдвигаем её влево на 8 разрядов → порядок p = +8. Получили **1,001110000101 × 2^8**.

Почему так: сдвиг запятой на 1 разряд влево — это деление на 2, поэтому, чтобы число не изменилось, к порядку прибавляем 1. Сдвиг вправо — наоборот, порядок уменьшается.

Шаг 4. Порядок со смещением

E = p + 1023 = 8 + 1023 = **1031**. Откуда 1023: смещение = $2^{11-1} - 1 = 1023$. Двоичная запись E на 11 бит: 1031 = 1024 + 4 + 2 + 1 → `10000000111`.

Шаг 5. Мантисса M

Берём всё, что стоит **после запятой** в нормализованной записи (единицу перед запятой **не пишем** — это скрытый бит): `001110000101`. До 52 бит дописываем нули справа: `0011100001010000000000000000000000000000000000000000000` (нули справа в дробной части не меняют значения).

Шаг 6. Собираем и переводим в hex

Склеиваем поля S | E | M: `1 | 10000000111 | 0011100001010000000000000000000000000000000000000000000`. Длина кода 64 бита.

группа из 4 бит	1100	0000	0111	0011	1000	0101	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
hex-цифра	**C**	**0**	**7**	**3**	**8**	**5**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**

Ответ: C073850000000000$_{16}$ (для проверки подставьте результат в алгоритм из раздела 2.4 — получится исходное число).

Пример 4. 0,1 → Single: бесконечная дробь и округление

Шаг 1. Знак

Число положительное, поэтому бит знака S = **0** (0 = плюс, 1 = минус). Дальше работаем с модулем **0,1**.

Шаг 2. Переводим модуль в двоичную систему

Целая часть 0 → в двоичной 0.

Дробная часть 0,1 умножаем на 2 и записываем целую часть результата (так мы выделяем двоичные цифры по одной):

№	дробь × 2	= результат	цифра
1	0,1 × 2	0,2	**0**
2	0,2 × 2	0,4	**0**
3	0,4 × 2	0,8	**0**
4	0,8 × 2	1,6	**1**

№	дробь × 2	= результат	цифра
5	0,6 × 2	1,2	**1**
6	0,2 × 2	0,4	**0**
7	0,4 × 2	0,8	**0**
8	0,8 × 2	1,6	**1**

Первые 8 цифр: **00011001**. Продолжая так же, получаем длинную запись: **0001100110011001100110011001100110011001100110011001...** (для Single нужно примерно 25–30 цифр, для Double — 55–60; такую дробь считают в Windows-калькуляторе или программой, а на экзамене чаще дают числа с конечной двоичной дробью).

Склеиваем: $0{,}1_{10}$ = **0,0001100110011001100110011001...**$_2$.

Шаг 3. Нормализация (получаем вид 1,M × 2^p)

Здесь после запятой 3 нуля и потом первая «1»; сдвигаем запятую вправо на 4 → порядок p = −4. Получили **1,10011001100110011001100...** × 2^{-4}.

Почему так: сдвиг запятой на 1 разряд влево — это деление на 2, поэтому, чтобы число не изменилось, к порядку прибавляем 1. Сдвиг вправо — наоборот, порядок уменьшается.

Шаг 4. Порядок со смещением

E = p + 127 = −4 + 127 = **123**. Откуда 127: смещение = $2^{8-1} - 1 = 127$. Двоичная запись E на 8 бит: 123 = 64 + 32 + 16 + 8 + 2 + 1 → `01111011`.

Шаг 5. Мантисса M

Берём всё, что стоит **после запятой** в нормализованной записи (единицу перед запятой **не пишем** — это скрытый бит): цифр больше, чем 23. Оставляем первые 23: `10011001100110011001100`. Следующие за ними цифры: `11001 1...`. Первая из отброшенных цифр — 1 (и дальше не одни нули), значит остаток больше половины младшего бита → **округляем вверх**: прибавляем 1 к последнему биту. Итог M = `10011001100110011001101`.

Шаг 6. Собираем и переводим в hex

Склеиваем поля S | E | M: `0 | 01111011 | 10011001100110011001101`. Длина кода 32 бита.

группа из 4 бит	0011	1101	1100	1100	1100	1100	1100	1101
hex-цифра	**3**	**D**	**C**	**C**	**C**	**C**	**C**	**D**

Ответ: 3DCCCCCD$_{16}$ (для проверки подставьте результат в алгоритм из раздела 2.4 — получится исходное число).

Вывод: число 0,1 в двоичной системе не записывается точно, поэтому в памяти хранится **приближение**. Обратный перевод даст 0,100000001490116..., а не ровно 0,1. Это нормально и объясняет известные «ошибки округления» 0,1 + 0,2 ≠ 0,3.

Осторожно: если в задании дробная часть не заканчивается (как у 0,1), последние бит/hex-цифры зависят от округления. Считай дробь минимум до 25 бит (Single) или 55 бит (Double) и смотри на первую отброшенную цифру.

2.3. Алгоритм: hex → десятичное число

Дано: код числа в hex (например, 430F0000) и формат (Single/Double)
Что нужно сделать: найти десятичное значение
Как записать ответ: число со знаком, например 143 или −49

1. Каждую hex-цифру замени 4 битами (таблица 0...15).
2. Раздели на поля: Single 1 | 8 | 23, Double 1 | 11 | 52.
3. S → знак. E → десятичное число; p = E − 127 (или 1023) — **вычитаем** смещение.
4. Запиши 1,M × 2^p (**верни скрытую «1»**), сдвинь запятую на p разрядов (p > 0 — вправо, p < 0 — влево).
5. Переведи полученное двоичное число в десятичное по весам разрядов, поставь знак.

Пример 1 (контрольное задание). A = $430F0000_{16}$, тип Single

Шаг 1. Hex → биты

Каждую hex-цифру записываем четырьмя битами: 4=0100 3=0011 0=0000 F=1111 0=0000 0=0000 0=0000 0=0000

Все 32 бита: `0100 0011 0000 1111 0000 0000 0000 0000`

Шаг 2. Режем на поля

Single: 1 + 8 + 23 бит. S = `0`, E = `10000110`, M = `00011110000000000000000`.

Шаг 3. Знак и порядок

S = 0 → число **положительное**. E = 10000110_2 = 134. Настоящий порядок p = E − 127 = 134 − 127 = **7** (вычитаем смещение, которое прибавляли при записи).

Шаг 4. Возвращаем скрытую единицу

Число = 1,M × 2^p = 1,0001111 × 2^7; запятую сдвигаем вправо на 7 разрядов: **10001111_2**.

Шаг 5. В десятичную систему

Складываем веса разрядов, где стоит 1: 128 + 8 + 4 + 2 + 1 = **143**. С учётом знака: **143**.

Ответ: 143.

Пример 2 (контрольное задание). A = $C2440000_{16}$, тип Single

Шаг 1. Hex → биты

Каждую hex-цифру записываем четырьмя битами: C=1100 2=0010 4=0100 4=0100 0=0000 0=0000 0=0000 0=0000

Все 32 бита: `1100 0010 0100 0100 0000 0000 0000 0000`

Шаг 2. Режем на поля

Single: 1 + 8 + 23 бит. S = `1`, E = `10000100`, M = `10001000000000000000000`.

Шаг 3. Знак и порядок

S = 1 → число **отрицательное**. E = 10000100_2 = 132. Настоящий порядок p = E − 127 = 132 − 127 = **5** (вычитаем смещение, которое прибавляли при записи).

Шаг 4. Возвращаем скрытую единицу

Число = 1,M × 2^p = 1,10001 × 2^5; запятую сдвигаем вправо на 5 разрядов: **110001_2**.

Шаг 5. В десятичную систему

Складываем веса разрядов, где стоит 1: 32 + 16 + 1 = **49**. С учётом знака: **−49**.

Ответ: −49. Первая цифра C = 1100 — старший бит 1 ⇒ число отрицательное.

Пример 3 (лабораторная). A = 3FEC600000000000$_{16}$, тип Double

Шаг 1. Hex → биты

Каждую hex-цифру записываем четырьмя битами: 3=0011 F=1111 E=1110 C=1100 6=0110 0=0000 0=0000 0=0000 …

Все 64 бита: `0011 1111 1110 1100 0110 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000`

Шаг 2. Режем на поля

Double: 1 + 11 + 52 бит. S = `0`, E = `01111111110`, M = `1000110000000000000000000000000000000000000000000000`.

Шаг 3. Знак и порядок

S = 0 → число **положительное**. E = 01111111110$_2$ = 1022. Настоящий порядок p = E − 1023 = 1022 − 1023 = **−1** (вычитаем смещение, которое прибавляли при записи).

Шаг 4. Возвращаем скрытую единицу

Число = 1,M × 2^p = 1,1100011 × 2^{-1}; запятую сдвигаем влево на 1 разряд: **0,11100011$_2$**.

Шаг 5. В десятичную систему

Складываем веса разрядов, где стоит 1: $2^{-1} + 2^{-2} + 2^{-3} + 2^{-7} + 2^{-8}$ = **0,88671875**. С учётом знака: **0,88671875**.

Ответ: 0,88671875. p получился отрицательным (E < 1023) — число меньше 1.

Запомни: быстрая прикидка по первым hex-цифрам Single: **3F8…** ≈ 1; **40…** — от 2 до 8; **C0…** — отрицательные от −2 до −8; **00000000** — ноль. Если E = 0 или E = 255 — особые значения.

2.4. Типичные ошибки

Ошибка	Как её избежать
Записали в мантиссу единицу перед запятой	Скрытая «1» в M не хранится. M — только цифры после запятой.
Забыли прибавить смещение (или прибавили не то)	Single — 127, Double — 1023. E = p + смещение; обратно — вычитание.
Перепутали направление сдвига запятой	Запятая сдвинулась влево → p положительное; вправо → p отрицательное (число < 1).
Мантисса короче 23 (52) бит	Дописываем нули справа до нужной длины.
Неверно разбили на hex-цифры	Группы по 4 бита считаем слева направо от знакового бита.

2.5. Ответы к вариантам лабораторной №2 (все 10 вариантов)

Дано: три числа для Single и три для Double (таблица вариантов)
Что нужно сделать: записать каждое число как Single и как Double в hex
Как записать ответ: 8 и 16 hex-цифр соответственно

Условие в лабораторной: для своего варианта записать три числа в формате Single и три числа в формате Double. Ниже сверенные ответы (формулу и шаги см. раздел 2.2).

Single, hex

Вар.	а	б	в
1	-12.365 `C145D70A`	0.0123 `3C4985F0`	-10.698 `C12B2B02`
2	-29.0065 `C1E80D50`	-0.0301 `BCF69446`	10.695 `412B1EB8`
3	-29.0135 `C1E81BA6`	-0.0221 `BCB50B0F`	16.065 `4180851F`
4	12.003 `41400C4A`	0.001 `3A83126F`	-22.99 `C1B7EB85`
5	-120.01 `C2F0051F`	0.00001 `3727C5AC`	2.001 `40001062`
6	-101.001 `C2CA0083`	0.0005 `3A03126F`	-20.9 `C1A73333`
7	111.1 `42DE3333`	-0.0001 `B8D1B717`	-9.9 `C11E6666`

8	-0.021 `BCAC0831`	1.01 `3F8147AE`	-3.001 `C0401062`
9	12.225 `4143999A`	-0.09 `BDB851EC`	125.125 `42FA4000`
10	1.0225 `3F82E148`	-0.129 `BE041893`	0.005 `3BA3D70A`

Double, hex

Вар.	а	б	в
1	302.125 `4072E20000000000`	-0.001 `BF50624DD2F1A9FC`	-100.001 `C0590010624DD2F2`
2	128.125 `4060040000000000`	0.0021 `3F613404EA4A8C15`	-512.125 `C080010000000000`
3	-28.125 `C03C200000000000`	-0.0021 `BF613404EA4A8C15`	52.025 `404A033333333333`
4	20.125 `4034200000000000`	0.0125 `3F8999999999999A`	20.02 `4034051EB851EB85`
5	50.05 `4049066666666666`	-25.025 `C039066666666666`	12.003 `40280189374BC6A8`
6	51.001 `40498020C49BA5E3`	-128.05 `C06001999999999A`	-120.1 `C05E066666666666`
7	1024.1 `4090006666666666`	-0.001 `BF50624DD2F1A9FC`	-10.166 `C02454FDF3B645A2`
8	-10.12 `C0243D70A3D70A3D`	-0.125 `BFC0000000000000`	550.66 `40813547AE147AE1`
9	10.12 `40243D70A3D70A3D`	-0.009 `BF826E978D4FDF3B`	513.0025 `408008051EB851EC`
10	110.12 `405B87AE147AE148`	-10.009 `C024049BA5E353F8`	-513.0025 `C08008051EB851EC`

Тема 3 Логические основы компьютера: вентили, таблицы истинности, булева алгебра

AND, OR, NOT, NAND, NOR, XOR · законы · упрощение · построение схем

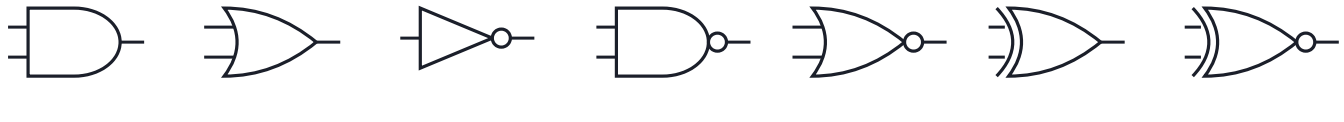
3.0. Сигнал 0/1, логическая функция, комбинационная схема

В цифровой схеме по проводам идут сигналы двух уровней: низкий (**0**, «ложь») и высокий (**1**, «истина»). **Логическая функция** F(A, B, …) — правило, которое для каждой комбинации входов говорит, будет на выходе 0 или 1. Схема, у которой выход зависит **только от текущих входов** (сумматор, дешифратор), называется **комбинационной**; схема с памятью (триггер, счётчик) — **последовательностная** (темы 6–7).

Любую функцию можно задать тремя способами, и в задачах РК нужно уметь переходить от одного к другому: **таблица истинности ↔ формула ↔ схема из вентилей**.

3.1. Вентили и таблицы истинности

Вентиль (логический элемент) — часть схемы, реализующая логическую операцию над сигналами 0/1 (низкое/высокое напряжение). Из вентилей строятся процессор и память. Таблица истинности перечисляет выход для **всех** комбинаций входов: для n входов будет 2^n строк (2 входа — 4, 3 входа — 8, 4 входа — 16).



И (AND)	ИЛИ (OR)	НЕ (NOT)	И-НЕ (NAND)	ИЛИ-НЕ (NOR)	XOR	XNOR
A·B	A+B	Ā	¬(A·B)	¬(A+B)	A⊕B	¬(A⊕B)

Кружок на выходе = инверсия. У XOR добавлена вторая дуга. Входы слева, выход справа.

A	B	A·B	A+B	$\overline{A}$	NAND	NOR	XOR	XNOR	A→B
0	0	**0**	**0**	**1**	**1**	**1**	**0**	**1**	**1**
0	1	**0**	**1**	**1**	**1**	**0**	**1**	**0**	**1**
1	0	**0**	**1**	**0**	**1**	**0**	**1**	**0**	**0**
1	1	**1**	**1**	**0**	**0**	**0**	**0**	**1**	**1**

Операция	Выход = 1, когда…	Как запомнить
AND (И)	**все** входы = 1	«умножение»: один ноль убивает всё
OR (ИЛИ)	**хотя бы один** вход = 1	«сложение», но 1+1 = 1
NOT (НЕ)	вход = 0	переворачивает
NAND	**не все** входы = 1 (0 только при 1,1)	AND наоборот
NOR	**все** входы = 0 (1 только при 0,0)	OR наоборот
XOR (⊕)	входы **различны** (нечётное число единиц)	сложение по модулю 2: 1⊕1 = 0
XNOR	входы **одинаковы**	равенство, XOR наоборот
A → B (импликация)	ложна только при A=1, B=0	«из истины не следует ложь»

Приоритет операций: скобки → НЕ → И → ИЛИ (XOR и импликацию лучше всегда брать в скобки). Пример: Ā·B + C = ((Ā)·B) + C.

Запомни: NAND и NOR — **универсальные**: через один только NAND можно получить всё. НЕ: NAND(A,A). И: НЕ от NAND. ИЛИ: NAND(Ā, B̄). XOR из 4 NAND: t = NAND(A,B); Y = NAND(NAND(A,t), NAND(B,t)).

Как составить таблицу истинности по формуле

Дано: логическая формула, например F = A·$\overline{B}$ + C
Что нужно сделать: составить таблицу истинности и сказать, при каких наборах F = 1
Как записать ответ: таблица со всеми 2^n строками и столбцом F; или список наборов, где F = 1

Откуда 2^n строк и такой порядок. у n входов существует ровно 2^n комбинаций (каждый вход — 0 или 1). Чтобы **не пропустить и не повторить** ни одну, строки записывают как двоичные числа по возрастанию: 000, 001, 010, 011, 100,

101, 110, 111. Старший столбец (A) меняется реже всех (4 нуля, 4 единицы), следующий (B) — парами, младший (C) — через один. Это тот же счёт, что в теме 1.

1. Выпиши все входные переменные и посчитай их число n. Нарисуй 2^n строк и заполни входы счётом 0…2^n−1 в двоичной.
2. **Разбей формулу на простые части** (по приоритету: скобки → НЕ → И → ИЛИ) и сделай для каждой части **отдельный столбец**. Так не запутаешься.
3. Заполняй столбцы слева направо: каждое значение вычисляй **из значений в той же строке** предыдущих столбцов.
4. В последнем столбце — значение F. Выпиши строки, где F = 1 (они пригодятся для ДНФ/карты Карно).

Пример. F = A·$\overline{B}$ + C (три входа → 8 строк)

A	B	C	$\overline{B}$	A·$\overline{B}$	F
0	0	0	**1**	**0**	**0**
0	0	1	**1**	**0**	**1**
0	1	0	**0**	**0**	**0**
0	1	1	**0**	**0**	**1**
1	0	0	**1**	**1**	**1**
1	0	1	**1**	**1**	**1**
1	1	0	**0**	**0**	**0**
1	1	1	**0**	**0**	**1**

Как заполнены столбцы.

1. $\overline{B}$ — просто переворот столбца B (0→1, 1→0).
2. A·$\overline{B}$ — И: равно 1 только когда A = 1 **и** столбец $\overline{B}$ = 1 (строки 100 и 101).
3. F = (A·$\overline{B}$) + C — ИЛИ: единица, если единица в столбце A·$\overline{B}$ **или** C = 1.

Ответ: F = 1 на наборах 001, 011, 100, 101, 111 (строки №1, 3, 4, 5, 7).

3.2. Законы алгебры логики

Закон	Формулы
Переместительный	A+B = B+A; A·B = B·A
Сочетательный	(A+B)+C = A+(B+C); (A·B)·C = A·(B·C)
Распределительный	A·(B+C) = A·B + A·C; **A + B·C = (A+B)·(A+C)** (второй не как в обычной алгебре!)
С константами	A+0 = A; A+1 = 1; A·0 = 0; A·1 = A
Идемпотентность	A+A = A; A·A = A
Дополнение	A + $\bar{A}$ = 1; A·$\bar{A}$ = 0; ¬¬A = A
Поглощение	A + A·B = A; A·(A+B) = A
Склеивание	A·B + A·$\bar{B}$ = A; (A+B)·(A+$\bar{B}$) = A
Частное правило	A + $\bar{A}$·B = A + B; A·($\bar{A}$ + B) = A·B
Де Морган	¬(A·B) = $\bar{A}$ + $\bar{B}$; ¬(A+B) = $\bar{A}$·$\bar{B}$ (инверсию «ломаем»: над знаком меняется И ↔ ИЛИ, черта переходит на переменные)
XOR	A⊕B = $\bar{A}$·B + A·$\bar{B}$; A⊕0 = A; A⊕1 = $\bar{A}$; A⊕A = 0; A⊕$\bar{A}$ = 1

Откуда берутся законы: проверка по таблице

Любой закон — это утверждение «две формулы дают одинаковую таблицу». Доказать его можно простым перебором всех наборов. Вот так проверяется закон Де Моргана и закон поглощения:

A	B	¬(A·B)	$\overline{A}$ + $\overline{B}$	A + A·B
0	0	**1**	**1**	**0**
0	1	**1**	**1**	**0**
1	0	**1**	**1**	**1**
1	1	**0**	**0**	**1**

Столбцы **¬(A·B)** и **$\overline{A}$ + $\overline{B}$** совпали во всех строках → закон Де Моргана верен.

Столбец **A + A·B** совпал со столбцом A (он же равен значению A в каждой строке) → поглощение: A + A·B = A.

Почему работают главные законы. Поглощение: если A = 1, то A + A·B = 1 + … = 1 (то есть равно A); если $\underline{A}$ = 0, то A·$\underline{B}$ = 0 и A + A·B = 0 (тоже равно A). Значит B вообще ни на что не влияет — «поглощается». **Склеивание**: $\underline{A}$·B + A·$\overline{B}$ = A·(B + $\overline{B}$) = A·1 = A — выносим общий множитель A, а $\underline{B}$ + $\overline{B}$ всегда 1 (одно из двух обязательно истинно). **A + $\overline{A}$·B = A + B**: по распределительному закону A + $\overline{A}$·B = (A + $\overline{A}$)·(A + B) = 1·(A + B) = A + B.

Алгоритм упрощения выражения

1. Раскрой инверсии над скобками по Де Моргану, избавься от XOR/импликации ($A \oplus B \to \bar{A}\cdot B + A\cdot\bar{B}$).
2. Раскрой скобки (распределительный закон), приведи к виду «сумма произведений».
3. Ищи **склеивание** ($A\cdot B + A\cdot\bar{B} = A$), **поглощение** ($A + AB = A$), **$A + \bar{A}B = A + B$**; убирай повторы.
4. Проверь: подставь пару наборов или составь таблицу истинности до и после — должна совпасть.

Примеры из силабуса (решения по шагам)

a) $F = \bar{A}\cdot B + \bar{A}\cdot\bar{B}$

Вынесем $\bar{A}$: $F = \bar{A}\cdot(B + \bar{B}) = \bar{A}\cdot 1 = \mathbf{\bar{A}}$. (Склеивание.)

b) $F = (A+B)' + \bar{A}\cdot B$

Де Морган: $(A+B)' = \bar{A}\cdot\bar{B}$. Тогда $F = \bar{A}\cdot\bar{B} + \bar{A}\cdot B = \bar{A}\cdot(\bar{B} + B) = \bar{A}\cdot 1 = \mathbf{\bar{A}}$.

c) $F = A\cdot\bar{B} + A\cdot B$

$F = A\cdot(\bar{B} + B) = A\cdot 1 = \mathbf{A}$.

Ещё два типичных: $A\cdot(A+B)\cdot(A+B+C)$ и $\neg(A+B) + A\cdot B$

1) $A\cdot(A+B) = A$ (поглощение), далее $A\cdot(A+B+C) = A$. Ответ **A**.

2) $\neg(A+B) = \bar{A}\cdot\bar{B}$, значит $F = \bar{A}\cdot\bar{B} + A\cdot B$ — это **эквивалентность (XNOR)**: $F = \neg(A \oplus B)$.

Подробный разбор упрощения (с названием закона на каждом шаге)

Дано: формула $F = A\cdot\bar{B} + A\cdot B + \bar{A}\cdot B$
Что нужно сделать: упростить до минимального вида
Как записать ответ: короткая формула; для проверки — совпадение таблиц

Шаг	Что делаем	Результат	Закон / почему
1	Берём первые два слагаемых, выносим A	$A\cdot(\bar{B} + B) + \bar{A}\cdot B$	распределительный: $A\cdot\bar{B} + A\cdot B = A\cdot(\bar{B} + B)$
2	Скобка $(\bar{B} + B)$ всегда 1	$A\cdot 1 + \bar{A}\cdot B$	закон дополнения: $B + \bar{B} = 1$
3	$A\cdot 1 = A$	$A + \bar{A}\cdot B$	закон с константой: $A\cdot 1 = A$
4	Применяем частное правило	**$A + B$**	$A + \bar{A}\cdot B = A + B$ (см. выше)

Проверка. Подставим все четыре набора в исходную формулу и в ответ:

A	B	исходная F	A + B
0	0	**0**	**0**
0	1	**1**	**1**
1	0	**1**	**1**
1	1	**1**	**1**

Столбцы одинаковые — упрощение верное. Получили ИЛИ (OR). Проверка таблицей занимает 30 секунд и спасает от арифметических ошибок — делай её всегда.

Дано: формула $F = \neg(A\cdot B) + A\cdot\bar{B}$
Что нужно сделать: упростить
Как записать ответ: краткая формула

Шаг	Что делаем	Результат	Закон
1	Раскрываем инверсию	$\bar{A} + \bar{B} + A\cdot\bar{B}$	Де Морган: $\neg(A\cdot B) = \bar{A} + \bar{B}$
2	Группируем $\bar{B}$ с $A\cdot\bar{B}$	$\bar{A} + \bar{B}\cdot(1 + A)$	вынесли $\bar{B}$
3	$1 + A = 1$	$\bar{A} + \bar{B}\cdot 1 = \bar{A} + \bar{B}$	закон с константой
	Итог	**$\bar{A} + \bar{B} = \neg(A\cdot B)$**	это NAND

3.3. Таблица истинности → формула → схема (синтез)

Как выписать формулу по таблице (ДНФ и КНФ)

1. **ДНФ (по единицам):** выдели строки, где F = 1. Для каждой запиши конъюнкцию всех входов: если вход в этой строке = 0 — берём его **с инверсией**, если 1 — как есть. Все конъюнкции соедини «+».
2. **КНФ (по нулям):** выдели строки, где F = 0. Для каждой запиши дизъюнкцию входов: если вход = 1 — берём **с инверсией**, если 0 — как есть. Все дизъюнкции соедини «·».
3. Упрости (законы или карта Карно).
4. Нарисуй схему: ДНФ → уровень И, затем одно ИЛИ; КНФ → уровень ИЛИ, затем одно И.

Запомни: выбирай форму по **меньшему числу** строк: если в таблице мало единиц — ДНФ, если мало нулей — КНФ.

Пример из лабораторной 4: F(X,Y) = 1, 0, 1, 0

X	Y	F
0	0	**1**
0	1	**0**
1	0	**1**
1	1	**0**

Единицы в строках 00 и 10: ДНФ = $\mathbf{\overline{X}\cdot\overline{Y} + X\cdot\overline{Y}} = \overline{Y}\cdot(\overline{X} + X) = \mathbf{\overline{Y}}$.

Нули в строках 01 и 11: КНФ = $(X + \overline{Y})\cdot(\overline{X} + \overline{Y}) = \mathbf{\overline{Y}}$. Ответ тот же.

Пример из силабуса: Q = 1, если хотя бы две из трёх входных A, B, C равны 1

A	B	C	Q
0	0	0	**0**
0	0	1	**0**
0	1	0	**0**
0	1	1	**1**
1	0	0	**0**
1	0	1	**1**
1	1	0	**1**
1	1	1	**1**

a) Таблица слева: единицы в строках 011, 101, 110, 111.

b) Исходная ДНФ: $Q = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$.

c) Упрощение: ABC добавим ещё два раза ($A + A = A$): $Q = (\overline{A}BC + ABC) + (A\overline{B}C + ABC) + (AB\overline{C} + ABC) = BC + AC + AB$.

Ответ: Q = AB + AC + BC (3 элемента И с двумя входами + ИЛИ против 4 элементов И с тремя входами + 3 НЕ — схема заметно проще).

Почему ДНФ и КНФ «работают»

Правило «0 → с инверсией» откуда берётся. Минтерм — произведение всех входов (с инверсиями у нулей), и он равен 1 **ровно на одном наборе**. Например, $A\cdot\overline{B}\cdot C$ для набора 101: A = 1, B = 1 (потому что B = 0), C = 1 → произведение 1; на любом другом наборе хотя бы один множитель равен 0 → произведение 0. Поэтому, если сложить (ИЛИ) минтермы всех нужных строк, сумма будет равна 1 ровно на этих строках — это и есть нужная функция. **КНФ** устроена зеркально: сумма-«макстерм» (с инверсиями у единиц) равна 0 ровно на одном наборе; перемножив такие суммы для всех строк с F = 0, получаем функцию, которая обращается в 0 именно там, где нужно.

A	B	C	$A\cdot\overline{B}\cdot C$	$\overline{A}+B+\overline{C}$
0	0	0	**0**	**1**
0	0	1	**0**	**1**
0	1	0	**0**	**1**
0	1	1	**0**	**1**
1	0	0	**0**	**1**
1	0	1	**1**	**0**
1	1	0	**0**	**1**
1	1	1	**0**	**1**

Левый столбец равен 1 **только** на наборе 101. Правый — 0 **только** на наборе 101. Один — минтерм, второй — макстерм одной и той же строки. Выбираем форму по количеству строк: мало единиц → ДНФ из минтермов; мало нулей → КНФ из макстермов.

Как построить схему по выражению (правило лабораторной)

1. Определи число **входов** (разных переменных).
2. Определи число и порядок **операций** (приоритет: НЕ, И, ИЛИ; скобки первыми).
3. Для каждой операции поставь вентиль. Вентили высшего приоритета — ближе к входам. Выход каждого подключай ко входу следующего.

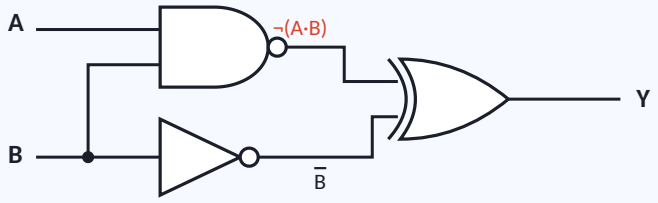
Пример: Y = Ā·B + C

Порядок: 1) НЕ от A; 2) И с B; 3) ИЛИ с C. Три входа A, B, C; три вентиля: НЕ → И → ИЛИ. Для Y = ¬(A+B)·(C+D): два ИЛИ (A+B и C+D), НЕ на первом выходе (или сразу NOR), и И на выходах.

Как прочитать схему и найти выражение (что выводит схема)

1. Подпиши входы. Иди **слева направо**: на каждом вентиле записывай выражение на его выходе через входы.
2. Особо следи за кружками (инверсия) и за XOR (двойная дуга).
3. Составь таблицу истинности по полученной формуле или упрости её.

Пример: схема из лабораторной 3 (NAND, NOT и XOR)



A	B	¬(AB)	B̄	Y
0	0	**1**	**1**	**0**
0	1	**1**	**0**	**1**
1	0	**1**	**1**	**0**
1	1	**0**	**0**	**0**

Y = ¬(A·B) ⊕ B̄. По таблице Y = 1 только при A=0, B=1, то есть **Y = $\overline{A}$·B**. Так проверяют: пройти по схеме для каждого набора входов.

Задача «что выводит схема»: пошаговое чтение

Дано: схема из NAND, NOT и XOR (рисунок выше): A и B подаются на NAND; B — на NOT; выходы NAND и NOT идут на XOR
Что нужно сделать: определить, какую функцию реализует схема (формула и таблица)
Как записать ответ: формула Y(A,B) и/или таблица; можно назвать известный вентиль

1. Подпиши **выход каждого вентиля** через входы: NAND(A,B) = ¬(A·B); NOT(B) = B̄; XOR(…) берёт оба эти сигнала.
2. Запиши итоговую формулу: **Y = ¬(A·B) ⊕ B̄**.
3. Для каждого набора (A,B) вычисли значения на проводах по порядку (таблица ниже) — это надёжнее, чем преобразовывать формулу.
4. Сравни столбец Y с известными таблицами (AND, OR, XOR…), либо запиши ответ формулой по единицам.

A	B	NAND(A,B)	NOT B	Y = XOR
0	0	1	1	1 ⊕ 1 = **0**
0	1	1	0	1 ⊕ 0 = **1**
1	0	1	1	1 ⊕ 1 = **0**
1	1	0	0	0 ⊕ 0 = **0**

Единица только в строке A = 0, B = 1 ⇒ **Y = $\overline{A}$·B** (строка единственная, поэтому ДНФ из одного минтерма).

Типовые выражения из лабораторной 4 (ответы)

В файле знак ⊕ показан как «Å». Для выражений даны вектор значений F (строки 000, 001, …, 111 или 0000…1111) и упрощение, найденное картой Карно.

№	Выражение	Вектор F	Минтермы	Упрощённое
1	A·$\overline{B}$ + B·$\overline{A}$ + C·$\overline{B}$	01111100	Σ(1,2,3,4,5)	**$\overline{A}$C + $\overline{A}$B + A$\overline{B}$**

3	$(A+B)\cdot\overline{A} + \overline{B}\cdot(A\oplus C)$	01111000	Σ(1,2,3,4)	$\overline{A}C + \overline{A}B + A\overline{B}\,\overline{C}$
4	$A + B\cdot(\neg(A\cdot C) \oplus C)$	00101111	Σ(2,4,5,6,7)	$B\overline{C} + A$
6	$(\overline{A} + \overline{B} + \overline{C})\cdot A\cdot C$	00000100	Σ(5)	$A\overline{B}C$
7	$((A\cdot B \oplus B\cdot C) + C\cdot D)\cdot\overline{A}$	0001001100000000	Σ(3,6,7)	$\overline{A}CD + \overline{A}BC$
8	$\neg((A+C)\cdot(B+D))$	1110110010100000	Σ(0,1,2,4,5,8,10)	$\overline{A}\,\overline{C} + \overline{B}\,\overline{D}$
9	$A\cdot(A+B)\cdot(A+B+C)$	00001111	Σ(4,5,6,7)	A
10	$\neg(A \oplus B \oplus C)$	10010110	Σ(0,3,5,6)	$\overline{A}\,\overline{B}\,\overline{C} + \overline{A}BC + A\overline{B}C + AB\overline{C}$

Лабораторная 4, задание 2: схемы по таблицам истинности — ответы (МДНФ)

Три входа x_1, x_2, x_3 (колонки F1…F9 из твоего файла). Ответ — минимальная ДНФ; схема строится по ней.

Ф-ция	Вектор (000…111)	Минтермы	МДНФ
F1	00110010	2,3,6	$\overline{x_1}\cdot x_2 + x_2\cdot\overline{x_3}$
F2	10001001	0,4,7	$\overline{x_2}\cdot\overline{x_3} + x_1\cdot x_2\cdot x_3$
F3	00001110	4,5,6	$x_1\cdot\overline{x_2} + x_1\cdot\overline{x_3}$
F4	10001000	0,4	$\overline{x_2}\cdot\overline{x_3}$
F5	11100000	0,1,2	$\overline{x_1}\cdot\overline{x_2} + \overline{x_1}\cdot\overline{x_3}$
F6	11000000	0,1	$\overline{x_1}\cdot\overline{x_2}$
F7	11001000	0,1,4	$\overline{x_1}\cdot\overline{x_2} + \overline{x_2}\cdot\overline{x_3}$
F8	00000101	5,7	$x_1\cdot x_3$
F9	00001111	4,5,6,7	x_1

Две переменные x_1, x_2:

Ф-ция	Вектор (00,01,10,11)	Минтермы	МДНФ
F10	1110	0,1,2	$\overline{x_1} + \overline{x_2}$
F11	0011	2,3	x_1
F13	1001	0,3	$\overline{x_1}\cdot\overline{x_2} + x_1\cdot x_2$

> **Запомни:** F10 = 1,1,1,0 — это **NAND** ($\overline{x_1} + \overline{x_2}$); F13 = 1,0,0,1 — **XNOR** (эквивалентность); F11 = 0,0,1,1 — просто x_1. Узнавать такие «стандартные» таблицы с первого взгляда — экономия времени на РК.

Тема 4 Формы логических выражений и карты Карно

ДНФ · КНФ · минимизация · карта Карно на 2, 3, 4 переменные

4.1. Зачем минимизировать и какие термины нужны

Формула, полученная прямо по таблице (СДНФ), обычно длинная: много слагаемых и много букв в каждом. Каждая операция в формуле — это вентиль, а каждый вентиль стоит места, денег и времени. **Минимизация** — упрощение формулы без изменения функции. Законы алгебры (тема 3) универсальны, но требуют догадки; **карта Карно** превращает упрощение в «рисование прямоугольников».

Термин	Что это	Пример (3 переменные)
Набор	комбинация значений входов; номер набора = двоичное число (старшая переменная — старший бит)	A=1, B=0, C=1 → 101_2 = 5
Минтерм	произведение **всех** переменных (0 → с инверсией), равное 1 на одном наборе	набор 5: $A\cdot\overline{B}\cdot C$
СДНФ	сумма минтермов всех строк с F = 1 (длинная, но однозначная)	$\overline{A}BC + A\overline{B}C$
ДНФ	любая сумма произведений	$AC + \overline{A}B$
СКНФ / КНФ	произведение сумм по строкам с F = 0 (1 → с инверсией)	$(A+B+C)(\overline{A}+B+C)$
МДНФ / МКНФ	**минимальные** формы — результат карты Карно, именно их реализуют схемой	BC

Как найти номер набора и минтерм

Набор A=1, $\underline{B}$=0, C=1. Читаем биты подряд как двоичное число: $\mathbf{101_2}$ = 1·4 + 0·2 + 1·1 = **5**. Это «минтерм №5»: его формула $A\cdot\overline{B}\cdot C$ — единица у A и C, ноль (с чертой) у B. Запись **Σ(1, 3, 5)** означает «функция равна 1 на наборах 1, 3 и 5, а на остальных 0».

4.2. Карта Карно: идея

Карта Карно — это таблица истинности, **переложенная в прямоугольник** так, чтобы соседние клетки отличались ровно одним битом. Тогда две соседние единицы можно склеить по закону $A\cdot B + A\cdot\overline{B} = A$: переменная, которая различается у соседей, исчезает.

Откуда порядок 00-01-11-10 и «замкнутость». посмотри на наборы 5 (101) и 7 (111): они отличаются только битом B. Минтермы: $A\cdot\overline{B}\cdot C$ и $A\cdot B\cdot C$. Их сумма $A\cdot C\cdot(\overline{B} + B) = A\cdot C$ — переменная B ушла. Чтобы такие «соседи» всегда стояли рядом на рисунке, оси карты подписывают в порядке **00 → 01 → 11 → 10** (код Грея: между соседними значениями меняется один бит), а не 00, 01, 10, 11. Поэтому и край карты соседствует с противоположным краем: после 10 снова идёт 00, их тоже отличает один бит (первый).

2 переменные

B\A	0	1
0	0 **0**	2 **0**
1	1 **0**	3 **0**

3 переменные

C\AB	00	01	11	10
0	0 **0**	2 **0**	6 **0**	4 **0**
1	1 **0**	3 **0**	7 **0**	5 **0**

4 переменные

CD\AB	00	01	11	10
00	0 **0**	4 **0**	12 **0**	8 **0**
01	1 **0**	5 **0**	13 **0**	9 **0**
11	3 **0**	7 **0**	15 **0**	11 **0**
10	2 **0**	6 **0**	14 **0**	10 **0**

Как найти клетку: как читать эту карту: столбцы подписаны первыми переменными (A, B), строки — последними (C или C, D). Номер клетки (маленькая цифра) = двоичное число из значений на её осях. Пример для 4 переменных: столбец AB = 01 и строка CD = 11 → 0111_2 = **7**.

Алгоритм минимизации картой Карно

Дано: функция, заданная таблицей истинности, формулой или списком Σ(…)
Что нужно сделать: найти минимальную ДНФ
Как записать ответ: формула из суммы коротких произведений (по одному слагаемому на каждую группу)

1. **Нарисуй карту** нужного размера (2, 3 или 4 переменные) и подпиши оси в порядке 00-01-11-10.
2. **Расставь единицы:** для каждого набора с F = 1 найди клетку по двоичному номеру и поставь 1. Остальные клетки — 0.
3. **Обведи группы** из единиц: прямоугольники по 1, 2, 4, 8 клеток (степень двойки). Группа должна быть **как можно больше**; клетку можно брать в несколько групп; групп должно быть **как можно меньше**; каждая 1 обязана попасть хотя бы в одну. Начинай с «одиноких» единиц, потом большие группы. Края карты считаются соседними.
4. **Прочитай каждую группу:** выпиши, какие переменные в группе не меняют значения. Была везде 1 → пишем букву, везде 0 → букву с чертой, меняется → букву не пишем.
5. Сложи (ИЛИ) слагаемые всех групп. Проверь, что все единицы покрыты и ни одной нуль-клетки не захвачено.

Частые ошибки: группы из 3, 5, 6, 7 клеток недопустимы; диагональные группы недопустимы; «ленивая» группа из 2 клеток там, где можно было взять 4 — тоже ошибка (формула получится длиннее минимальной).

Пример 1 (силабус). Q = Σ(1, 3, 5, 7, 10, 11, 14, 15), 4 переменные

Дано: Q = Σ(1, 3, 5, 7, 10, 11, 14, 15)
Что нужно сделать: найти МДНФ
Как записать ответ: Q = …

Шаг 1. Ставим единицы. Номер клетки = двоичное число ABCD, например 10 = 1010 → столбец AB = 10, строка CD = 10. Так для каждого из восьми номеров.

Шаг 2. Группы. Единицы 1, 3, 5, 7 образуют квадрат 2×2; единицы 10, 11, 14, 15 — тоже. Восемь единиц покрыты двумя группами по 4.

CD\AB	00	01	11	10
00	0: **0**	4: **0**	12: **0**	8: **0**
01	1: **1** ●	5: **1** ●	13: **0**	9: **0**
11	3: **1** ●	7: **1** ●	15: **1** ●	11: **1** ●
10	2: **0**	6: **0**	14: **1** ●	10: **1** ●

● клетки 1,3,5,7 → $\bar{A}D$
● клетки 10,11,14,15 → **AC**

Группа (клетки)	A	B	C	D	Что остаётся
● 1, 3, 5, 7	всегда 0 → $\bar{A}$	меняется → **убираем**	меняется → **убираем**	всегда 1 → **D**	$\bar{A}D$
● 10, 11, 14, 15	всегда 1 → **A**	меняется → **убираем**	всегда 1 → **C**	меняется → **убираем**	**AC**

Шаг 3. Читаем группы (таблица выше: если значение переменной внутри группы меняется — убираем её, если постоянно — оставляем).

Ответ: Q = $\bar{A}D$ + AC.

Пример 2 (силабус). F = $\bar{A}\cdot\bar{B}\cdot\bar{C}$ + $\bar{A}\cdot B\cdot C$ + $A\cdot B\cdot\bar{C}$ + $A\cdot B\cdot C$ (СДНФ → минимальная ДНФ)

Дано: формула в виде суммы четырёх минтермов
Что нужно сделать: упростить картой Карно
Как записать ответ: минимальная ДНФ

Шаг 1. Ищем номера минтермов. Слагаемое $\bar{A}\cdot\bar{B}\cdot\bar{C}$ — это набор 000 = №0; $\bar{A}\cdot B\cdot C$ — набор 011 = №3 (нулю соответствует буква с чертой); $A\cdot B\cdot\bar{C}$ — набор 110 = №6; $A\cdot B\cdot C$ — набор 111 = №7.

A	B	C	F
0	0	0	**1**
0	0	1	**0**
0	1	0	**0**
0	1	1	**1**
1	0	0	**0**
1	0	1	**0**
1	1	0	**1**
1	1	1	**1**

Минтермы: Σ(0, 3, 6, 7).

C\AB	00	01	11	10
0	0 **1**	2 **0**	6 **1**	4 **0**
1	1 **0**	3 **1**	7 **1**	5 **0**

- клетки 0 → **A B C**
- клетки 3,7 → **BC**
- клетки 6,7 → **AB**

Группа (клетки)	A	B	C	Что остаётся
0	всегда 0 → **$\bar{A}$**	всегда 0 → **$\bar{B}$**	всегда 0 → **$\bar{C}$**	**$\bar{A}\bar{B}\bar{C}$**
3, 7	меняется → **убираем**	всегда 1 → **B**	всегда 1 → **C**	**BC**
6, 7	всегда 1 → **A**	всегда 1 → **B**	меняется → **убираем**	**AB**

Шаг 2. Клетка 0 — «одинокая» (ни один сосед не равен 1), её придётся брать целиком: $\bar{A}\cdot\bar{B}\cdot\bar{C}$. Клетки 3 и 7 дают группу из двух (A меняется), клетки 6 и 7 — вторую (C меняется). Клетка 7 попала в две группы — так можно.
Ответ: F = $A\bar{B}\bar{C}$ + BC + AB.

Пример 3. Четыре угла: F = Σ(0, 2, 8, 10)

Дано: F = Σ(0, 2, 8, 10)
Что нужно сделать: найти МДНФ
Как записать ответ: F = …

Карта замкнута и по горизонтали, и по вертикали, поэтому четыре угловые клетки — соседи (после 10 снова идёт 00 как по строке, так и по столбцу) и образуют одну группу из 4.

CD\AB	00	01	11	10
00	0 **1**	4 **0**	12 **0**	8 **1**
01	1 **0**	5 **0**	13 **0**	9 **0**
11	3 **0**	7 **0**	15 **0**	11 **0**
10	2 **1**	6 **0**	14 **0**	10 **1**

- клетки 0,2,8,10 → **$\bar{B}\bar{D}$**

Группа (клетки)	A	B	C	D	Что остаётся
0, 2, 8, 10	меняется → **убираем**	всегда 0 → **$\bar{B}$**	меняется → **убираем**	всегда 0 → **$\bar{D}$**	**$\bar{B}\bar{D}$**

Ответ: F = $\bar{B}\bar{D}$.

Пример 4. «Безразличные» наборы X: функция «число от 4 до 9» (наборы 10…15 не бывают)

Дано: F = 1 на наборах 4…9; наборы 10…15 запрещены (безразличные, обозначаются X или d)
Что нужно сделать: найти МДНФ, используя безразличные клетки
Как записать ответ: формула

Идея. Если набор никогда не встречается, нам всё равно, чему равна функция на нём. Поэтому такие клетки можно **присоединять к группам**, если группа от этого становится больше (формула короче), но покрывать их не обязательно.

CD\AB	00	01	11	10
00	0 **0**	4 **1**	12 **X**	8 **1**
01	1 **0**	5 **1**	13 **X**	9 **1**
11	3 **0**	7 **1**	15 **X**	11 **X**
10	2 **0**	6 **1**	14 **X**	10 **X**

- клетки 4,5,6,7,12,13,14,15 → **B**
- клетки 8,9,10,11,12,13,14,15 → **A**

Группа (клетки)	A	B	C	D	Что остаётся

● 4, 5, 6, 7, 12, 13, 14, 15	меняется → **убираем**	всегда 1 → **B**	меняется → **убираем**	меняется → **убираем**	**B**
● 8, 9, 10, 11, 12, 13, 14, 15	всегда 1 → **A**	меняется → **убираем**	меняется → **убираем**	меняется → **убираем**	**A**

Ответ: F = B + A.

Пример 5. КНФ по карте: группируем нули

Дано: F = 0 только на наборах 3 (011) и 7 (111), на остальных 1
Что нужно сделать: найти минимальную КНФ
Как записать ответ: произведение сумм

Когда нулей мало, выгоднее группировать их. Группа нулей читается «наоборот»: в группе нулей переменная, равная 1, пишется **с инверсией**, равная 0 — без; внутри группы переменные соединяются **ИЛИ**, а сами группы — **И**.

C\AB	00	01	11	10
0	0 **1**	2 **1**	6 **1**	4 **1**
1	1 **1**	3 **0** ●	7 **0** ●	5 **1**

● клетки 3,7 → $(\overline{B} + \overline{C})$

Группа (клетки)	A	B	C	Что остаётся
● 3, 7	меняется → **убираем**	всегда 1 → $\overline{B}$	всегда 1 → $\overline{C}$	$(\overline{B} + \overline{C})$

Ответ: F = $\overline{B}$ + $\overline{C}$ = ¬(B·C). Проверка: F = 0, только когда B = 1 и C = 1 ✓.

4.3. Таблицы лабораторной 5: ответы (МДНФ)

Для каждой функции из таблиц лабораторной №5 записан результат минимизации картой Карно. Если у функции несколько равноценных минимальных записей, дана одна из них (по числу термов и букв они одинаково хороши).

а) две переменные (строки 00, 01, 10, 11)

Ф-ция	Вектор	МДНФ
F1	0111	$x_2 + x_1$
F2	1111	1
F3	1011	$\overline{x_2} + x_1$
F4	1100	$\overline{x_1}$
F5	1110	$\overline{x_1} + \overline{x_2}$
F6	0011	x_1
F7	1001	$\overline{x_1}\cdot\overline{x_2} + x_1\cdot x_2$

б) три переменные (строки 000…111)

Ф-ция	Вектор	Минтермы	МДНФ
F1	00110010	2,3,6	$\overline{x_1}\cdot x_2 + x_2\cdot\overline{x_3}$
F2	10001001	0,4,7	$\overline{x_2}\cdot\overline{x_3} + x_1\cdot x_2\cdot x_3$
F3	00001110	4,5,6	$x_1\cdot\overline{x_2} + x_1\cdot\overline{x_3}$
F4	10001000	0,4	$\overline{x_2}\cdot\overline{x_3}$
F5	11100000	0,1,2	$\overline{x_1}\cdot\overline{x_2} + \overline{x_1}\cdot\overline{x_3}$
F6	11000000	0,1	$\overline{x_1}\cdot\overline{x_2}$
F7	11001000	0,1,4	$\overline{x_1}\cdot\overline{x_2} + \overline{x_2}\cdot\overline{x_3}$
F8	00000101	5,7	$x_1\cdot x_3$
F9	00001111	4,5,6,7	x_1
F10	11100011	0,1,2,6,7	$\overline{x_1}\cdot\overline{x_2} + \overline{x_1}\cdot\overline{x_3} + x_1\cdot x_2$
F11	10000101	0,5,7	$\overline{x_1}\cdot\overline{x_2}\cdot\overline{x_3} + x_1\cdot x_3$
F12	00000011	6,7	$x_1\cdot x_2$

в) четыре переменные (строки 0000…1111)

Ф-ция	Вектор	Минтермы	МДНФ
F1	1100000111100011	0,1,7,8,9,10,14,15	$\bar{x_2}\cdot\bar{x_3} + x_2\cdot x_3\cdot x_4 + x_1\cdot x_3\cdot\bar{x_4}$
F2	0001111110010110	3,4,5,6,7,8,11,13,14	$\bar{x_2}\cdot x_3\cdot x_4 + \bar{x_1}\cdot x_2 + x_2\cdot\bar{x_3}\cdot x_4 + x_2\cdot x_3\cdot\bar{x_4} + x_1\cdot\bar{x_2}\cdot\bar{x_3}\cdot\bar{x_4}$
F3	1110010000110011	0,1,2,5,10,11,14,15	$\bar{x_1}\cdot\bar{x_2}\cdot\bar{x_4} + \bar{x_1}\cdot\bar{x_3}\cdot x_4 + x_1\cdot x_3$
F4	1000011001110011	0,5,6,9,10,11,14,15	$\bar{x_1}\cdot\bar{x_2}\cdot\bar{x_3}\cdot\bar{x_4} + \bar{x_1}\cdot x_2\cdot\bar{x_3}\cdot x_4 + x_2\cdot x_3\cdot\bar{x_4} + x_1\cdot\bar{x_2}\cdot x_4 + x_1\cdot x_3$
F5	1111001111100001	0,1,2,3,6,7,8,9,10,15	$\bar{x_2}\cdot\bar{x_3} + x_2\cdot\bar{x_4} + \bar{x_1}\cdot x_3 + x_2\cdot x_3\cdot x_4$
F6	0001110001110111	3,4,5,9,10,11,13,14,15	$\bar{x_2}\cdot x_3\cdot x_4 + \bar{x_1}\cdot x_2\cdot\bar{x_3} + x_1\cdot x_4 + x_1\cdot x_3$
F7	0111011111011100	1,2,3,5,6,7,8,9,11,12,13	$\bar{x_1}\cdot x_4 + x_2\cdot\bar{x_4} + \bar{x_1}\cdot x_3 + x_1\cdot\bar{x_3}$
F8	1001001001001100	0,3,6,9,12,13	$\bar{x_1}\cdot\bar{x_2}\cdot\bar{x_3}\cdot\bar{x_4} + \bar{x_1}\cdot\bar{x_2}\cdot x_3\cdot x_4 + \bar{x_1}\cdot x_2\cdot x_3\cdot\bar{x_4} + x_1\cdot\bar{x_3}\cdot x_4 + x_1\cdot x_2\cdot\bar{x_3}$
F9	0001110001110111	3,4,5,9,10,11,13,14,15	$\bar{x_2}\cdot x_3\cdot x_4 + \bar{x_1}\cdot x_2\cdot\bar{x_3} + x_1\cdot x_4 + x_1\cdot x_3$
F10	1010001010001000	0,2,6,8,12	$\bar{x_1}\cdot\bar{x_2}\cdot\bar{x_4} + \bar{x_1}\cdot x_3\cdot\bar{x_4} + x_1\cdot\bar{x_3}\cdot\bar{x_4}$
F11	1101010101010101	0,1,3,5,7,9,11,13,15	$\bar{x_1}\cdot\bar{x_2}\cdot\bar{x_3} + x_4$
F12	1100110100110011	0,1,4,5,7,10,11,14,15	$\bar{x_1}\cdot\bar{x_3} + \bar{x_1}\cdot x_2\cdot x_4 + x_1\cdot x_3$

Запомни: в твоих таблицах переменные названы x_1, x_2, x_3, x_4, а строки идут по возрастанию двоичного числа. Минтерм №m = десятичное значение битов $x_1x_2x_3x_4$. На экзамене можно вместо этого писать A, B, C, D — смысл тот же.

4.4. «Какая схема правильная»: как проверять

Дано: таблица истинности (или словесное условие) и несколько вариантов формул/схем (а, б, в)
Что нужно сделать: выбрать вариант, который реализует заданную функцию
Как записать ответ: буква варианта + короткое обоснование: на каком наборе остальные не подходят

1. Для каждого варианта **запиши формулу** (читай схему слева направо).
2. Выбери 2–3 набора, где функция из условия принимает значения 0 и 1 (первыми бери 000, 111 и те, где F = 1 редко).
3. Подставь эти наборы в каждую формулу. Вариант, давший хоть одно неверное значение, **вычёркивается** — полная таблица не нужна.
4. Оставшийся вариант проверь на остальных строках.

Пример. Функция F = A ⊕ B (единица, когда A ≠ B). Какая из формул правильная?

Набор AB	Нужно F	а) $A\cdot B + \bar{A}\cdot\bar{B}$	б) (A + B)·¬(A·B)	в) A + B
00	0	✗ (1)	✓ (0)	✓ (0)
01	1	✗ (0)	✓ (1)	✓ (1)
11	0	✗ (1)	✓ (0)	✗ (1)

Разбор. Набор 00: формула «а» даёт 1, а нужно 0 ⇒ «а» отпадает (она реализует XNOR). Набор 11: формула «в» даёт 1, а нужно 0 ⇒ «в» отпадает (это OR). Остаётся «б»; она верна на 00, 01, 11, проверим ещё набор 10: (1+0)·¬(0) = 1 ✓. **Ответ: б** (это развёрнутый XOR: «хотя бы одно, но не оба»).

Тема 5 Комбинационные схемы: сумматоры, умножители, дешифраторы, шифраторы

полусумматор · полный сумматор · умножитель 2×2 и 3×3 · дешифратор · шифратор · MUX/DEMUX

Комбинационная схема — выход в каждый момент однозначно определяется текущими входами; **памяти нет** (в отличие от триггеров, тема 6). Алгоритм синтеза всегда один: **словесное условие → таблица истинности → формулы (ДНФ) → упрощение → схема**.

Общий метод: как придумать любую комбинационную схему

Дано: словесное описание, что схема должна делать (например, «сложить два бита»)
Что нужно сделать: построить схему из вентилей
Как записать ответ: формулы выходов + рисунок схемы из вентилей

1. **Выпиши входы и выходы** и дай им имена. Сколько входов — столько же столбцов слева в таблице, сколько выходов — столько функций.
2. **Составь таблицу истинности** (2^n строк): для каждой комбинации входов определи выходы по смыслу задачи.
3. Для каждого выхода получи формулу (ДНФ по единицам) и **упрости** (законы или карта Карно, тема 4).
4. Нарисуй схему: НЕ → И → ИЛИ. Проверь схему на 2–3 наборах.

5.1. Полусумматор (Half Adder)

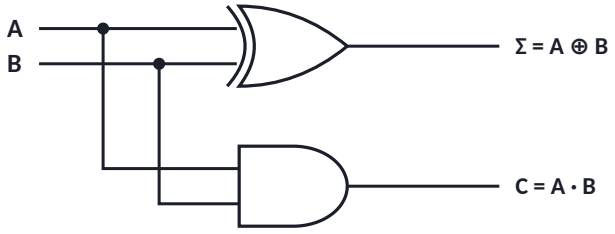
Складывает два одноразрядных числа A и B. Два входа, два выхода: **сумма Σ** и **перенос C** (Carry). Используется только в **младшем** разряде, потому что не умеет принимать перенос.

A	B	Σ (Sum)	C (Carry)
0	0	**0**	**0**
0	1	**1**	**0**
1	0	**1**	**0**
1	1	**0**	**1**

Σ = 1, когда единица ровно одна → **Σ = A ⊕ B**.

C = 1 только при 1+1 → **C = A · B**.

Проверка: $1 + 1 = 10_2$ → Σ = 0, C = 1.



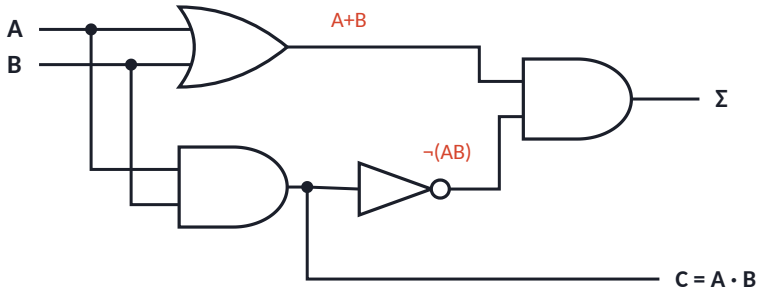
Как получается полусумматор «из условия»

Задача: сложить два одноразрядных двоичных числа. Таблица сложения: 0+0 = 0, 0+1 = 1, 1+0 = 1, 1+1 = $\mathbf{10}_2$ — результат может быть **двухразрядным**, поэтому нужны **два выхода**: младший разряд суммы Σ и старший — перенос C.

A	B	A + B	перенос C (старший бит)	сумма Σ (младший бит)
0	0	$0+0 = 00_2$	0	0
0	1	$0+1 = 01_2$	0	1
1	0	$1+0 = 01_2$	0	1
1	1	$1+1 = 10_2$	1	0

Вывод формул. столбец Σ равен 1, когда единица ровно одна (0+1 и 1+0), то есть когда входы различны — это определение XOR. Столбец C равен 1 только при 1+1 — это определение AND. Значит, **полусумматор — это просто пара вентилей XOR и AND на одних и тех же входах**: Σ = A⊕B, C = A·B.

Вариант на базовых вентилях (как в лабораторной 6): XOR можно собрать из OR, AND и NOT: Σ = (A + B)·¬(A·B), C = A·B. Схема: OR(A,B) и AND(A,B); выход AND идёт и на выход переноса, и через НЕ на второй вход конечного AND.



5.2. Полный сумматор (Full Adder)

Три входа: **A, B и Cin** (перенос из младшего разряда). Два выхода: **Σ** и **Cout** (перенос в старший). Нужен во всех разрядах, кроме младшего.

A	B	Cin	Σ	Cout
0	0	0	**0**	**0**
0	0	1	**1**	**0**
0	1	0	**1**	**0**
0	1	1	**0**	**1**
1	0	0	**1**	**0**
1	0	1	**0**	**1**
1	1	0	**0**	**1**
1	1	1	**1**	**1**

Σ = A ⊕ B ⊕ Cin (нечётное число единиц на входах).

Cout = A·B + Cin·(A ⊕ B) = A·B + A·Cin + B·Cin (единиц на входах две или три).

Σ и Cout — это просто сумма трёх бит: $1+1+1 = 11_2 \rightarrow \Sigma = 1$, Cout = 1.

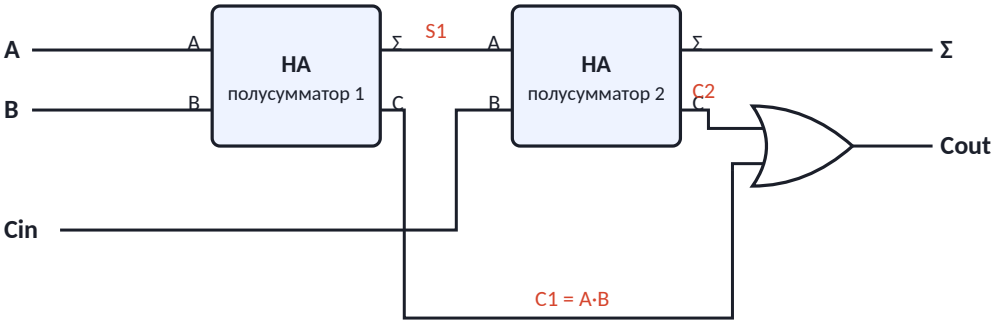
Как получается полный сумматор

В любом разряде, кроме младшего, нужно сложить **три** бита: A, B и перенос из соседнего младшего разряда Cin. Сумма трёх бит может быть от 0 до 3 (11_2) — снова два выхода.

A	B	Cin	A+B+Cin	Cout	Σ
0	0	0	$0+0+0 = 00_2$	0	0
0	0	1	$0+0+1 = 01_2$	0	1
0	1	0	$0+1+0 = 01_2$	0	1
0	1	1	$0+1+1 = 10_2$	1	0
1	0	0	$1+0+0 = 01_2$	0	1
1	0	1	$1+0+1 = 10_2$	1	0
1	1	0	$1+1+0 = 10_2$	1	0
1	1	1	$1+1+1 = 11_2$	1	1

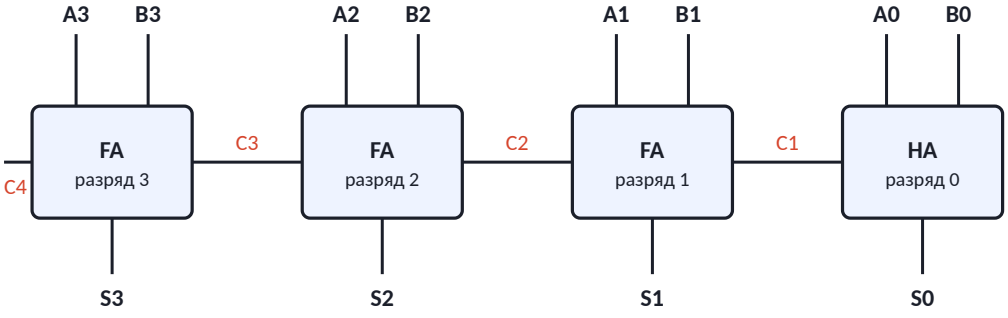
Вывод формул и схемы. Σ — младший бит суммы трёх чисел, он равен 1, когда единиц на входе нечётное число (1 или 3): это A ⊕ B ⊕ Cin. **Cout** — старший бит суммы, он равен 1, когда единиц **две или три**: Cout = A·B + A·Cin + B·Cin (функция «большинство»). **Почему «два полусумматора + ИЛИ»:** складываем сначала A и B (HA1 даёт S1 = A⊕B и перенос C1 = A·B), затем S1 и Cin (HA2 даёт Σ и перенос C2 = S1·Cin). Переносы C1 и C2 не могут быть равны 1 одновременно (если C1 = 1, то A = B = 1, значит S1 = 0 и C2 = 0), поэтому итоговый перенос Cout = C1 + C2 — простое ИЛИ.

Схема из двух полусумматоров и ИЛИ: первый HA складывает A и B (даёт S1 и C1), второй HA складывает S1 и Cin (даёт итоговую Σ и C2); Cout = C1 + C2.



5.3. Многоразрядное сложение

1. Складываем разряды от младшего к старшему. Младший — полусумматор (или полный с Cin = 0).
2. Перенос C_i каждого разряда подаётся на Cin следующего (последовательный перенос).
3. Для n-разрядных слагаемых нужно **1 полусумматор + (n−1) полных сумматоров**; результат — n+1 бит (старший = последний перенос).



Пример из лекции: $11_2 + 11_2 = 110_2$ (3 + 3 = 6)

Разряд	A	B	Cin	Σ	Cout
2^0	1	1	0	**0**	1
2^1	1	1	1	**1**	1
2^2	0	0	1	**1**	0

Разряд единиц: HA даёт Σ = 0, перенос 1. Разряд двоек: 1+1+1 → Σ = 1, Cout = 1. Разряд четвёрок: 0+0+1 → Σ = 1, Cout = 0. Результат: $\Sigma_2\Sigma_1\Sigma_0$ = **110_2** = 6. Аппаратура: **1 HA и 2 FA**.

Пример (задание лабораторной 6): 7-разрядное сложение 27 + 30

27 = 0011011_2, 30 = 0011110_2. Нужны 1 HA + 6 FA.

Разряд	A	B	Cin	Σ	Cout
2^0	1	0	0	**1**	0
2^1	1	1	0	**0**	1
2^2	0	1	1	**0**	1
2^3	1	1	1	**1**	1
2^4	1	1	1	**1**	1
2^5	0	0	1	**1**	0
2^6	0	0	0	**0**	0

Результат: 00111001_2 = **57** ✓ (последний перенос 0).

Запомни: для вычитания A − B берут сумматор, на вход B подают **инвертированные** биты (через XOR) и Cin = 1 (A + $\overline{B}$ + 1 = A − B, дополнительный код!).

Почему перенос идёт по цепочке. в десятичном сложении столбиком мы тоже идём справа налево и «запоминаем в уме» перенос. Схема делает то же: каждый сумматор считает свой разряд, а перенос по проводу уходит в соседний старший. Минус: старший разряд ждёт всю цепочку переносов, поэтому большие сумматоры медленнее (в быстрых процессорах применяют схемы ускоренного переноса).

5.4. Двоичный умножитель

Умножение «столбиком»: каждый бит множителя даёт **частичное произведение** (множимое, если бит 1, или нули, если бит 0), сдвинутое влево на номер разряда. Затем все частичные произведения **складываются**. Частичное произведение двух бит — это элемент **И**: 0·0 = 0, 0·1 = 0, 1·0 = 0, 1·1 = 1.

Пример из лекции: $111_2 \times 101_2 = 7 \times 5 = 35$

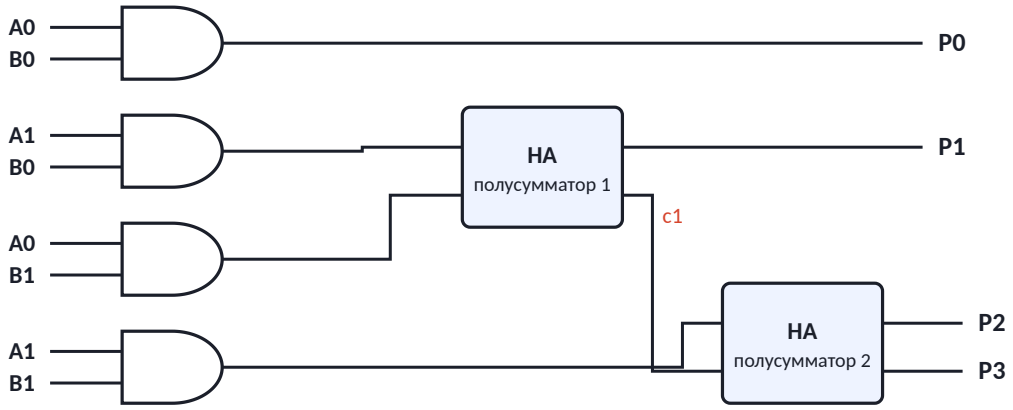
```
    111
  × 101
  -----
    111     ← множимое × 1 (бит 0)
   000      ← × 0 (бит 1), сдвиг на 1
  111       ← × 1 (бит 2), сдвиг на 2
  -------
 100011   = 35
```

Откуда берётся схема умножителя. умножение столбиком в двоичной системе очень простое: цифры множителя только 0 и 1, поэтому каждое частичное произведение — либо копия множимого (если цифра 1), либо нули (если цифра 0), сдвинутая на номер разряда. Бит частичного произведения $a_i \cdot b_j$ — это просто элемент И. Затем **складываем столбцы**: в каждом столбце несколько битов и переносы из предыдущего столбца, поэтому в схеме стоят НА (два бита) и ФА (три бита). **Откуда числа 9, 3 и 3:** битов-произведений 3×3 = 9 (9 элементов И); столбцов-сложений в матрице столько, что нужно 3 полусумматора и 3 полных сумматора.

Умножитель 2×2 (из лекции: 4 элемента И + 2 полусумматора)

Числа A = A1A0, B = B1B0, результат P3P2P1P0. Частичные произведения: A0B0 (разряд 0), A1B0 и A0B1 (разряд 1), A1B1 (разряд 2).

P0 = A0·B0; **P1 = A1B0 ⊕ A0B1**, перенос c1 = A1B0·A0B1; **P2 = A1B1 ⊕ c1**, **P3 = A1B1 · c1**.



Умножитель 3×3 (9 элементов И + 3 полусумматора + 3 полных сумматора)

A = a2a1a0, B = b2b1b0, результат P5…P0 (6 бит). Матрица частичных произведений (по столбцам-разрядам):

	разр. 4	разр. 3	разр. 2	разр. 1	разр. 0
b0 · A			a2b0	a1b0	a0b0
b1 · A (сдвиг 1)		a2b1	a1b1	a0b1	
b2 · A (сдвиг 2)	a2b2	a1b2	a0b2		

Складываем столбцы справа налево (все блоки проверены перебором всех 64 пар чисел):

Столбец	Что складываем	Переносы дальше
Разряд 0	P0 = a0b0	—

Разряд 1	HA1(a1b0, a0b1): **P1 = Σ**	перенос c1
Разряд 2	FA1(a2b0, a1b1, a0b2) → s; HA2(s, c1): **P2 = Σ**	переносы c2a (от FA1), c2b (от HA2)
Разряд 3	FA2(a2b1, a1b2, c2a) → s; HA3(s, c2b): **P3 = Σ**	переносы c3a (от FA2), c3b (от HA3)
Разряды 4, 5	FA3(a2b2, c3a, c3b): **P4 = Σ, P5 = Cout**	—

Пример: 6 × 5 = 30 на умножителе 3×3

A = 110, B = 101. Частичные: b0·A = 110; b1·A = 000 (сдвиг 1); b2·A = 110 (сдвиг 2).

```
    110
  × 101
  -----
    110
   000
  110
  ------
  11110  = 30
```

Блок-схема даёт P5…P0 = 011110 = **30** ✓.

Запомни: число блоков n×n умножителя: n^2 элементов И и около $n^2 - n$ сумматоров (для 3×3: 3 HA + 3 FA). Для 2×2: 4 И + 2 HA. Именно эти числа любят спрашивать.

Разбор работы умножителя 3×3 на примере 5 × 3 (значение каждого провода)

Дано: A = 5 = 101_2 (a2a1a0 = 101), B = 3 = 011_2 (b2b1b0 = 011)
Что нужно сделать: проследить сигналы по блокам умножителя 3×3 и получить P5…P0
Как записать ответ: число P5P4P3P2P1P0 и проверка десятичным умножением

Сначала каждое произведение $a_i \cdot b_j$ считает элемент И (девять штук). Затем блоки складывают столбцы справа налево. В таблице: что подано на вход блока, чему равна сумма и какой перенос ушёл дальше.

Разряд	Блок	Что на входе	Что считает блок	Выход Σ	Перенос
0	—	a0·b0 = 1·1	—	**P0 = 1**	—
1	HA1	a1b0 = 0, a0b1 = 1	0 ⊕ 1 = 1	**P1 = 1**	c1 = 0·1 = 0
2	FA1	a2b0 = 1, a1b1 = 0, a0b2 = 0	сумма трёх бит = 1 = 01_2	s = 1	c2a = 0
2	HA2	s = 1, c1 = 0	1 ⊕ 0 = 1	**P2 = 1**	c2b = 1·0 = 0
3	FA2	a2b1 = 1, a1b2 = 0, c2a = 0	сумма = 1 = 01_2	s = 1	c3a = 0
3	HA3	s = 1, c2b = 0	1 ⊕ 0 = 1	**P3 = 1**	c3b = 1·0 = 0
4–5	FA3	a2b2 = 0, c3a = 0, c3b = 0	сумма = 0 = 00_2	**P4 = 0**	**P5 = 0**

Результат: P5 P4 P3 P2 P1 P0 = **001111_2** = **15**. Проверка: 5 × 3 = 15 ✓.

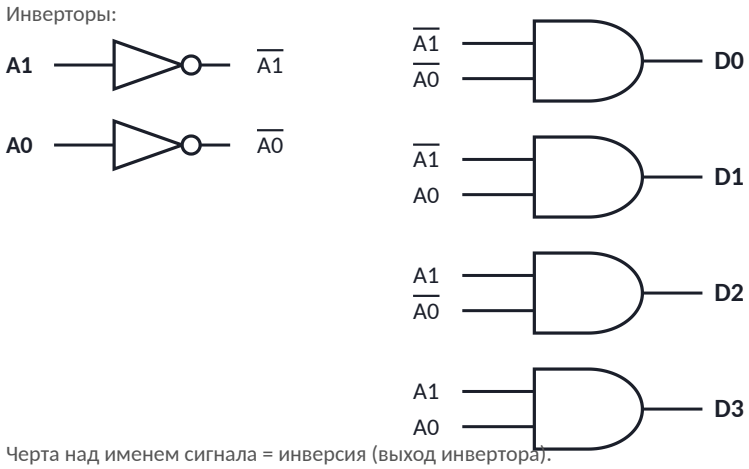
5.5. Дешифратор, шифратор, мультиплексор, демультиплексор

Дешифратор (decoder) n→2^n: двоичный код на входе → единица на **одном** выходе. Применение: адресация памяти, выбор регистра, декодирование команд.

A1	A0	D0	D1	D2	D3
0	0	**1**	0	0	0
0	1	0	**1**	0	0
1	0	0	0	**1**	0
1	1	0	0	0	**1**

D0 = $\overline{A1} \cdot \overline{A0}$; D1 = $\overline{A1} \cdot A0$; D2 = $A1 \cdot \overline{A0}$; D3 = A1·A0 (каждый выход — минтерм).

Схема: **2 НЕ + 4 И** (дешифратор 3→8: 3 НЕ + 8 И с тремя входами).



Черта над именем сигнала = инверсия (выход инвертора).

Шифратор (encoder) $2^n \to n$: обратная задача: номер активного входа → двоичный код. Предполагается, что активен **ровно один** вход.

I3	I2	I1	I0	A1	A0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

A1 = I2 + I3; A0 = I1 + I3 — только два элемента ИЛИ. Если активны несколько входов, обычный шифратор даёт ошибку (например, I1 и I2 → код 11 = «вход 3»). Поэтому применяют **приоритетный шифратор** (побеждает старший активный вход: A1 = I3 + I2, A0 = I3 + $\overline{I2}$·I1) и выход **Valid** V = I3 + I2 + I1 + I0 (отличает «все нули» от «активен вход 0»).

Откуда формулы дешифратора и шифратора. дешифратор $n \to 2^n$ — это «таблица всех минтермов»: у каждого выхода формула — произведение входов (с инверсиями), равное 1 ровно на одном коде. Поэтому для 3 входов нужно $2^3 = 8$ элементов И с тремя входами (по одному на выход) и 3 инвертора (для инверсий входов). Шифратор делает обратное: единица на входе I_k «вносит» в выходной код те двоичные разряды, которые есть в числе k: бит A1 равен 1 для k = 2 и 3, поэтому A1 = I2 + I3; бит A0 равен 1 для k = 1 и 3, поэтому A0 = I1 + I3.

Мультиплексор (MUX): выбирает на выход **один из N информационных входов** D_i по коду на управляющих входах S (**для $N = 2^n$ нужно n входов S**). Применение: выбор регистра на вход АЛУ.

MUX	Функция	Состав
2:1	$Y = \overline{S}\cdot D0 + S\cdot D1$	1 НЕ, 2 И, 1 ИЛИ
4:1	$Y = \overline{S1}\,\overline{S0}\cdot D0 + \overline{S1}\,S0\cdot D1 + S1\overline{S0}\cdot D2 + S1S0\cdot D3$	собирается из трёх MUX 2:1 (два по S0, один по S1)
8:1 / 16:1	—	3 / 4 управляющих входа

Демультиплексор (DEMUX): обратное MUX — один вход D направляется на **один из выходов**, номер задаёт код S; остальные выходы = 0. DEMUX 1:4 = дешифратор 2:4, у которого каждый элемент И получил ещё вход D: $Y_0 = D\cdot\overline{S1}\cdot\overline{S0}$ и т. д.

Устройство	Что делает	Пример применения
MUX	много входов → один выход (выбор)	выбор регистра на вход АЛУ
DEMUX	один вход → один из многих выходов	распределение сигнала по каналам
Дешифратор	код → один активный выход	адресация памяти, выбор регистра
Шифратор	активный вход → код	кодирование запросов прерываний

Мини-примеры из лекции 6

MUX 4:1 при D0=0, D1=1, D2=1, D3=0 и S1S0 = 10: выбирается D2 → Y = **1**.
DEMUX 1:4 при D=1 и S1S0 = 01: Y1 = **1**, остальные 0.
Дешифратор 2:4 при A1A0 = 11: активен **D3**.
Шифратор 4:2 при I2 = 1 (остальные 0): код **10**.

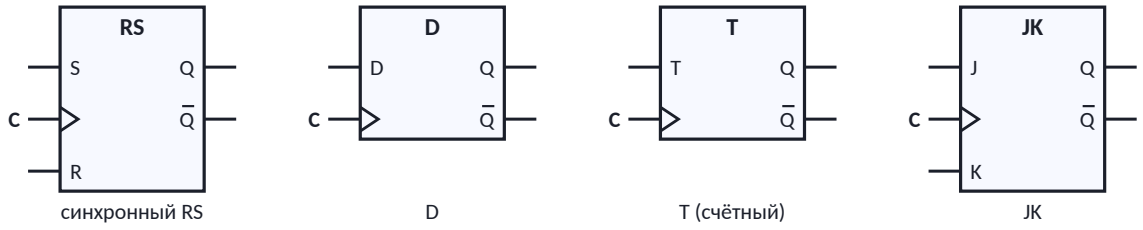
Тема 6 Триггеры

RS · D · T · JK · таблицы переходов · временные диаграммы

6.1. Что такое триггер

Комбинационные схемы запоминать не умеют. **Последовательностные** — выход зависит и от входов, и от **предыдущего состояния** (памяти). **Триггер** (flip-flop / latch) — базовая ячейка памяти: хранит **1 бит**, имеет **два устойчивых состояния** (Q = 0 или Q = 1) и два выхода: прямой **Q** и инверсный $\overline{Q}$ (парафазные выходы). Состояние держится, пока есть питание. Память держится на **положительной обратной связи** (выход одного вентиля идёт на вход другого).

Признак	Виды	Суть
Способ управления	**асинхронные** / **синхронные (тактируемые)**	асинхронный реагирует на входы сразу; синхронный — только по сигналу тактового входа C (Clock)
Как воспринимает C	**статический** (защёлка, latch) / **динамический** (flip-flop)	latch: пока C = 1, выход повторяет вход («прозрачный»). Flip-flop: запоминает вход **только в момент фронта** C (0→1 или 1→0)
Функция	**RS, D, T, JK**	закон переключения (таблица)
Структура	одноступенчатый / двухступенчатый (Master-Slave)	Master-Slave: ведущий и ведомый триггеры, выход меняется строго по фронту



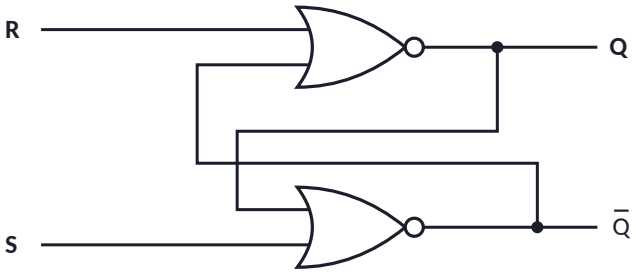
синхронный RS | D | T (счётный) | JK

Треугольник на входе C — динамический вход (срабатывание по фронту). Q — прямой, $\overline{Q}$ (с чертой) — инверсный выход.

Откуда берётся память. обычная комбинационная схема не помнит: убери вход — пропадёт выход. Чтобы запомнить бит, нужно **замкнуть выход на вход**: два вентиля соединяют перекрёстно, и каждый «удерживает» другой. У такой пары вентилей есть два устойчивых состояния: (Q = 0, $\overline{Q}$ = 1) и (Q = 1, $\overline{Q}$ = 0). Короткий импульс на входе S или R переводит схему в нужное состояние, а когда импульс исчезает, **обратная связь продолжает держать состояние** — это и есть память. Триггер — самая маленькая ячейка памяти на 1 бит.

6.2. RS-триггер (Set / Reset)

Входы: **S** (Set — установить Q = 1) и **R** (Reset — сбросить Q = 0). Два вентиля, перекрёстные обратные связи.



$Q = \neg(R + \bar{Q})$, $\bar{Q} = \neg(S + Q)$ — две перекрёстно связанные ячейки ИЛИ-НЕ

Схема на ИЛИ-НЕ (NOR) — входы активны по **1**:

S	R	Q(t+1)	Режим
0	0	Q_n (хранение)	держит прежнее состояние
1	0	1	установка (Set)
0	1	0	сброс (Reset)
1	1	Q = $\bar{Q}$ = 0	**запрещено** (после снятия обоих сигналов состояние непредсказуемо)

Как работает RS-триггер на NOR: прослеживаем сигналы

Схема: Q = NOR(R, $\overline{Q}$), $\overline{Q}$ = NOR(S, Q) (выход каждого вентиля идёт на вход другого). NOR даёт 1 только когда **оба** входа равны 0, иначе 0.

№	Что делаем	S	R	Q	$\overline{Q}$
1	исходно Q = 0	0	0	**0**	1
2	подали S = 1 (Set)	1	0	**1**	0
3	сняли S — **хранение**	0	0	**1**	0
4	подали R = 1 (Reset)	0	1	**0**	1
5	сняли R — **хранение**	0	0	**0**	1
6	снова S = 1	1	0	**1**	0
7	сняли S — хранение	0	0	**1**	0

Разбор шагов. Шаг 2: S = 1 ⇒ $\overline{Q}$ = NOR(1, …) = 0; тогда Q = NOR(R = 0, $\overline{Q}$ = 0) = 1. Шаг 3: S снова 0, но $\overline{Q}$ = NOR(S = 0, Q = 1) = 0 и Q = NOR(0, 0) = 1 — состояние **удержалось**. Шаг 4: R = 1 ⇒ Q = NOR(1, …) = 0 ⇒ $\overline{Q}$ = NOR(0, 0) = 1. Шаг 5 — снова хранение, но уже нуля. Если S = R = 1 одновременно, оба вентиля дают 0 (Q = $\overline{Q}$ = 0) — противоречит самой идее «$\overline{Q}$ — это инверсия Q», поэтому комбинация запрещена.

Схема на И-НЕ (NAND) — входы активны по **0** (инверсные $\overline{S}$, $\overline{R}$). Именно такая таблица дана в твоей лабораторной 7:

S	R	Q	Режим
0	0	Q = $\bar{Q}$ = 1	**запрещено**
0	1	1	установка Q в 1
1	0	0	установка Q в 0
1	1	Q_n	хранение

> **Не перепутай:** в лекции RS-триггер показан на NOR (запрет при S=R=1), в лабораторной — на NAND (запрет при S=R=0). Всегда смотри, **на каких вентилях** собрана схема в задаче: у NOR активная единица, у NAND — активный ноль.

Уравнение: Q(t+1) = S + $\overline{R}$·Q(t) при условии S·R = 0. **Применение:** защита от дребезга контактов кнопок и переключателей.

Синхронный RS (с входом C): при C = 0 входы игнорируются (хранение), при C = 1 работает как обычный RS. Условие запрета остаётся: нельзя подавать S = R = 1 при C = 1.

6.3. D-триггер (Data / Delay)

Один информационный вход D. Запрещённых состояний нет: внутри **R = $\overline{D}$**, S = D. **Q(t+1) = D** — по тактовому импульсу данные со входа переписываются на выход (задержка на такт). Двухступенчатый (Master-Slave) D-триггер меняет выход строго по фронту C. Применение: регистры хранения и сдвига, ячейки статической памяти (SRAM).

C	D	Q(t+1)	Режим
0 (или нет фронта)	×	Q_n	хранение
фронт ↑	0	0	запись 0
фронт ↑	1	1	запись 1

6.4. T-триггер (Toggle, счётный)

По каждому активному фронту C при T = 1 выход **инвертируется**: Q(t+1) = T⊕Q(t) = T·$\overline{Q}$ + $\overline{T}$·Q. При T = 0 — хранение. За два входных импульса на выходе один полный период → **делит частоту на 2** ($f_{out} = f_{in} / 2$), форма — меандр. Получить T-триггер можно из **D** (соединить $\overline{Q}$ → D) или из **JK** (J = K = 1).

T	Q(t+1)	Режим
0	Q_n	хранение
1	$\overline{Q}_n$	переключение (toggle)

6.5. JK-триггер

Универсальный: J — аналог Set, K — аналог Reset, но комбинация **J = K = 1 не запрещена**, а переводит триггер в **счётный режим** (инвертирует Q). **Q(t+1) = J·$\overline{Q}$ + $\overline{K}$·Q**.

J	K	Q(t+1)	Режим

0	0	Q_n	хранение
0	1	0	сброс (Reset)
1	0	1	установка (Set)
1	1	$\overline{Q}_n$	**переключение** (toggle)

Сводка по триггерам

Тип	Входы	Q(t+1)	Как запомнить
RS	S, R	$S + \overline{R}\cdot Q$	S=1 → 1; R=1 → 0; 11 запрещено
D	D	D	«копия»: что на входе, то и запомнил
T	T	$T \oplus Q$	T=1 → переворот; делитель на 2
JK	J, K	$J\cdot\overline{Q} + \overline{K}\cdot Q$	RS без запрета: 11 = переворот

Таблица возбуждения (нужна для синтеза счётчиков)

Q(t) → Q(t+1)	J	K	D	T
0 → 0	0	×	0	0
0 → 1	1	×	1	1
1 → 0	×	1	0	1
1 → 1	×	0	1	0

× — безразличное значение (может быть 0 или 1).

6.6. Временные диаграммы: как рисовать

1. Определи, как триггер **срабатывает**: по **уровню** (C = 1 — «защёлка») или по **фронту** (C ↑: переход 0→1; C ↓: переход 1→0).
2. Проведи вертикальные линии (красные пунктиры ниже) в моменты **активных фронтов**. Вне этих моментов (для динамических) триггер игнорирует входы.
3. В каждый момент срабатывания **прочитай входы непосредственно ДО фронта** и по таблице определи новое Q.
4. Нарисуй Q: между моментами срабатывания выход **не меняется**, изменение — вертикальный скачок точно на фронте.
5. Начальное состояние берётся из условия (в твоих заданиях Q(0) = 0).

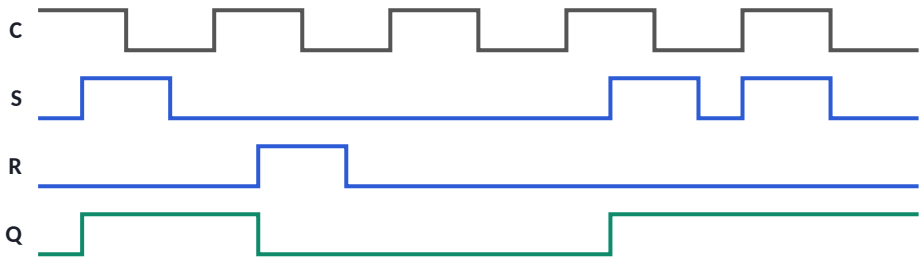
Осторожно: для динамических триггеров не ориентируйся на значение входа во время фронта и после него — важно значение **до** фронта (setup). Для триггера-защёлки (по уровню) выход следит за входом всё время, пока C = 1.

Дано: временная диаграмма: тактовый сигнал C, входы (D, T, или J и K, или S и R) и начальное состояние Q(0) = 0
Что нужно сделать: нарисовать выход Q(t) (на диаграмме — какие значения принимает Q на каждом такте)
Как записать ответ: график Q ниже входов; на экзамене удобно подписать значения Q после каждого фронта

Универсальный порядок оформления (по нему построены все четыре задания ниже): 1) отметь фронты C красными линиями; 2) для каждого фронта в **таблице событий** выпиши входы **до** фронта; 3) по таблице триггера найди новое Q; 4) перенеси значения Q на график ступеньками.

Задание 1. Синхронный RS-триггер, тактирование по уровню C = 1, Q(0) = 0

S, R подаются как на рисунке; Q меняется только пока C = 1



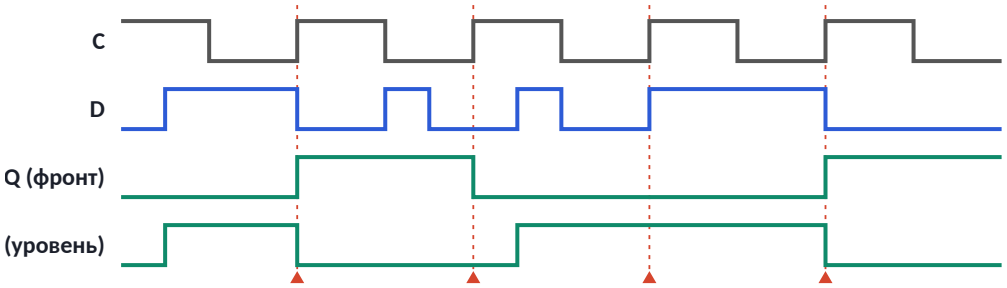
Разбор. Окна C = 1: единицы времени 0–1, 4–5, 8–9, 12–13, 16–17. В первом окне S = 1 с момента 1 → Q = 1 (и остаётся при C = 0, хотя S = 1 в момент 2 — вход игнорируется). Во втором окне R = 1 с момента 5 → Q = 0. В третьем S = R = 0 → хранение. В четвёртом S = 1 → Q = 1. В пятом S = 1 → Q остаётся 1. S = R = 1 при C = 1 не подавалось (запрещено).

Таблица событий (окна C = 1). Триггер по уровню: пока C = 1, выход следит за S и R; при C = 0 — хранит. Строка таблицы — одно окно C = 1.

окно	моменты t	S в окне	R в окне	что происходит	Q до окна	Q в конце окна
1	0…1	01	00	Set: Q = 1	0	1
2	4…5	00	01	Reset: Q = 0	1	0
3	8…9	00	00	хранение	0	0
4	12…13	01	00	Set: Q = 1	0	1
5	16…17	11	00	Set: Q = 1	1	1

Задание 2. D-триггер, тактирование по переднему фронту C ↑, Q(0) = 0

Сравнение: Q_{edge} — D-триггер по фронту; Q_{latch} — защёлка по уровню C = 1 (для контраста)



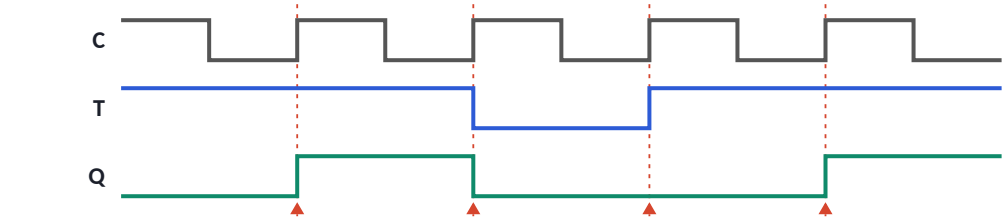
Разбор. Фронты C ↑ в моменты 4, 8, 12, 16 (первый фронт при t = 0 — это начало диаграммы, Q(0) = 0 задано). Перед фронтом 4 вход D = 1 → Q = 1. Перед 8: D = 0 → Q = 0. Перед 12: D = 0 → Q = 0. Перед 16: D = 1 → Q = 1. Короткий импульс D в момент 6 между фронтами никак не влияет — триггер его «не видит». У защёлки (нижний график) любое изменение D при C = 1 проходит на выход сразу.

Таблица событий (фронты C ↑). D берётся в момент **перед** фронтом (t − 1).

фронт №	момент t	D до фронта	Q до	правило D-триггера	Q после
1	4	1	0	Q := D = 1	**1**
2	8	0	1	Q := D = 0	**0**
3	12	0	0	Q := D = 0	**0**
4	16	1	0	Q := D = 1	**1**

Задание 3. T-триггер, счётный режим, фронт C ↑, Q(0) = 0

T = 1 на фронтах 4, 8, 16 (переключение); T = 0 на фронте 12 (хранение)



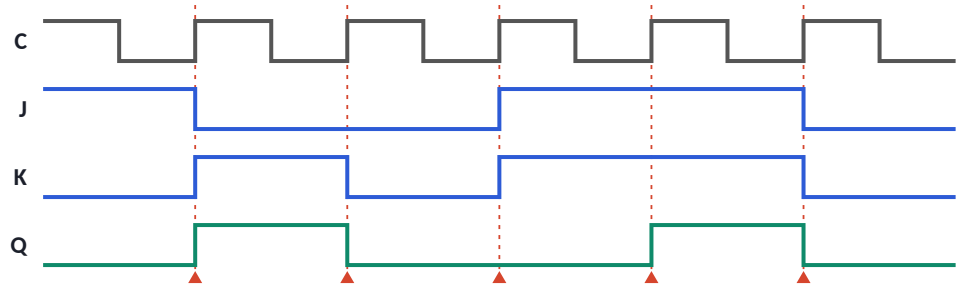
Разбор. Фронт 4: T = 1 → Q инвертируется 0 → 1. Фронт 8: T = 1 → 1 → 0. Фронт 12: T = 0 → хранение, Q = 0. Фронт 16: T = 1 → 0 → 1. При T = 1 постоянно Q переключался бы на каждом фронте — период Q вдвое больше периода C (деление частоты на 2).

Таблица событий.

фронт №	момент t	T до фронта	Q до	правило T-триггера	Q после
1	4	1	0	Q := $\overline{Q}$ (переключение)	**1**
2	8	1	1	Q := $\overline{Q}$ (переключение)	**0**
3	12	0	0	Q := Q (хранение)	**0**
4	16	1	0	Q := $\overline{Q}$ (переключение)	**1**

Задание 4. JK-триггер, фронт C ↑, Q(0) = 0

Четыре режима по очереди: Set, Reset, Hold, Toggle, Toggle



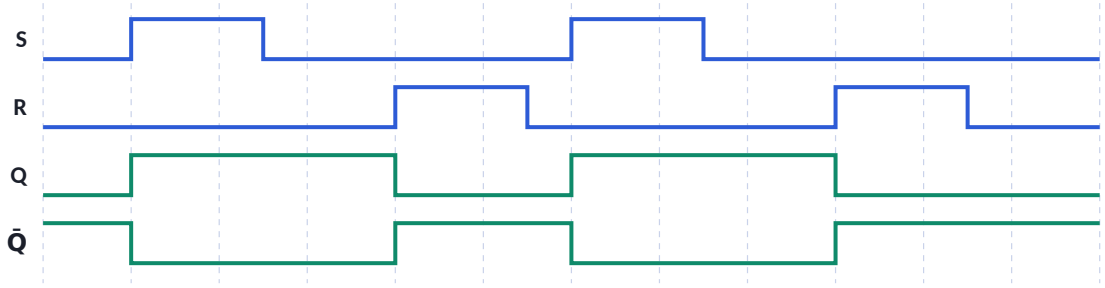
Разбор. Перед фронтом 4: J=1, K=0 → Q = 1 (Set). Перед 8: J=0, K=1 → Q = 0 (Reset). Перед 12: J=0, K=0 → хранение (Q = 0). Перед 16: J=1, K=1 → переключение 0 → 1. Перед 20: J=1, K=1 → снова переключение 1 → 0.

Таблица событий.

фронт №	момент t	J до	K до	Q до	правило JK-триггера	Q после
1	4	1	0	0	Set: Q := 1	**1**
2	8	0	1	1	Reset: Q := 0	**0**
3	12	0	0	0	хранение	**0**
4	16	1	1	0	переключение	**1**
5	20	1	1	1	переключение	**0**

Асинхронный RS-триггер на NOR (лабораторная 7): наблюдение «Word Generator + Logic Analyzer»

S и R — импульсы; пока оба 0 — триггер хранит состояние



Видно: S = 1 устанавливает Q = 1 и это состояние сохраняется после снятия S (память!); R = 1 сбрасывает Q = 0. $\bar{Q}$ всегда противоположен Q. Одновременно S = R = 1 подавать нельзя.

6.7. Время установки, метастабильность, применение

Понятие	Суть
Setup time t_{SU}	минимальное время, в течение которого вход должен быть **стабилен ДО** фронта C
Hold time t_H	минимальное время, в течение которого вход должен оставаться стабильным **ПОСЛЕ** фронта C
Метастабильность	если вход меняется в окне $[t_{SU} + t_H]$ вокруг фронта, выход «зависает» между 0 и 1 на непредсказуемое время → сбои
Защита	двухкаскадный синхронизатор: два D-триггера подряд на асинхронный вход — вероятность сбоя резко падает

Где применяют	Что
Регистры хранения	группа D-триггеров хранит n-разрядное слово
Регистры сдвига	данные сдвигаются на 1 разряд за такт (умножение/деление на 2)
Счётчики импульсов	цепочка T-триггеров, ёмкость 2^n, деление частоты на 2, 4, 8, 16…
SRAM	ячейка памяти — триггер (6 транзисторов), хранит данные без регенерации

Тема 7 Последовательностные схемы: счётчики

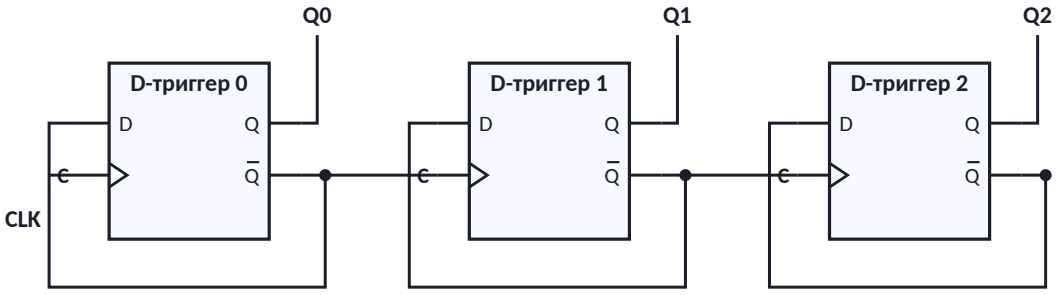
асинхронный (ripple) · синхронный · счётчик по модулю 10 · временные диаграммы

Счётчик — устройство, считающее импульсы на тактовом входе; строится из триггеров. Состояние хранится в n триггерах → **модуль счёта K = 2^n** (3 триггера: 0…7, 4 триггера: 0…15). Каждый T-триггер делит частоту на 2, поэтому выход i-го разряда имеет частоту $f_{CLK} / 2^{i+1}$.

Тип	Как тактируются триггеры	Плюсы / минусы
Асинхронный (последовательный, ripple)	CLK подаётся только на 1-й триггер, остальные тактируются выходом предыдущего	простая схема; но задержки **накапливаются** ($n \cdot t_{зад}$), на промежуточных состояниях бывают ложные «пики»
Синхронный (параллельный)	CLK подан на **все** триггеры одновременно; нужные переключения задаёт логика на входах J/K (или D)	быстрый (все переключаются разом, нет накопления задержек); схема сложнее

Почему n триггеров = 2^n состояний. каждый триггер хранит 1 бит и имеет 2 состояния; n триггеров вместе — 2^n различных комбинаций, то есть счётчик умеет считать 2^n разных состояний от 0 до $2^n - 1$. Число 2^n называют **модулем** счёта. Когда счётчик сосчитал до максимума, следующий импульс «переполняет» его — и счёт начинается с нуля, как в километраже автомобиля.

7.1. 3-разрядный асинхронный суммирующий счётчик на D-триггерах



D = $\bar{Q}$ (каждый триггер переключается), тактом следующего служит $\bar{Q}$ предыдущего (фронт 0→1)

1. Каждый D-триггер превращён в счётный: **D = $\overline{Q}$** (вход соединён со своим инверсным выходом), поэтому он переключается на каждом тактовом фронте 0→1.
2. Первому триггеру тактом служит CLK. Тактом каждого следующего — **$\overline{Q}$ предыдущего**: когда предыдущий Q переходит 1 → 0, $\overline{Q}$ идёт 0 → 1, и следующий триггер переключается (это «перенос»).
3. Получается двоичный счёт 000, 001, 010, … 111, 000… (Q0 — младший разряд).

Временная диаграмма: CLK и Q0, Q1, Q2 (9 тактов)

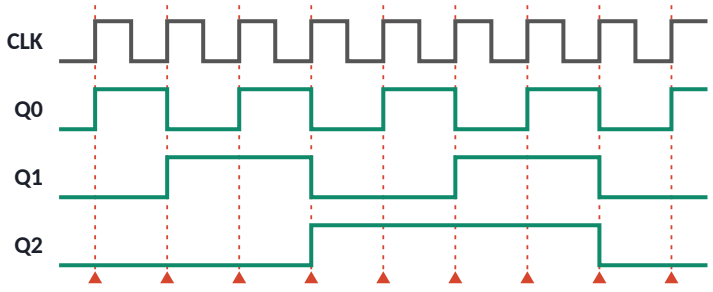


Таблица состояний: что происходит на каждом импульсе

Дано: 3-разрядный асинхронный счётчик на D-триггерах, исходное состояние 000
Что нужно сделать: выписать состояния Q2Q1Q0 после каждого из 9 импульсов
Как записать ответ: последовательность двоичных/десятичных кодов; начиная с 9-го импульса состояние повторяется

импульс	было Q2Q1Q0	переключились	стало Q2Q1Q0	десятичное	почему
1	000	Q0	**001**	1	Q0 переключается на каждый CLK
2	001	Q0, Q1	**010**	2	Q0 переключается на каждый CLK; Q0: 1→0 ⇒ $\overline{Q}$0 растёт ⇒ Q1 переключается
3	010	Q0	**011**	3	Q0 переключается на каждый CLK
4	011	Q0, Q1, Q2	**100**	4	Q0 переключается на каждый CLK; Q0: 1→0 ⇒ $\overline{Q}$0 растёт ⇒ Q1 переключается; Q1: 1→0 ⇒ $\overline{Q}$1 растёт ⇒ Q2 переключается
5	100	Q0	**101**	5	Q0 переключается на каждый CLK
6	101	Q0, Q1	**110**	6	Q0 переключается на каждый CLK; Q0: 1→0 ⇒ $\overline{Q}$0 растёт ⇒ Q1 переключается
7	110	Q0	**111**	7	Q0 переключается на каждый CLK
8	111	Q0, Q1, Q2	**000**	0	Q0 переключается на каждый CLK; Q0: 1→0 ⇒ $\overline{Q}$0 растёт ⇒ Q1 переключается; Q1: 1→0 ⇒ $\overline{Q}$1 растёт ⇒ Q2 переключается
9	000	Q0	**001**	1	Q0 переключается на каждый CLK

Правило, которое видно из таблицы: **младший разряд меняется на каждом импульсе; следующий меняется тогда, когда предыдущий перешёл из 1 в 0** (у предыдущего $\overline{Q}$ при этом идёт 0 → 1 — фронт для тактового входа следующего).

Состояния после каждого импульса: **001**, **010**, **011**, **100**, **101**, **110**, **111**, **000**, **001** (после 8-го импульса счётчик возвращается в 000: переполнение).

> **Запомни:** для триггеров, срабатывающих по **спаду** (1→0), тактом следующего берут **Q** предыдущего (суммирующий счётчик). Если нужен **вычитающий** счётчик на триггерах по фронту ↑ — подают $\overline{Q}$ (не Q) предыдущего.

Ложные состояния: разбор перехода 011 → 100

> **Дано:** асинхронный 3-разрядный счётчик; задержка срабатывания одного триггера 10 нс; в момент t = 0 в состоянии 011 приходит фронт CLK ↑
> **Что нужно сделать:** выписать, какие коды видны на выходах Q2Q1Q0 с течением времени
> **Как записать ответ:** последовательность кодов и время установления конечного

момент	событие	Q2Q1Q0	десятичное	пояснение
t = 0	CLK ↑	011	011 (=3)	исходное состояние
t = 10 нс	Q0: 1→0	010	010 (=2)	Q0 переключился; $\overline{Q}$0 пошёл вверх и запустил триггер Q1
t = 20 нс	Q1: 1→0	000	000 (=0)	Q1 переключился; $\overline{Q}$1 пошёл вверх и запустил триггер Q2
t = 30 нс	Q2: 0→1	100	100 (=4)	Q2 переключился — установилось окончательное значение

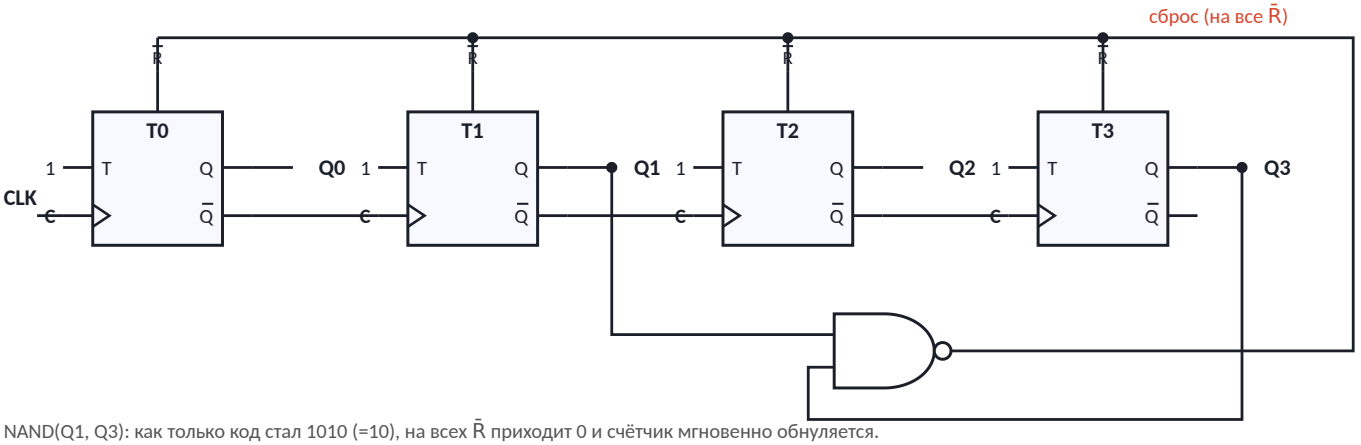
Итог: пока идёт цепочка переключений, на выходах кратковременно присутствуют **«чужие» коды 010 и 000**, которых нет в нормальной последовательности. Если подключён дешифратор, он на мгновение выдаст ложные сигналы (глитчи). Полное время установления равно $n \cdot t_{зад} = 3 \cdot 10 =$ **30 нс**, а минимальный период тактов не может быть короче: $f_{max} \approx 1 / 30$ нс ≈ 33 МГц.

Недостатки асинхронного счётчика (вопрос из силабуса)

- **Накопление задержки:** старший разряд меняется через $n \cdot t_{зад}$ после тактового импульса, максимальная частота ограничена.
- **Ложные состояния:** при переходе 011 → 100 выходы меняются по очереди (011 → 010 → 000 → 100), на короткое время возникают «чужие» коды; дешифратор может дать ложный сигнал (глитч).
- Выходы нельзя считывать сразу после фронта — нужно подождать окончания переходного процесса.

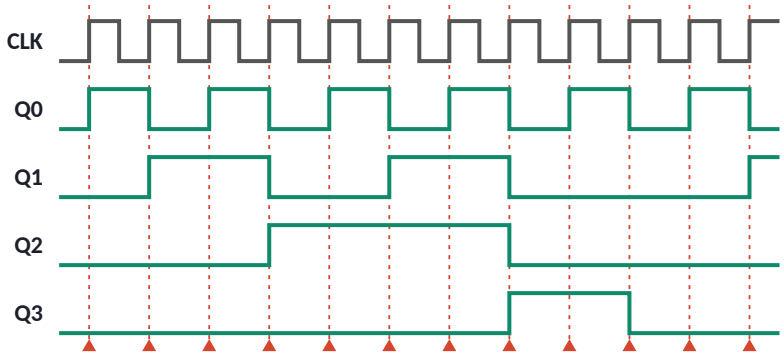
7.2. Счётчик по модулю 10 (декадный) на основе асинхронного

Нужны **4 триггера** ($2^3 = 8 < 10 \le 16 = 2^4$). Идея: пусть счётчик считает как обычно, но в момент, когда он достигает состояния **10 = 1010**, его **асинхронно сбрасывают** в 0000. Состояние 1010 — это Q3 = 1 и Q1 = 1 (другие разряды в этот момент ещё не важны, т.к. 1010 — первое состояние, где одновременно Q3 = Q1 = 1). Логика сброса — один элемент **NAND(Q3, Q1)**, выход соединяется со входами сброса $\overline{R}$ (активный низкий) всех триггеров.



NAND(Q1, Q3): как только код стал 1010 (=10), на всех R̄ приходит 0 и счётчик мгновенно обнуляется.

Диаграмма счётчика по модулю 10: 0, 1, …, 9, 0, 1 (12 тактов)



Последовательность состояний: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2. Состояние 1010 существует доли наносекунды (до срабатывания сброса) и в счёте не участвует.

Почему сброс именно по NAND(Q3, Q1)

Дано: нужен счётчик, который считает 0, 1, …, 9 и сразу после 9 возвращается в 0
Что нужно сделать: выбрать условие, по которому обнуляется счётчик, и логический элемент для него
Как записать ответ: элемент (NAND, NOR…) и его входы

Следим за состояниями по порядку. Нужно найти **самый первый код после 9** (то есть 10 = 1010) и убедиться, что **ни один из кодов 0…9 не вызовет сброс**. Для этого смотрим, в каких разрядах у 1010 стоят единицы: Q3 и Q1. Выбираем минимальный набор разрядов, который однозначно выделяет 1010 среди 0…10: только состояния 10 и выше (1010, 1011, …) имеют Q3 = Q1 = 1 (в 8 и 9 разряд Q1 равен 0, в 0…7 разряд Q3 равен 0).

число	код Q3Q2Q1Q0	Q3	Q1	NAND(Q3,Q1)	результат
0	0000	0	0	1	—
1	0001	0	0	1	—
2	0010	0	1	1	—
3	0011	0	1	1	—
4	0100	0	0	1	—
5	0101	0	0	1	—
6	0110	0	1	1	—
7	0111	0	1	1	—
8	1000	1	0	1	—
9	1001	1	0	1	—
10	1010	1	1	0	**СБРОС → 0000**

Пока NAND = 1, сброс неактивен. На числе 10 оба входа равны 1 → NAND = 0 → активный низкий уровень на входах сброса всех триггеров → счётчик обнуляется. Использован один 2-входовой элемент И-НЕ.

Как вывести цифры 0…9 (дешифрация)

Нужен **дешифратор 4→10**: каждому выходу соответствует свой код Q3Q2Q1Q0. Например, цифра 7 = 0111: Y7 = Q3·Q2·Q1·Q0 (элемент И-НЕ/И на 4 входа с инверсией Q3), цифра 0: Y0 = Q3·Q2·Q1·Q0 и т. д. Итого **10 элементов И** (на 4 входа) и 4 инвертора. Удобно использовать элемент **NAND** — получаются выходы с активным нулём. Из-за ложных состояний асинхронного счётчика выходы дешифратора стробируют (считывают в определённый момент).

7.3. Синхронные счётчики

У всех триггеров **общий CLK**. Какой триггер переключится на данном такте, решает логика на входах J, K: разряд должен переключиться, если **все младшие разряды равны 1**.

Счётчик	Входы JK-триггеров	Комментарий
3-разрядный суммирующий	J0 = K0 = 1 J1 = K1 = Q0 J2 = K2 = Q0·Q1	каждый старший разряд «видит», что все младшие = 1 (аналог переноса)
Декадный (0…9)	J0 = K0 = 1 J1 = Q3·Q0, K1 = Q0 J2 = K2 = Q0·Q1 J3 = Q0·Q1·Q2, K3 = Q0	после 9 (1001) возвращается в 0000 без сброса

Запомни: алгоритм синтеза синхронного счётчика: 1) выпиши таблицу состояний; 2) по **таблице возбуждения** (тема 6) для каждого перехода найди нужные J, K (или D); 3) заполни карты Карно для J_i, K_i; 4) упрости и нарисуй схему.

Как вывести J и K для синхронного счётчика (по таблице возбуждения)

Дано: 3-разрядный синхронный суммирующий счётчик на JK-триггерах
Что нужно сделать: найти выражения для J0, K0, J1, K1, J2, K2
Как записать ответ: формулы J и K через выходы Q

1. Выпиши все состояния Q2Q1Q0 и следующие состояния (счёт +1).
2. Для каждого разряда и каждой строки посмотри переход Q(t) → Q(t+1) и по **таблице возбуждения** (тема 6) выпиши J и K (× — безразлично): 0→0: J=0, K=×; 0→1: J=1, K=×; 1→0: J=×, K=1; 1→1: J=×, K=0.
3. Для каждого входа J или K найди, при каких состояниях он равен 1; клетки с × можно считать как 0, так и как 1 — выбирай так, чтобы формула получилась короче.

Q2Q1Q0	следующее	J2	K2	J1	K1	J0	K0
000	001	1	×	0	×	0	×
001	010	×	1	1	×	0	×
010	011	1	×	×	0	0	×
011	100	×	1	×	1	1	×
100	101	1	×	0	×	×	0
101	110	×	1	1	×	×	0
110	111	1	×	×	0	×	0
111	000	×	1	×	1	×	1

Чтение таблицы. J0 = K0 = 1 во всех строках (младший триггер переключается всегда). J1: единица стоит ровно в строках, где Q0 = 1 и Q1 = 0; там, где Q1 = 1, стоит ×, поэтому J1 = Q0. K1 = 1 в строках, где Q0 = 1, Q1 = 1, а × — где Q1 = 0 ⇒ K1 = Q0. J2: единица только в строке 011 (Q0 = Q1 = 1, Q2 = 0), в строках с Q2 = 1 стоит ×, значит J2 = Q0·Q1; K2 = 1 только в 111 ⇒ K2 = Q0·Q1. **Итого: J0=K0=1; J1=K1=Q0; J2=K2=Q0·Q1** — как в таблице выше. Смысл формул: разряд переключается, когда **все младшие разряды равны 1**.

Деление частоты и ёмкость

n триггеров	Модуль К = 2^n	Макс. число	$f_{Q(n-1)}$
2	4	3	f / 4
3	8	7	f / 8
4	16	15	f / 16
8	256	255	f / 256

Вычитающий счётчик считает 111, 110, …, 000: на триггерах по фронту ↑ достаточно тактировать следующий триггер прямым выходом Q предыдущего. **Реверсивный** — умеет и вверх, и вниз (переключается логикой).

Тема 8 Electronics Workbench: что делать в программе

шпаргалка по лабораторным 3–7 · Logic Converter · Word Generator · Logic Analyzer

На практике схемы собирают в **Electronics Workbench (EWB)**. На РК могут спросить «как построить», «что покажет прибор» или дать скриншот. Ниже всё, что нужно знать.

Панель компонентов	Что там
Basic	соединитель (точка, connector), переключатели, резисторы; точки подписываются через **Component Properties → Label** (A, B, C)
Logic Gates	AND, OR, NOT, NOR, NAND, XOR, XNOR
Digital	готовые блоки: полусумматор (half-adder), полный сумматор (full-adder), MUX, DEC (дешифратор), ENC (шифратор), FF (триггеры RS, D, JK), счётчики, регистры
Indicators	лампочка (Bulb), пробник (Probe), дисплей (Display, показывает число в 16-й системе)
Instruments	Word Generator, Logic Analyzer, Logic Converter, осциллограф и др.

Как собрать схему (лабораторная 3)

1. По выражению определи входы и поставь по одному коннектору на каждый (подпиши A, B, C через Label).
2. Перетащи нужные вентили из Logic Gates и соедини: тяни провод от вывода одного элемента к выводу другого (порядок — по приоритету операций).
3. Выход подключи к Logic Converter (или лампочке). Для проверки меняй входы переключателем.

Logic Converter (Instruments → Logic Converter)

Подключи входы (A, B, C…) и выход схемы к прибору, двойной щелчок открывает окно. Сверху — какие входы включены (до 8), справа — шесть кнопок преобразований:

Кнопка	Что делает	Когда нужна
1. Схема → таблица истинности	по построенной схеме заполняет таблицу	проверить схему
2. Таблица → выражение	записывает СДНФ по таблице	синтез по таблице (задание 2 лаб. 4)
3. Таблица → упрощённое выражение	минимизирует (как карта Карно)	проверка карты Карно
4. Выражение → таблица	строит таблицу по формуле	ввод выражения вручную
5. Выражение → схема	рисует схему на вентилях по формуле	быстрое построение
6. Выражение → схема на NAND	перестраивает всё на И-НЕ	реализация в базисе NAND

> **Запомни:** как проверить минимизацию (лаб. 5): собери схему для исходного выражения и для упрощённого, подключи оба выхода к Logic Converter и построи таблицы. **Таблицы должны совпасть**.

Word Generator и Logic Analyzer (лабораторные 6 и 7)

1. **Word Generator** («генератор слов») подаёт на входы схемы последовательность двоичных слов. Каждый бит поля **Binary** — свой выход; слева в списке то же слово в 16-й системе. Кнопка **Step** — один шаг, **Cycle** — по кругу, **Burst** — серия.
2. **Logic Analyzer** показывает временные диаграммы сигналов: подключи к нему входы и выходы триггера, нажимай Step в генераторе слов.
3. **Лампочки и Display** на выходах сумматора показывают результат: лампочки — биты, Display — число в 16-й системе.

Сумматор, умножитель, триггеры

1. **Полусумматор:** готовый компонент из Digital или схема XOR + AND (либо OR/AND/NOT, как в лаб. 6). Входы A, B от генератора слов, выходы Σ и C на лампочки.
2. **Полный сумматор:** готовый компонент (A, B, Cin → Σ, Cout) либо два полусумматора и ИЛИ.
3. **Многоразрядный сумматор:** цепочка сумматоров, Cout каждого — на Cin следующего. Слагаемые разряды подаются на входы A всех сумматоров и на входы B; результат — на лампочки и Display (пример лаб.: 3-разрядный, и задание на 7-разрядный: 27 + 30).
4. **Умножитель 2×2:** 4 элемента И и 2 полусумматора (см. тему 5). 3×3: 9 И + 3 НА + 3 ФА.
5. **Триггеры:** RS на 2 элементах ИЛИ-НЕ или И-НЕ либо готовый компонент FF из Digital; D и JK — готовые компоненты. Для исследования: Word Generator задаёт S/R (или D, J/K и C), Logic Analyzer — диаграммы. Результаты оформляют

в Word: картинка рабочей области, окна с входными данными и временные диаграммы.

Типичные проблемы: если схема «ничего не показывает» — проверь, что кнопка питания/запуска (вкл/выкл в правом верхнем углу) включена и что все выводы реально подключены (точка на соединении).

Тема 9 Тренировка: задачи с условиями и разбором

в каждой задаче написано: что дано, что нужно сделать и как записать ответ

Как работать с этим разделом. 1) Прочитай условие: блок «Дано» — исходные данные, «Что нужно сделать» — цель, «Как записать ответ» — формат, который ожидает преподаватель. 2) Реши на бумаге **до** просмотра решения. 3) Сверь ответ и ход мысли: если ответ верный, но шаги другие — проверь, нет ли у тебя пропущенных пояснений. 4) Ошибся — вернись в нужную тему и повтори алгоритм. Решения идут сразу после условий.

Условия

A. Системы счисления

1 **Дано:** десятичное число 237_{10}
Что нужно сделать: перевести его в двоичную, восьмеричную и шестнадцатеричную системы
Как записать ответ: три числа с индексами основания: $\ldots_2$, $\ldots_8$, $\ldots_{16}$

2 **Дано:** двоичное число 10111011_2
Что нужно сделать: перевести в десятичную и шестнадцатеричную системы
Как записать ответ: два числа: десятичное и $\ldots_{16}$

3 **Дано:** шестнадцатеричное число $3F7_{16}$
Что нужно сделать: перевести в двоичную, восьмеричную и десятичную системы
Как записать ответ: $\ldots_2$, $\ldots_8$, десятичное

4 **Дано:** десятичная дробь 0,8125 и число 0,1
Что нужно сделать: перевести 0,8125 в двоичную; объяснить, что получится для 0,1
Как записать ответ: двоичная дробь 0,... и короткое пояснение

5 **Дано:** число $110101{,}011_2$
Что нужно сделать: перевести в десятичную, восьмеричную и шестнадцатеричную системы
Как записать ответ: три числа с индексами

6 **Дано:** восьмеричное число 745_8
Что нужно сделать: перевести в двоичную и шестнадцатеричную системы
Как записать ответ: $\ldots_2$ и $\ldots_{16}$

7 **Дано:** число $2A{,}8_{16}$
Что нужно сделать: перевести в десятичную и двоичную системы
Как записать ответ: десятичная дробь и $\ldots_2$

B. Целые числа: прямой и дополнительный код

8 **Дано:** числа 173 и 99; формат — 8-битное целое **без знака**
Что нужно сделать: записать прямой код каждого числа
Как записать ответ: по 8 двоичных цифр на число, лучше группами по 4

9 **Дано:** числа 45, −45, −128, −1; формат — 8-битное целое **со знаком**
Что нужно сделать: записать дополнительный код каждого числа
Как записать ответ: по 8 двоичных цифр на число

10 **Дано:** числа 1000, −1000, 32767, −32768; формат — 16-битное целое со знаком
Что нужно сделать: записать дополнительный код каждого числа (и в hex, и в двоичной)
Как записать ответ: 16 двоичных цифр или 4 hex-цифры

11 **Дано:** дополнительные коды: а) 11010110 (8 бит); б) 01111111 (8 бит); в) 10000000 (8 бит); г) 1111111000001100 (16 бит)
Что нужно сделать: записать десятичное значение каждого числа
Как записать ответ: десятичное число со знаком

12 **Дано:** числа 37 и 58; 8-битный дополнительный код
Что нужно сделать: вычислить 37 − 58 через сложение и перевести результат в десятичную
Как записать ответ: двоичный результат и десятичное число

13 **Дано:** 8-битные числа: а) 90 + 70, интерпретация — со знаком; б) 200 + 100, интерпретация — без знака
Что нужно сделать: сложить и выяснить, правильный ли получился результат
Как записать ответ: результат в 8 битах и вывод: есть ли переполнение

14 **Дано:** 12-битное целое

Что нужно сделать: найти диапазон значений без знака и со знаком
Как записать ответ: два интервала

C. Вещественные числа (Single/Double)

15 **Дано:** десятичное число -5,75; формат Single
Что нужно сделать: записать машинный код числа в шестнадцатеричной системе
Как записать ответ: 8 hex-цифр

16 **Дано:** десятичное число 0,15625; формат Single
Что нужно сделать: записать машинный код числа в шестнадцатеричной системе
Как записать ответ: 8 hex-цифр

17 **Дано:** десятичное число 100,5; формат Double
Что нужно сделать: записать машинный код числа в шестнадцатеричной системе
Как записать ответ: 16 hex-цифр

18 **Дано:** десятичное число 0,2; формат Single
Что нужно сделать: записать машинный код числа в шестнадцатеричной системе
Как записать ответ: 8 hex-цифр

19 **Дано:** код 41C80000 (hex), формат Single
Что нужно сделать: найти десятичное значение числа
Как записать ответ: десятичное число со знаком

20 **Дано:** код BF400000 (hex), формат Single
Что нужно сделать: найти десятичное значение числа
Как записать ответ: десятичное число со знаком

21 **Дано:** код 4005000000000000 (hex), формат Double
Что нужно сделать: найти десятичное значение числа
Как записать ответ: десятичное число со знаком

22 **Дано:** форматы Single и Double
Что нужно сделать: назвать число бит под знак, порядок, мантиссу; смещение; диапазон порядка в Single
Как записать ответ: короткая таблица / несколько фраз

D. Логика, таблицы истинности, упрощение, чтение схем

23 **Дано:** формула $F = (A \oplus B)\cdot\overline{C}$ (три входа A, B, C)
Что нужно сделать: составить таблицу истинности и найти наборы, на которых F = 1
Как записать ответ: таблица 8 строк + список наборов ABC

24 **Дано:** формула $F = A\cdot B + A\cdot\overline{B} + \overline{A}\cdot B$
Что нужно сделать: упростить формулу до минимального вида
Как записать ответ: короткая формула; проверка таблицей

25 **Дано:** формула $F = (A + B)\cdot(A + \overline{B})\cdot(\overline{A} + B)$
Что нужно сделать: упростить
Как записать ответ: короткая формула

26 **Дано:** формула $F = \neg(A + B) + A\cdot B$
Что нужно сделать: упростить и сказать, какой это стандартный вентиль
Как записать ответ: формула и название вентиля

27 **Дано:** таблица: F(A,B,C) = 1 только на наборах 010, 011, 101
Что нужно сделать: записать СДНФ, СКНФ и минимальную ДНФ
Как записать ответ: три формулы

28 **Дано:** функция A ⊕ B (XOR); разрешены только элементы И-НЕ (NAND)

Что нужно сделать: построить схему и сказать, сколько элементов нужно
Как записать ответ: названия сигналов на выходах элементов + число элементов

29 **Дано:** схема: входы A, B идут на NAND и на OR; выходы NAND и OR подаются на AND
Что нужно сделать: определить, какую функцию реализует схема
Как записать ответ: формула и название вентиля (если известно)

30 **Дано:** функция «хотя бы две из трёх входных A, B, C равны 1»
Что нужно сделать: сравнить число элементов в схеме по исходной ДНФ и в упрощённой форме
Как записать ответ: два числа элементов

31 **Дано:** схема: A → НЕ → (выход X); X и B → И → (выход Y); Y и C → ИЛИ → выход F
Что нужно сделать: найти формулу F и значения F на наборах ABC = 010, 100, 111, 001
Как записать ответ: формула и четыре значения

32 **Дано:** функция «F = 1, если хотя бы две из трёх входных A, B, C равны 1»; три варианта формулы: а) AB + AC + BC; б) A⊕B⊕C; в) A + B + C
Что нужно сделать: выбрать правильную формулу, проверяя наборы
Как записать ответ: буква варианта и набор, на котором ошибаются остальные

E. Карты Карно

33 **Дано:** функция F = Σ(1, 3, 5, 6, 7) от трёх переменных A, B, C
Что нужно сделать: минимизировать картой Карно
Как записать ответ: минимальная ДНФ

34 **Дано:** функция F = Σ(0, 2, 5, 7, 8, 10, 13, 15) от четырёх переменных
Что нужно сделать: минимизировать картой Карно
Как записать ответ: минимальная ДНФ

35 **Дано:** функция F = Σ(2, 3, 6, 7, 8, 9, 12, 13)
Что нужно сделать: минимизировать картой Карно
Как записать ответ: минимальная ДНФ

36 **Дано:** таблица истинности F(A,B,C) = 1, 1, 0, 0, 1, 1, 1, 0 (строки по возрастанию: 000, 001, …, 111)
Что нужно сделать: найти минимальную ДНФ
Как записать ответ: формула

37 **Дано:** функция F(A,B,C,D) = 1 на наборах 1, 5, 7, 9, 13; на наборах 3, 11, 15 значение безразлично
Что нужно сделать: найти минимальную ДНФ, используя безразличные наборы
Как записать ответ: формула

F. Сумматоры, умножители, дешифраторы

38 **Дано:** входы A = 1, B = 1 (первый случай) и A = 1, B = 0 (второй)
Что нужно сделать: вычислить выходы полусумматора Σ и C и записать формулы
Как записать ответ: значения Σ, C для двух случаев + формулы

39 **Дано:** числа 1011_2 и 0110_2; 4-разрядный сумматор
Что нужно сделать: сложить по разрядам, показывая A, B, Cin, Σ, Cout
Как записать ответ: таблица разрядов и результат

40 **Дано:** n-разрядное сложение; n = 8, затем n = 4
Что нужно сделать: определить число полусумматоров и полных сумматоров
Как записать ответ: два числа для каждого n

41 **Дано:** A = 7 = 111_2, B = 6 = 110_2; умножитель 3×3
Что нужно сделать: показать частичные произведения и результат
Как записать ответ: частичные произведения столбиком и ответ

Дано: дешифратор 3→8

42 **Что нужно сделать:** определить число элементов И и НЕ; сказать, какой выход активен при входе 101
Как записать ответ: числа элементов + имя выхода

43 **Дано:** мультиплексоры 8:1 и 16:1; код S2S1S0 = 110
Что нужно сделать: найти число управляющих входов, выбранный вход и число MUX 2:1 для сборки MUX 8:1
Как записать ответ: четыре числа/ответа

44 **Дано:** приоритетный шифратор 4→2; входы I3I2I1I0 = 0110
Что нужно сделать: найти выходы A1A0 и V
Как записать ответ: двоичный код и бит валидности

G. Триггеры и счётчики

45 **Дано:** JK-триггер, фронт ↑, Q(0) = 0; пары (J, K) перед шестью фронтами: 10, 00, 11, 01, 11, 10
Что нужно сделать: найти Q после каждого фронта
Как записать ответ: шесть значений Q

46 **Дано:** D-триггер, Q(0) = 0; D перед фронтами: 1, 0, 0, 1, 1, 0
Что нужно сделать: найти Q после каждого фронта
Как записать ответ: шесть значений Q

47 **Дано:** асинхронный RS-триггер на NOR, Q(0) = 0; пары (S, R) по порядку: 10, 00, 01, 00, 10
Что нужно сделать: найти Q после каждого шага; назвать запрещённую комбинацию
Как записать ответ: пять значений Q + запрещённая пара

48 **Дано:** цепочка из 4 T-триггеров (каждый делит частоту на 2); на вход подано 16 кГц
Что нужно сделать: найти частоты на выходах Q0…Q3
Как записать ответ: четыре частоты

49 **Дано:** а) 3-разрядный суммирующий счётчик в состоянии 000; б) 4-разрядный вычитающий из 0000; в) счётчик по модулю 10
Что нужно сделать: найти состояние после 13 / 3 / 23 импульсов
Как записать ответ: три состояния

50 **Дано:** асинхронный счётчик из 4 триггеров; задержка одного триггера 15 нс
Что нужно сделать: найти полную задержку установки кода, предельную частоту и сказать, чем лучше синхронный счётчик
Как записать ответ: время, частота, одно предложение

51 **Дано:** задача: получить счётчик с модулем 6 из асинхронного счётчика
Что нужно сделать: указать число триггеров и условие/элемент сброса
Как записать ответ: число и название элемента с входами

Решения Разбор задач для тренировки

A. Системы счисления

1 **Задача 1. Дано: десятичное число 237_{10}. Нужно: перевести его в двоичную, восьмеричную и шестнадцатеричную системы.**

Идея: делим на 16 (коротко), а двоичную и восьмеричную получаем перегруппировкой битов — это быстрее, чем делить на 2 и на 8.

237 : 16 = 14, остаток 13 → цифра **D**

14 : 16 = 0, остаток 14 → цифра **E**

Читаем остатки **снизу вверх**: 237_{10} = **ED_{16}**

hex: 237 = **ED_{16}**. Каждая hex-цифра → 4 бита: E=1110 D=1101 ⇒ **11101101_2** (ведущие нули не пишем). Для восьмеричной группируем двоичную запись по 3 бита справа налево: 011 101 101 ⇒ **355_8**.

Ответ: **$11101101_2 = 355_8 = ED_{16}$**

2 **Задача 2. Дано: двоичное число 10111011_2. Нужно: перевести в десятичную и шестнадцатеричную системы.**

Идея: hex — группы по 4 бита; десятичное — сумма весов разрядов с единицей.

Тетрады: 1011 | 1011 ⇒ B | B ⇒ **BB_{16}**.

вес	128	64	32	16	8	4	2	1
бит	**1**	0	**1**	**1**	**1**	0	**1**	**1**

Складываем веса там, где 1: 128 + 32 + 16 + 8 + 2 + 1 = **187**. Проверка через hex: 11·16 + 11 = 187 ✓.

Ответ: **187, BB_{16}**

3 **Задача 3. Дано: шестнадцатеричное число $3F7_{16}$. Нужно: перевести в двоичную, восьмеричную и десятичную системы.**

3 = 0011, F = 1111, 7 = 0111 ⇒ **1111110111_2** (ведущие нули отбрасываем). Для восьмеричной группируем по 3 справа: 001 111 110 111 ⇒ 1·7·6·7 ⇒ **1767_8**. Десятичная: $3F7_{16} = 3 \cdot 16^2 + F \cdot 16 + 7 = 768 + 240 + 7 =$ **1015**.

Ответ: **$1111110111_2 = 1767_8 = 1015$**

4 **Задача 4. Дано: десятичная дробь 0,8125 и число 0,1. Нужно: перевести 0,8125 в двоичную; объяснить, что получится для 0,1.**

Идея: дробную часть умножаем на 2, целую часть результата записываем как очередную цифру, дробь оставляем и повторяем.

№	дробь × 2	= результат	цифра
1	0,8125 × 2	1,625	**1**
2	0,625 × 2	1,25	**1**
3	0,25 × 2	0,5	**0**
4	0,5 × 2	1	**1**

Цифры сверху вниз: **$0{,}1101_2$**. Для 0,1 дробь после 4-го шага возвращается к 0,2 — цикл ⇒ 0,0001100110011… бесконечная периодическая дробь, точно записать невозможно.

Ответ: **$0{,}1101_2$; 0,1 — бесконечная**

5 **Задача 5. Дано: число $110101{,}011_2$. Нужно: перевести в десятичную, восьмеричную и шестнадцатеричную системы.**

вес	32	16	8	4	2	1
бит	**1**	**1**	0	**1**	0	**1**

Целая часть: 32 + 16 + 4 + 1 = 53. Дробная: $0{,}011_2 = 0 \cdot ½ + 1 \cdot ¼ + 1 \cdot ⅛ = 0{,}25 + 0{,}125 = 0{,}375$ ⇒ **53,375**.

Восьмеричная — по триадам от запятой: целая 110|101 ⇒ 6|5; дробная 011 ⇒ 3 ⇒ **$65{,}3_8$**. Шестнадцатеричная — тетрады: целая 0011|0101 (дописали 00 слева) ⇒ 3|5; дробная 0110 (дописали 0 справа) ⇒ 6 ⇒ **$35{,}6_{16}$**.

Ответ: **$53{,}375 = 65{,}3_8 = 35{,}6_{16}$**

6 **Задача 6. Дано: восьмеричное число 745_8. Нужно: перевести в двоичную и шестнадцатеричную системы.**

7 = 111, 4 = 100, 5 = 101 ⇒ **111100101_2**. Регруппируем по 4 справа налево: 1|1110|0101 ⇒ допишем 000 слева: 0001 1110 0101 ⇒ 1·E·5 ⇒ **$1E5_{16}$**. Проверка: 7·64 + 4·8 + 5 = 485; 1·256 + 14·16 + 5 = 485 ✓.

Ответ: **$111100101_2 = 1E5_{16}$**

7 **Задача 7. Дано: число $2A,8_{16}$. Нужно: перевести в десятичную и двоичную системы.**

Целая часть: $2A_{16} = 2 \cdot 16 + 10 = 42$. Дробная часть 8_{16}: одна hex-цифра после запятой весит 16^{-1}, поэтому $8/16 = 0{,}5 \Rightarrow$ **42,5**. Двоичная: 2 = 0010, A = 1010, цифра 8 = 1000 ⇒ 00101010,1000 ⇒ **$101010{,}1_2$** (нули слева у целой и справа у дробной части отбрасываем).

Ответ: **$42{,}5 = 101010{,}1_2$**

B. Целые числа: прямой и дополнительный код

8 **Задача 8. Дано: числа 173 и 99; формат — 8-битное целое без знака. Нужно: записать прямой код каждого числа.**

а) 173: Диапазон 0…255: 173 входит. Деление на 16: 173 : 16 = 10, остаток 13; 10 : 16 = 0, остаток 10. Читаем остатки снизу вверх: **AD_{16}**. Каждую цифру в 4 бита: A=1010 + D=1101 → **1010 1101**. Проверка: 128 + 32 + 8 + 4 + 1 = 173 ✓.

б) 99: Диапазон 0…255: 99 входит. Деление на 16: 99 : 16 = 6, остаток 3; 6 : 16 = 0, остаток 6. Читаем остатки снизу вверх: **63_{16}**. Каждую цифру в 4 бита: 6=0110 + 3=0011 → **0110 0011**. Проверка: 64 + 32 + 2 + 1 = 99 ✓.

Ответ: **1010 1101; 0110 0011**

9 **Задача 9. Дано: числа 45, −45, −128, −1; формат — 8-битное целое со знаком. Нужно: записать дополнительный код каждого числа.**

45: Диапазон 0…255: 45 входит. Деление на 16: 45 : 16 = 2, остаток 13; 2 : 16 = 0, остаток 2. Читаем остатки снизу вверх: **$2D_{16}$**. Каждую цифру в 4 бита: 2=0010 + D=1101 → **0010 1101**. Проверка: 32 + 8 + 4 + 1 = 45 ✓. Число положительное, поэтому доп. код = прямой, старший бит 0.

-45: Диапазон -128…127: -45 входит. **Модуль** 45: 45 : 16 = 2, остаток 13; 2 : 16 = 0, остаток 2 → $2D_{16}$ → 0010 1101. **Инверсия:** 1101 0010. **+1:** 1101 0010 + 1 = **1101 0011**. Проверка: $2^8 - 45 = 211 = D3_{16} = 1101\ 0011$ ✓.

-128: Диапазон -128…127: -128 входит. **Модуль** 128: 128 : 16 = 8, остаток 0; 8 : 16 = 0, остаток 8 → 80_{16} → 1000 0000. **Инверсия:** 0111 1111. **+1:** 0111 1111 + 1 = **1000 0000**. Проверка: $2^8 - 128 = 128 = 80_{16} = 1000\ 0000$ ✓. Особый случай: у −128 нет положительного «двойника» в 8 битах; код 1000 0000 получается из 256 − 128 = 128.

-1: Диапазон -128…127: -1 входит. **Модуль** 1: 1 : 16 = 0, остаток 1 → 01_{16} → 0000 0001. **Инверсия:** 1111 1110. **+1:** 1111 1110 + 1 = **1111 1111**. Проверка: $2^8 - 1 = 255 = FF_{16} = 1111\ 1111$ ✓.

Ответ: **0010 1101; 1101 0011; 1000 0000; 1111 1111**

10 **Задача 10. Дано: числа 1000, −1000, 32767, −32768; формат — 16-битное целое со знаком. Нужно: записать дополнительный код каждого числа (и в hex, и в двоичной).**

1000: Диапазон 0…65535: 1000 входит. Деление на 16: 1000 : 16 = 62, остаток 8; 62 : 16 = 3, остаток 14; 3 : 16 = 0, остаток 3. Читаем остатки снизу вверх: **$03E8_{16}$**. Каждую цифру в 4 бита: 0=0000 + 3=0011 + E=1110 + 8=1000 → **0000 0011 1110 1000**. Число положительное, поэтому доп. код = прямой, старший бит 0.

-1000: Диапазон -32768…32767: -1000 входит. **Модуль** 1000: 1000 : 16 = 62, остаток 8; 62 : 16 = 3, остаток 14; 3 : 16 = 0, остаток 3 → $03E8_{16}$ → 0000 0011 1110 1000. **Инверсия:** 1111 1100 0001 0111. **+1:** 1111 1100 0001 0111 + 1 = **1111 1100 0001 1000**. Проверка: $2^{16} - 1000 = 64536 = FC18_{16} = 1111\ 1100\ 0001\ 1000$ ✓.

32767: Диапазон 0…65535: 32767 входит. Деление на 16: 32767 : 16 = 2047, остаток 15; 2047 : 16 = 127, остаток 15; 127 : 16 = 7, остаток 15; 7 : 16 = 0, остаток 7. Читаем остатки снизу вверх: **$7FFF_{16}$**. Каждую цифру в 4 бита: 7=0111 + F=1111 + F=1111 + F=1111 → **0111 1111 1111 1111**. Число положительное, поэтому доп. код = прямой, старший бит 0.

-32768: Диапазон -32768…32767: -32768 входит. **Модуль** 32768: 32768 : 16 = 2048, остаток 0; 2048 : 16 = 128, остаток 0; 128 : 16 = 8, остаток 0; 8 : 16 = 0, остаток 8 → 8000_{16} → 1000 0000 0000 0000. **Инверсия:** 0111 1111 1111 1111. **+1:** 0111 1111 1111 1111 + 1 = **1000 0000 0000 0000**. Проверка: $2^{16} - 32768 = 32768 = 8000_{16} = 1000\ 0000\ 0000\ 0000$ ✓.

Ответ: **03E8; FC18; 7FFF; 8000**

11 **Задача 11. Дано: дополнительные коды: а) 11010110 (8 бит); б) 01111111 (8 бит); в) 10000000 (8 бит); г) 1111111000001100 (16 бит). Нужно: записать десятичное значение каждого числа.**

1101 0110: Группы по 4 бита: 1101 0110 → hex D6. Старший бит **1** → число отрицательное. **Инверсия:** 0010 1001 (hex 29). **+1:** 0010 1010 (hex 2A). $2A_{16} = 2 \cdot 16 + A = 32 + 10 =$ **42**. Ставим минус: **−42**. Проверка: 214 − 256 = -42 ✓.

0111 1111: Группы по 4 бита: 0111 1111 → hex **7F**. Старший бит 0 → число положительное. $7F_{16} = 7 \cdot 16 + F = 112 + 15 =$ **127**.

1000 0000: Группы по 4 бита: 1000 0000 → hex 80. Старший бит **1** → число отрицательное. **Инверсия:** 0111 1111 (hex 7F). **+1:** 1000 0000 (hex 80). $80_{16} = 8 \cdot 16 + 0 = 128 + 0 =$ **128**. Ставим минус: **−128**. Проверка: 128 − 256 = -128 ✓.

1111 1110 0000 1100: Группы по 4 бита: 1111 1110 0000 1100 → hex FE0C. Старший бит **1** → число отрицательное. **Инверсия:** 0000 0001 1111 0011 (hex 01F3). **+1:** 0000 0001 1111 0100 (hex 01F4). $01F4_{16} = 0 \cdot 16^3 + 1 \cdot 16^2 + F \cdot 16 + 4 = 0 + 256 + 240 + 4 =$ **500**. Ставим минус: **−500**. Проверка: 65036 − 65536 = -500 ✓.

Ответ: **−42; 127; −128; −500**

---

12 **Задача 12. Дано: числа 37 и 58; 8-битный дополнительный код. Нужно: вычислить 37 − 58 через сложение и перевести результат в десятичную.**

Идея: A − B = A + (−B), то есть складываем 37 с дополнительным кодом числа −58.

Шаг 1. 37 = 0010 0101. **Шаг 2.** Код −58: модуль 0011 1010 ⇒ инверсия 1100 0101 ⇒ +1 ⇒ **1100 0110**.

```
  00100101
+ 11000110
= 0 11101011
```

Шаг 3. Перенос (девятый бит) отбрасываем: 11101011. Старший бит 1 ⇒ отрицательное. Инверсия 00010100, +1 ⇒ 00010101 = 21 ⇒ **−21**. Проверка: 37 − 58 = −21 ✓.

Ответ: **1110 1011 = −21**

---

13 **Задача 13. Дано: 8-битные числа: а) 90 + 70, интерпретация — со знаком; б) 200 + 100, интерпретация — без знака. Нужно: сложить и выяснить, правильный ли получился результат.**

а) 90 = 0101 1010, 70 = 0100 0110. Сумма 1010 0000. Складывали два **положительных** числа, а старший бит стал 1 ⇒ результат читается как -96, хотя должно быть 160 (в диапазон −128…127 оно не входит). Это **переполнение знаковых** чисел.

б) 200 = 1100 1000, 100 = 0110 0100. Сумма 300 = 1 0010 1100 — 9 бит; девятый (перенос) отбрасывается ⇒ **0010 1100 = 44** вместо 300. Для чисел без знака перенос из старшего разряда = переполнение.

Ответ: **а) -96 (переполнение); б) 44 (переполнение)**

---

14 **Задача 14. Дано: 12-битное целое. Нужно: найти диапазон значений без знака и со знаком.**

Без знака: 12 бит дают $2^{12} = 4096$ комбинаций; значения 0…4095 (то есть $2^{12} - 1$).

Со знаком: старший бит — знаковый, на величину остаётся 11 бит ⇒ от $-2^{11} = -2048$ до $2^{11} - 1 = 2047$.

Ответ: **0…4095; −2048…2047**

C. Вещественные числа (Single/Double)

15 **Задача 15. Дано: десятичное число -5,75; формат Single. Нужно: записать машинный код числа в шестнадцатеричной системе.**

Шаг 1. Знак

Число отрицательное, поэтому бит знака S = **1** (0 = плюс, 1 = минус). Дальше работаем с модулем **5,75**.

Шаг 2. Переводим модуль в двоичную систему

Целая часть 5 (делением на 2 или 16):

5 : 2 = 2, остаток 1 → цифра **1**

2 : 2 = 1, остаток 0 → цифра **0**

1 : 2 = 0, остаток 1 → цифра **1**

Читаем остатки **снизу вверх**: $5_{10} = \mathbf{101}_2$

Дробная часть 0,75 умножаем на 2 и записываем целую часть результата (так мы выделяем двоичные цифры по одной):

№	дробь × 2	= результат	цифра
1	0,75 × 2	1,5	**1**
2	0,5 × 2	1	**1**

Дробь стала 0 — запись закончилась: 0,**11**.

Склеиваем: $5{,}75_{10} = \mathbf{101{,}11}_2$.

Шаг 3. Нормализация (получаем вид $1{,}M \times 2^p$)

Здесь запятая стоит после 3-й цифры; чтобы слева осталась одна «1», сдвигаем её влево на 2 разряда → порядок p = +2. Получили **$1{,}0111 \times 2^2$**.

Почему так: сдвиг запятой на 1 разряд влево — это деление на 2, поэтому, чтобы число не изменилось, к порядку прибавляем 1. Сдвиг вправо — наоборот, порядок уменьшается.

Шаг 4. Порядок со смещением

E = p + 127 = 2 + 127 = **129**. Откуда 127: смещение = $2^{8-1} - 1 = 127$. Двоичная запись E на 8 бит: 129 = 128 + 1 → `10000001`.

Шаг 5. Мантисса M

Берём всё, что стоит **после запятой** в нормализованной записи (единицу перед запятой **не пишем** — это скрытый бит): `0111`. До 23 бит дописываем нули справа: `01110000000000000000000` (нули справа в дробной части не меняют значения).

Шаг 6. Собираем и переводим в hex

Склеиваем поля S | E | M: `1 | 10000001 | 01110000000000000000000`. Длина кода 32 бита.

группа из 4 бит	`1100`	`0000`	`1011`	`1000`	`0000`	`0000`	`0000`	`0000`
hex-цифра	**C**	**0**	**B**	**8**	**0**	**0**	**0**	**0**

Ответ: C0B80000$_{16}$ (для проверки подставьте результат в алгоритм из раздела 2.4 — получится исходное число).

Ответ: **C0B80000**

---

16 **Задача 16. Дано: десятичное число 0,15625; формат Single. Нужно: записать машинный код числа в шестнадцатеричной системе.**

Шаг 1. Знак

Число положительное, поэтому бит знака S = **0** (0 = плюс, 1 = минус). Дальше работаем с модулем **0,15625**.

Шаг 2. Переводим модуль в двоичную систему

Целая часть 0 → в двоичной 0.

Дробная часть 0,15625 умножаем на 2 и записываем целую часть результата (так мы выделяем двоичные цифры по одной):

№	дробь × 2	= результат	цифра
1	0,15625 × 2	0,3125	**0**
2	0,3125 × 2	0,625	**0**
3	0,625 × 2	1,25	**1**
4	0,25 × 2	0,5	**0**
5	0,5 × 2	1	**1**

Дробь стала 0 — запись закончилась: 0,**00101**.

Склеиваем: $0{,}15625_{10}$ = **0,00101**$_2$.

Шаг 3. Нормализация (получаем вид $1{,}M \times 2^p$)

Здесь после запятой 2 нуля и потом первая «1»; сдвигаем запятую вправо на 3 → порядок p = −3. Получили **1,01 × 2^{-3}**.

Почему так: сдвиг запятой на 1 разряд влево — это деление на 2, поэтому, чтобы число не изменилось, к порядку прибавляем 1. Сдвиг вправо — наоборот, порядок уменьшается.

Шаг 4. Порядок со смещением

E = p + 127 = −3 + 127 = **124**. Откуда 127: смещение = $2^{8-1} - 1 = 127$. Двоичная запись E на 8 бит: 124 = 64 + 32 + 16 + 8 + 4 → `01111100`.

Шаг 5. Мантисса M

Берём всё, что стоит **после запятой** в нормализованной записи (единицу перед запятой **не пишем** — это скрытый бит): `01`. До 23 бит дописываем нули справа: `01000000000000000000000` (нули справа в дробной части не меняют значения).

Шаг 6. Собираем и переводим в hex

Склеиваем поля S | E | M: `0 | 01111100 | 01000000000000000000000`. Длина кода 32 бита.

группа из 4 бит	`0011`	`1110`	`0010`	`0000`	`0000`	`0000`	`0000`	`0000`
hex-цифра	**3**	**E**	**2**	**0**	**0**	**0**	**0**	**0**

Ответ: 3E200000$_{16}$ (для проверки подставьте результат в алгоритм из раздела 2.4 — получится исходное число).

Ответ: **3E200000**

---

17 **Задача 17. Дано: десятичное число 100,5; формат Double. Нужно: записать машинный код числа в шестнадцатеричной системе.**

Шаг 1. Знак

Число положительное, поэтому бит знака S = **0** (0 = плюс, 1 = минус). Дальше работаем с модулем **100,5**.

Шаг 2. Переводим модуль в двоичную систему

Целая часть 100 (делением на 2 или 16):

100 : 16 = 6, остаток 4 → цифра **4**

6 : 16 = 0, остаток 6 → цифра **6**

Читаем остатки **снизу вверх**: 100_{10} = **64_{16}**

Это hex **64**; каждая цифра → 4 бита: 6=0110 4=0100 → **1100100** (ведущие нули не пишем).

Дробная часть 0,5 умножаем на 2 и записываем целую часть результата (так мы выделяем двоичные цифры по одной):

№	дробь × 2	= результат	цифра
1	0,5 × 2	1	**1**

Дробь стала 0 — запись закончилась: 0,**1**.

Склеиваем: $100{,}5_{10}$ = **$1100100{,}1_{2}$**.

Шаг 3. Нормализация (получаем вид $1{,}M \times 2^{p}$)

Здесь запятая стоит после 7-й цифры; чтобы слева осталась одна «1», сдвигаем её влево на 6 разрядов → порядок p = +6. Получили **$1{,}1001001 \times 2^{6}$**.

Почему так: сдвиг запятой на 1 разряд влево — это деление на 2, поэтому, чтобы число не изменилось, к порядку прибавляем 1. Сдвиг вправо — наоборот, порядок уменьшается.

Шаг 4. Порядок со смещением

E = p + 1023 = 6 + 1023 = **1029**. Откуда 1023: смещение = $2^{11-1} - 1 = 1023$. Двоичная запись E на 11 бит: 1029 = 1024 + 4 + 1 → `10000000101`.

Шаг 5. Мантисса M

Берём всё, что стоит **после запятой** в нормализованной записи (единицу перед запятой **не пишем** — это скрытый бит): `1001001`. До 52 бит дописываем нули справа: `1001001000000000000000000000000000000000000000000000` (нули справа в дробной части не меняют значения).

Шаг 6. Собираем и переводим в hex

Склеиваем поля S | E | M: `0 | 10000000101 | 1001001000000000000000000000000000000000000000000000`. Длина кода 64 бита.

группа из 4 бит	`0100`	`0000`	`0101`	`1001`	`0010`	`0000`	`0000`	`0000`	`0000`	`0000`	`0000`	`0000`	`0000`	`0000`	`0000`	`0000`
hex-цифра	**4**	**0**	**5**	**9**	**2**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**

Ответ: **4059200000000000_{16}** (для проверки подставьте результат в алгоритм из раздела 2.4 — получится исходное число).

Ответ: **4059200000000000**

---

18 Задача 18. Дано: десятичное число 0,2; формат Single. Нужно: записать машинный код числа в шестнадцатеричной системе.

Шаг 1. Знак

Число положительное, поэтому бит знака S = **0** (0 = плюс, 1 = минус). Дальше работаем с модулем **0,2**.

Шаг 2. Переводим модуль в двоичную систему

Целая часть 0 → в двоичной 0.

Дробная часть 0,2 умножаем на 2 и записываем целую часть результата (так мы выделяем двоичные цифры по одной):

№	дробь × 2	= результат	цифра
1	0,2 × 2	0,4	**0**
2	0,4 × 2	0,8	**0**
3	0,8 × 2	1,6	**1**
4	0,6 × 2	1,2	**1**

№	дробь × 2	= результат	цифра
5	0,2 × 2	0,4	**0**
6	0,4 × 2	0,8	**0**
7	0,8 × 2	1,6	**1**
8	0,6 × 2	1,2	**1**

Первые 8 цифр: **00110011**. Продолжая так же, получаем длинную запись: **0011001100110011001100110011001100110011001100110011...** (для Single нужно примерно 25–30 цифр, для Double — 55–60; такую дробь считают в Windows-калькуляторе или программой, а на экзамене чаще дают числа с конечной двоичной дробью).

Склеиваем: $0{,}2_{10}$ = **0,0011001100110011001100110011...**$_2$.

Шаг 3. Нормализация (получаем вид 1,M × 2^p)

Здесь после запятой 2 нуля и потом первая «1»; сдвигаем запятую вправо на 3 → порядок p = −3. Получили **1,10011001100110011001100110011... × 2^{-3}**.

Почему так: сдвиг запятой на 1 разряд влево — это деление на 2, поэтому, чтобы число не изменилось, к порядку прибавляем 1. Сдвиг вправо — наоборот, порядок уменьшается.

Шаг 4. Порядок со смещением

E = p + 127 = −3 + 127 = **124**. Откуда 127: смещение = 2^{8-1} − 1 = 127. Двоичная запись E на 8 бит: 124 = 64 + 32 + 16 + 8 + 4 → `01111100`.

Шаг 5. Мантисса M

Берём всё, что стоит **после запятой** в нормализованной записи (единицу перед запятой **не пишем** — это скрытый бит): цифр больше, чем 23. Оставляем первые 23: `10011001100110011001100`. Следующие за ними цифры: `110011...`. Первая из отброшенных цифр — 1 (и дальше не одни нули), значит остаток больше половины младшего бита → **округляем вверх**: прибавляем 1 к последнему биту. Итог M = `10011001100110011001101`.

Шаг 6. Собираем и переводим в hex

Склеиваем поля S | E | M: `0 | 01111100 | 10011001100110011001101`. Длина кода 32 бита.

группа из 4 бит	`0011`	`1110`	`0100`	`1100`	`1100`	`1100`	`1100`	`1101`
hex-цифра	**3**	**E**	**4**	**C**	**C**	**C**	**C**	**D**

Ответ: **3E4CCCCD**$_{16}$ (для проверки подставьте результат в алгоритм из раздела 2.4 — получится исходное число).

Ответ: **3E4CCCCD**

---

19 Задача 19. Дано: код 41C80000 (hex), формат Single. Нужно: найти десятичное значение числа.

Шаг 1. Hex → биты

Каждую hex-цифру записываем четырьмя битами: 4=0100 1=0001 C=1100 8=1000 0=0000 0=0000 0=0000 0=0000

Все 32 бита: `0100 0001 1100 1000 0000 0000 0000 0000`

Шаг 2. Режем на поля

Single: 1 + 8 + 23 бит. S = `0`, E = `10000011`, M = `10010000000000000000000`.

Шаг 3. Знак и порядок

S = 0 → число **положительное**. E = 10000011_2 = 131. Настоящий порядок p = E − 127 = 131 − 127 = **4** (вычитаем смещение, которое прибавляли при записи).

Шаг 4. Возвращаем скрытую единицу

Число = 1,M × 2^p = 1,1001 × 2^4; запятую сдвигаем вправо на 4 разряда: **11001_2**.

Шаг 5. В десятичную систему

Складываем веса разрядов, где стоит 1: 16 + 8 + 1 = **25**. С учётом знака: **25**.

Ответ: **25**

---

20 Задача 20. Дано: код BF400000 (hex), формат Single. Нужно: найти десятичное значение числа.

Шаг 1. Hex → биты

Каждую hex-цифру записываем четырьмя битами: B=1011 F=1111 4=0100 0=0000 0=0000 0=0000 0=0000 0=0000

Все 32 бита: `1011 1111 0100 0000 0000 0000 0000 0000`

Шаг 2. Режем на поля

Single: 1 + 8 + 23 бит. S = `1`, E = `01111110`, M = `10000000000000000000000`.

Шаг 3. Знак и порядок

S = 1 → число **отрицательное**. E = 01111110_2 = 126. Настоящий порядок p = E − 127 = 126 − 127 = **−1** (вычитаем смещение, которое прибавляли при записи).

Шаг 4. Возвращаем скрытую единицу

Число = 1,M × 2^p = 1,1 × 2^{-1}; запятую сдвигаем влево на 1 разряд: **$0{,}11_2$**.

Шаг 5. В десятичную систему

Складываем веса разрядов, где стоит 1: $2^{-1} + 2^{-2}$ = **0,75**. С учётом знака: **−0,75**.

Ответ: **-0,75**

21 **Задача 21. Дано: код 4005000000000000 (hex), формат Double. Нужно: найти десятичное значение числа.**

Шаг 1. Hex → биты

Каждую hex-цифру записываем четырьмя битами: 4=0100 0=0000 0=0000 5=0101 0=0000 0=0000 0=0000 0=0000 …

Все 64 бита: `0100 0000 0000 0101 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000`

Шаг 2. Режем на поля

Double: 1 + 11 + 52 бит. S = `0`, E = `10000000000`, M = `0101000000000000000000000000000000000000000000000000`.

Шаг 3. Знак и порядок

S = 0 → число **положительное**. E = 10000000000_2 = 1024. Настоящий порядок p = E − 1023 = 1024 − 1023 = **1** (вычитаем смещение, которое прибавляли при записи).

Шаг 4. Возвращаем скрытую единицу

Число = 1,M × 2^p = 1,0101 × 2^1; запятую сдвигаем вправо на 1 разряд: **10,101**$_2$.

Шаг 5. В десятичную систему

Складываем веса разрядов, где стоит 1: $2 + 2^{-1} + 2^{-3}$ = **2,625**. С учётом знака: **2,625**.

Ответ: **2,625**

22 **Задача 22. Дано: форматы Single и Double. Нужно: назвать число бит под знак, порядок, мантиссу; смещение; диапазон порядка в Single.**

Single: S = 1, E = 8, M = 23 бит (всего 32), смещение $2^7 - 1$ = **127**. Double: 1 + 11 + 52 = 64 бита, смещение $2^{10} - 1$ = **1023**.

В Single зарезервированы E = 0 (ноль и денормализованные числа) и E = 255 (бесконечность и NaN), поэтому у обычных чисел E = 1…254 и порядок p = E − 127 = **−126 … +127**.

Ответ: **см. разбор**

D. Логика, таблицы истинности, упрощение, чтение схем

23 **Задача 23. Дано: формула F = (A ⊕ B)·$\overline{C}$ (три входа A, B, C). Нужно: составить таблицу истинности и найти наборы, на которых F = 1.**

Идея: считаем по столбцам: сначала A⊕B, потом инверсию C, потом И.

A	B	C	A⊕B	$\overline{C}$	F
0	0	0	**0**	**1**	**0**
0	0	1	**0**	**0**	**0**
0	1	0	**1**	**1**	**1**
0	1	1	**1**	**0**	**0**
1	0	0	**1**	**1**	**1**
1	0	1	**1**	**0**	**0**
1	1	0	**0**	**1**	**0**
1	1	1	**0**	**0**	**0**

Считаем по столбцам: сначала A⊕B, затем инверсию C, затем И. F = 1 при наборах **010 и 100** (A ≠ B и C = 0).

Ответ: **010, 100**

24 **Задача 24. Дано: формула F = A·B + A·$\overline{B}$ + $\overline{A}$·B. Нужно: упростить формулу до минимального вида.**

Идея: склеить первые два слагаемых, затем применить A + ĀB = A + B.

AB + A$\overline{B}$ = A (склеивание по B). Тогда F = A + $\overline{A}$B = **A + B** (частное правило A + ĀB = A + B). Проверка: F = 0 только при A = B = 0, как у ИЛИ ✓.

Ответ: **A + B**

25 **Задача 25. Дано: формула F = (A + B)·(A + B)·(A + B). Нужно: упростить.**

Идея: склеивание для первой пары скобок, потом раскрыть остаток.

(A + B)(A + $\overline{B}$) = A (склеивание). Остаётся A·($\overline{A}$ + B) = A$\overline{A}$ + AB = 0 + AB = **AB**.

Ответ: **AB**

---

26 **Задача 26. Дано: формула F = ¬(A + B) + A·B. Нужно: упростить и сказать, какой это стандартный вентиль.**

Идея: Де Морган для первой части, затем сравнить с известными таблицами.

По Де Моргану ¬(A+B) = $\overline{A}$·$\overline{B}$. F = $\overline{A}\,\overline{B}$ + AB — равна 1 при одинаковых входах → это **XNOR (эквивалентность)**: F = ¬(A ⊕ B).

Ответ: **XNOR**

---

27 **Задача 27. Дано: таблица: F(A,B,C) = 1 только на наборах 010, 011, 101. Нужно: записать СДНФ, СКНФ и минимальную ДНФ.**

Идея: СДНФ по единицам, СКНФ по нулям, затем склеивание.

СДНФ (по единицам, 0 → с инверсией): $\overline{A}B\overline{C}$ + $\overline{A}$BC + AB$\overline{C}$.

СКНФ (по нулям 000, 001, 100, 110, 111; 1 → с инверсией): (A + B + C) · (A + B + $\overline{C}$) · ($\overline{A}$ + B + C) · ($\overline{A}$ + $\overline{B}$ + C) · ($\overline{A}$ + $\overline{B}$ + $\overline{C}$).

МДНФ: $\overline{A}$ B $\overline{C}$ + $\overline{A}$ B C = $\overline{A}$ B (склеивание по C), A $\overline{B}$ C остаётся: **$\overline{A}$ B + A $\overline{B}$ C** (на карте Карно: пара клеток 2,3 и одиночная 5).

Ответ: **$\bar{A}$B + A$\bar{B}$C**

---

28 **Задача 28. Дано: функция A ⊕ B (XOR); разрешены только элементы И-НЕ (NAND). Нужно: построить схему и сказать, сколько элементов нужно.**

Идея: t = NAND(A,B); затем NAND(A,t) и NAND(B,t); последний NAND объединяет их.

t = NAND(A,B); u = NAND(A,t); v = NAND(B,t); Y = NAND(u,v). Проверка: A=0,B=1: t=1, u = NAND(0,1) = 1, v = NAND(1,1) = 0, Y = NAND(1,0) = 1 ✓. Для A=B: u и v одинаковы или Y=0. Нужно **4 элемента И-НЕ**.

Ответ: **4 NAND**

---

29 **Задача 29. Дано: схема: входы A, B идут на NAND и на OR; выходы NAND и OR подаются на AND. Нужно: определить, какую функцию реализует схема.**

Идея: выпиши выражение на выходе каждого вентиля и проверь четыре набора.

Y = ¬(A·B)·(A + B). Таблица: A,B = 00 → 1·0 = 0; 01 → 1·1 = 1; 10 → 1; 11 → 0·1 = 0. Это **A ⊕ B** (XOR).

Ответ: **XOR**

---

30 **Задача 30. Дано: функция «хотя бы две из трёх входных A, B, C равны 1». Нужно: сравнить число элементов в схеме по исходной ДНФ и в упрощённой форме.**

Идея: ДНФ → схема по ДНФ; затем карта Карно → схема по МДНФ.

Исходная: 4 элемента И с тремя входами + 3 НЕ + ИЛИ на 4 входа. Упрощённая Q = AB + AC + BC: **3 элемента И с двумя входами + одно ИЛИ на 3 входа** (инверторов нет) — схема намного дешевле.

Ответ: **см. разбор**

---

31 **Задача 31. Дано: схема: A → НЕ → (выход X); X и B → И → (выход Y); Y и C → ИЛИ → выход F. Нужно: найти формулу F и значения F на наборах ABC = 010, 100, 111, 001.**

Идея: идём слева направо, на каждом вентиле записываем выражение: X = $\overline{A}$; Y = X·B = $\overline{A}$·B; F = Y + C. Формула **F = $\overline{A}$·B + C**.

ABC	X = $\overline{A}$	Y = X·B	F = Y + C
010	1	1	1 + 0 = **1**
100	0	0	0 + 0 = **0**
111	0	0	0 + 1 = **1**
001	1	0	0 + 1 = **1**

F = 1 на 010, 111, 001 и F = 0 на 100.

Ответ: **F = $\bar{A}$·B + C; 1, 0, 1, 1**

---

32 **Задача 32. Дано: функция «F = 1, если хотя бы две из трёх входных A, B, C равны 1»; три варианта формулы: а) AB + AC + BC; б) A⊕B⊕C; в) A + B + C. Нужно: выбрать правильную формулу, проверяя наборы.**

Идея: не доказываем алгебраически, а подставляем наборы, на которых варианты различаются.

ABC	нужно F	а) A·B + A·C + B·C	б) A ⊕ B ⊕ C	в) A + B + C
001	0	✓ (0)	✗ (1)	✗ (1)
011	1	✓ (1)	✗ (0)	✓ (1)
110	1	✓ (1)	✗ (0)	✓ (1)

Набор 001: нужно 0; формулы «б» и «в» дают 1 ⇒ они отпадают. Остаётся «а»; на 011 и 110 она даёт 1 ✓.

Ответ: **а**

E. Карты Карно

33 **Задача 33. Дано: функция F = Σ(1, 3, 5, 6, 7) от трёх переменных A, B, C. Нужно: минимизировать картой Карно.**

Идея: нарисуй карту 2×4, найди группы из 4 и 2.

C\AB	00	01	11	10
0	0	0	1	0
1	1	1	1	1

- клетки 1,3,5,7 → **C**
- клетки 6,7 → **AB**

Группа 1,3,5,7: C = 1 постоянно → C. Группа 6,7: A = 1, B = 1 → AB. **F = C + AB**.

Ответ: **C + AB**

34 **Задача 34. Дано: функция F = Σ(0, 2, 5, 7, 8, 10, 13, 15) от четырёх переменных. Нужно: минимизировать картой Карно.**

Идея: угловые клетки и центр — две группы по 4.

CD\AB	00	01	11	10
00	1	0	0	1
01	0	1	1	0
11	0	1	1	0
10	1	0	0	1

- клетки 0,2,8,10 → $\overline{B}\,\overline{D}$
- клетки 5,7,13,15 → **BD**

Углы 0,2,8,10: B = 0, D = 0 → $\overline{B}\,\overline{D}$. Центр 5,7,13,15: B = 1, D = 1 → BD. **F = $\overline{B}\,\overline{D}$ + BD** — это XNOR(B, D).

Ответ: **$\overline{B}\,\overline{D}$ + BD**

35 **Задача 35. Дано: функция F = Σ(2, 3, 6, 7, 8, 9, 12, 13). Нужно: минимизировать картой Карно.**

Идея: две группы по 4, одна из них не прямоугольник «в углу».

CD\AB	00	01	11	10
00	0	0	1	1
01	0	0	1	1
11	1	1	0	0
10	1	1	0	0

- клетки 2,3,6,7 → $\overline{A}\,C$
- клетки 8,9,12,13 → $A\overline{C}$

Группа 2,3,6,7: A = 0, C = 1 → $\overline{A}C$. Группа 8,9,12,13: A = 1, C = 0 → $A\overline{C}$. **F = $\overline{A}C + A\overline{C}$** = A ⊕ C.

Ответ: **$\overline{A}C + A\overline{C}$**

36 **Задача 36. Дано: таблица истинности F(A,B,C) = 1, 1, 0, 0, 1, 1, 1, 0 (строки по возрастанию: 000, 001, ..., 111). Нужно: найти минимальную ДНФ.**

Идея: сначала найти номера строк с единицами, потом карта.

Минтермы Σ(0, 1, 4, 5, 6).

C\AB	00	01	11	10

- клетки 0,1,4,5 → $\overline{B}$

0	0 **1**	2 **0**	6 **1**	4 **1**
1	1 **1**	3 **0**	7 **0**	5 **1**

● клетки 4,6 → **AC**

Группа 0,1,4,5: B = 0 → $\overline{B}$. Группа 4,6: A = 1, C = 0 → $A\overline{C}$. **F = $\overline{B}$ + $A\overline{C}$**.

Ответ: **$\overline{B}$ + $A\overline{C}$**

---

37 **Задача 37. Дано: функция F(A,B,C,D) = 1 на наборах 1, 5, 7, 9, 13; на наборах 3, 11, 15 значение безразлично. Нужно: найти минимальную ДНФ, используя безразличные наборы.**

Идея: безразличные клетки добавляем к группам, если они их укрупняют.

CD\AB	00	01	11	10
00	0 **0**	4 **0**	12 **0**	8 **0**
01	1 **1**	5 **1**	13 **1**	9 **1**
11	3 **X**	7 **1**	15 **X**	11 **X**
10	2 **0**	6 **0**	14 **0**	10 **0**

● клетки 1,3,5,7,9,11,13,15 → **D**

Безразличные клетки X берём в группы, если они укрупняют группу. **F = D**.

Ответ: **D**

F. Сумматоры, умножители, дешифраторы

38 **Задача 38. Дано: входы A = 1, B = 1 (первый случай) и A = 1, B = 0 (второй). Нужно: вычислить выходы полусумматора Σ и C и записать формулы.**

Идея: сложи числа в двоичной и разложи результат на два бита.

Σ = A ⊕ B, C = A·B. При A = 1, B = 1: Σ = 0, C = 1 ($1 + 1 = 10_2$). При A = 1, B = 0: Σ = 1, C = 0.

Ответ: **Σ=0,C=1; Σ=1,C=0**

---

39 **Задача 39. Дано: числа 1011_2 и 0110_2; 4-разрядный сумматор. Нужно: сложить по разрядам, показывая A, B, Cin, Σ, Cout.**

Идея: идём от младшего разряда к старшему, перенос передаём дальше.

Разряд	A	B	Cin	Σ	Cout
2^0	1	0	0	**1**	0
2^1	1	1	0	**0**	1
2^2	0	1	1	**0**	1
2^3	1	0	1	**0**	1

Результат: 10001_2 = **17**.

Ответ: **10001_2 = 17**

---

40 **Задача 40. Дано: n-разрядное сложение; n = 8, затем n = 4. Нужно: определить число полусумматоров и полных сумматоров.**

Идея: младший разряд без входного переноса — полусумматор.

Младший разряд — полусумматор, остальные — полные: для 8 разрядов **1 HA + 7 FA**, для 4 разрядов **1 HA + 3 FA** (результат на 1 бит длиннее). Можно и все полные сумматоры с Cin = 0 у младшего.

Ответ: **1 HA + 7 FA; 1 HA + 3 FA**

---

41 **Задача 41. Дано: A = 7 = 111_2, B = 6 = 110_2; умножитель 3×3. Нужно: показать частичные произведения и результат.**

Идея: каждый бит множителя даёт сдвинутую копию множимого или нули.

A = 111, B = 110. b0 = 0 → 000; b1 = 1 → 111 со сдвигом 1; b2 = 1 → 111 со сдвигом 2.

```
   111
 × 110
 -----
```

```
    000
   111
  111
 ------
 101010
```

Результат **101010 = 42** (проверено моделью умножителя).

Ответ: **101010**

---

42 **Задача 42. Дано: дешифратор 3→8. Нужно: определить число элементов И и НЕ; сказать, какой выход активен при входе 101.**

Идея: каждый выход — минтерм из трёх переменных.

Каждый из 8 выходов — минтерм из трёх переменных: **8 элементов И с 3 входами и 3 инвертора**. Код $101_2 = 5$ → активен выход **D5** = A2·$\overline{A1}$·A0.

Ответ: **8 AND3 + 3 NOT; D5**

---

43 **Задача 43. Дано: мультиплексоры 8:1 и 16:1; код S2S1S0 = 110. Нужно: найти число управляющих входов, выбранный вход и число MUX 2:1 для сборки MUX 8:1.**

Идея: 2^n входов ⇔ n управляющих; код 110 = число 6.

MUX 8:1: **3** ($2^3 = 8$), MUX 16:1: **4**. Код $110_2 = 6$ → выбран **D6**. Каскад: 4 + 2 + 1 = **7** MUX 2:1.

Ответ: **3; 4; D6; 7**

---

44 **Задача 44. Дано: приоритетный шифратор 4→2; входы I3I2I1I0 = 0110. Нужно: найти выходы A1A0 и V.**

Идея: побеждает старший активный вход.

Побеждает старший активный вход I2 → A1A0 = 10 (номер 2), V = 1. Обычный шифратор по формулам A1 = I2 + I3 = 1, A0 = I1 + I3 = 1 дал бы 11 — ошибка (номер 3, хотя вход 3 не активен).

Ответ: **A1A0 = 10, V = 1**

G. Триггеры и счётчики

45 **Задача 45. Дано: JK-триггер, фронт ↑, Q(0) = 0; пары (J, K) перед шестью фронтами: 10, 00, 11, 01, 11, 10. Нужно: найти Q после каждого фронта.**

Идея: используем таблицу JK: 00 хранит, 10 Set, 01 Reset, 11 переключает.

(10) Q=1 → (00) Q=1 → (11) Q=0 → (01) Q=0 → (11) Q=1 → (10) Q=1

10 — Set (Q = 1); 00 — хранение; 11 — переключение; 01 — Reset; 11 — переключение; 10 — Set. Ответ: **1, 1, 0, 0, 1, 1**.

Ответ: **1, 1, 0, 0, 1, 1**

---

46 **Задача 46. Дано: D-триггер, Q(0) = 0; D перед фронтами: 1, 0, 0, 1, 1, 0. Нужно: найти Q после каждого фронта.**

Идея: Q после фронта равно D до фронта.

Q(t+1) = D (значение до фронта): **1, 0, 0, 1, 1, 0**. Триггер просто запоминает вход.

Ответ: **1,0,0,1,1,0**

---

47 **Задача 47. Дано: асинхронный RS-триггер на NOR, Q(0) = 0; пары (S, R) по порядку: 10, 00, 01, 00, 10. Нужно: найти Q после каждого шага; назвать запрещённую комбинацию.**

Идея: на NOR активная единица; 00 — хранение.

10 → Q = 1; 00 → хранит 1; 01 → Q = 0; 00 → хранит 0; 10 → Q = 1. Ответ: **1, 1, 0, 0, 1**. Запрещено S = R = 1 (оба выхода 0, после снятия сигналов состояние непредсказуемо).

Ответ: **1, 1, 0, 0, 1**

---

48 **Задача 48. Дано: цепочка из 4 T-триггеров (каждый делит частоту на 2); на вход подано 16 кГц. Нужно: найти частоты на выходах Q0…Q3.**

Идея: каждый разряд вдвое медленнее предыдущего.

Каждый триггер делит частоту на 2: **8, 4, 2, 1 кГц**. Ёмкость счётчика $2^4 = 16$.

Ответ: **8, 4, 2, 1 кГц**

---

49 **Задача 49. Дано: а) 3-разрядный суммирующий счётчик в состоянии 000; б) 4-разрядный вычитающий из 0000; в) счётчик по модулю 10. Нужно: найти состояние после 13 / 3 / 23 импульсов.**

Идея: остаток от деления числа импульсов на модуль.

3 разряда: 13 mod 8 = 5 → **101**. Вычитающий 4-разрядный: 0 − 3 = −3 ≡ 16 − 3 = 13 → **1101**. Модуль 10: 23 mod 10 = **3** (0011).

Ответ: **101; 1101; 3**

---

50 **Задача 50. Дано: асинхронный счётчик из 4 триггеров; задержка одного триггера 15 нс. Нужно: найти полную задержку установки кода, предельную частоту и сказать, чем лучше синхронный счётчик.**

Идея: задержки складываются.

Задержки складываются: 4 × 15 = **60 нс** (старший разряд устанавливается последним). Период тактов должен быть не меньше 60 нс → f_{max} ≈ 1/60 нс ≈ **16,7 МГц**. В синхронном счётчике все триггеры тактируются одновременно, задержка ≈ одного триггера (плюс логика) → частота выше.

Ответ: **60 нс; ≈16,7 МГц**

---

51 **Задача 51. Дано: задача: получить счётчик с модулем 6 из асинхронного счётчика. Нужно: указать число триггеров и условие/элемент сброса.**

Идея: $6 = 110_2$ — первое «лишнее» состояние.

Нужно 3 триггера ($2^2 = 4 < 6 \le 8$). Состояния 0…5; первое запрещённое состояние 6 = 110. Сброс: **NAND(Q2, Q1)** → на асинхронные входы сброса всех триггеров. При коде 110 (Q2 = Q1 = 1) все триггеры обнуляются.

Ответ: **3 триггера, NAND(Q2,Q1)**

Финиш Шпаргалка: что помнить без подсказок

Числа

- 10→P: делим, остатки снизу вверх. Дробь ×P, цифры сверху вниз.
- 2 ↔ 16: группы по 4, 2 ↔ 8: по 3; в целой части нули слева, в дробной — справа.
- Unsigned N бит: 0…2^N−1. Signed: -2^{N-1}…2^{N-1}−1.
- Доп. код отрицательного: модуль → инверсия → +1. Обратно: старший бит 1 → инверсия → +1 → минус.
- A−B = A + доп.код(−B), перенос из старшего отбрасывается.

IEEE 754

- Single 1|8|23, смещение 127, 8 hex-цифр. Double 1|11|52, смещение 1023, 16 hex-цифр.
- Число → 1,M × 2^p; E = p + смещение; M = дробная часть без скрытой 1.
- Обратно: p = E − смещение, вернуть скрытую 1, сдвинуть запятую.
- Single: p от −126 до 127; E = 0 → ноль, E = 255 → ∞/NaN.

Логика

- NAND = ¬(AB); NOR = ¬(A+B); XOR = 1 при различных; XNOR — при одинаковых.
- Де Морган: ¬(AB) = $\overline{A} + \overline{B}$; ¬(A+B) = $\overline{A}\,\overline{B}$.
- Склеивание $AB + A\overline{B} = A$; поглощение $A + AB = A$; $A + \overline{A}B = A + B$.
- ДНФ — по единицам (0 → с чертой), КНФ — по нулям (1 → с чертой).
- Карта Карно: порядок 00-01-11-10; группы 1, 2, 4, 8; края замкнуты; читаем постоянные переменные.

Схемы

- HA: Σ = A⊕B, C = AB. FA: Σ = A⊕B⊕Cin, Cout = AB + Cin(A⊕B). FA = 2 HA + OR.
- n-разрядный сумматор: 1 HA + (n−1) FA. Умножитель 2×2: 4 И + 2 HA. 3×3: 9 И + 3 HA + 3 FA.
- Дешифратор n→2^n: 2^n элементов И. Шифратор 4→2: A1 = I2+I3, A0 = I1+I3. MUX 2^n:1 — n управляющих входов.

Триггеры

- RS: S=1 → Q=1, R=1 → Q=0, 00 хранение, 11 запрещено (на NOR). На NAND — наоборот (активный 0).
- D: Q(t+1)=D. T: Q(t+1)=T⊕Q, делит f на 2. JK: 11 = toggle; $Q(t+1) = J\overline{Q} + \overline{K}Q$.
- Диаграмма: найти фронты C, смотреть входы **до** фронта, Q меняется только на фронте.

Счётчики

- n триггеров → модуль 2^n. Каждый триггер делит f на 2.
- Асинхронный суммирующий на D: $D = \overline{Q}$, такт следующего = $\overline{Q}$ предыдущего.
- Минус: задержки складываются (n·t), ложные состояния.
- Модуль 10: 4 триггера + NAND(Q3, Q1) на сброс. Синхронный: общий CLK, $J_i = K_i$ = произведение младших Q.